

MACHINE LEARNING, TOLD SIMPLY · PRACTICE SETS

The Complete Set · Modules 1–5

The Machine Learning Practice Set

Sixty exam-style problems — conceptual, scenario-based and numerical — on everything from the definition of learning to pruned decision trees, each followed immediately by a plan of attack, the answers at a glance and a full step-by-step solution.

Machine Learning · AIML ZG565 / DSE ZG565 · BITS Pilani WILP

Practice for Contact Sessions 1–8 (Modules 1–5)

Practice set prepared by Prof. Saurabh

Contents

Five modules, sixty problems, one method. Problem 3.7 means Module 3, Problem 7. Every solution states what the question is testing, how to plan the attempt, the answers at a glance, then the working one line at a time — and where marks are usually lost.

How to Use This Set		Sixty problems, five modules, one method — and how to get the most out of them.
Formula Cards, Modules 1–5		Every formula and definition the sixty problems use, one card per module. Print these pages and keep them beside you while you work.
Module 1	Introduction to Machine Learning	Contact Session 1 Definitions of learning, the families, batch and online learning, instance- vs model-based, the workflow, generalisation.
Part A	What Learning Is	Problems 1.1–1.4: the definition of learning, why rules fail, the families, and when ML is the wrong tool.
Part B	Measuring, Feeding and Storing	Problems 1.5–1.7: performance measures with numbers, how data is fed to a learner, and instance-based versus model-based prediction.
Part C	Workflow and Generalisation	Problems 1.8–1.10: putting the workflow in order, judging models honestly, and reading a dataset with the right vocabulary.
Module 2	ML Workflow and Data Preprocessing	Contact Session 2 Attribute types, data quality, outliers (IQR, 3-sigma), scaling, sampling and partitioning, binning, encoding, feature engineering, hypotheses.
Part A	Knowing Your Attributes	Problems 2.1–2.2: attribute types and the operations that make sense.
Part B	Cleaning, Scaling, Sampling, Encoding	Problems 2.3–2.7: the numerical core of Module 2.
Part C	Quality, Features and Hypotheses	Problems 2.8–2.10: diagnosing dirty data, engineering features, and reasoning about hypotheses.
Module 3	Linear Models for Regression	Contact Sessions 3 & 4 Closed form and the normal equation, gradient descent by hand, diagnosing runs, metrics, polynomial and basis functions, bias–variance, ridge and lasso.
Part A	Solving It: Closed Form and the Cost	Problems 3.1–3.3: the two formulas, the normal equation, and the shape of J .
Part B	Walking Down: Gradient Descent	Problems 3.4–3.8: one-feature and two-feature updates, regularised updates, recovering α , diagnosing logs, and stochastic descent.
Part C	Judging and Bending the Line	Problems 3.9–3.11: evaluation metrics, polynomial regression, and choosing complexity.

Part D Taming and Understanding

Problems 3.12–3.13: regularisation with numbers, and the derivations and comparisons the exam asks for.

Module 4 Logistic Regression and Classifier Evaluation

Contact Sessions 5 & 6
Sigmoid, boundaries and odds, cross-entropy and its gradient, regularised updates, confusion-matrix metrics, ROC and AUC, imbalance, multi-class.

Part A The Model and How It Learns

Problems 4.1–4.5: sigmoids and boundaries, gradient descent by hand, regularisation, log-loss, and the derivation.

Part B Counting Right and Wrong

Problems 4.6–4.8: confusion-matrix metrics from counts, from a scenario, and from raw predictions.

Part C Every Threshold at Once: ROC and AUC

Problems 4.9–4.11: constructing the curve, computing the area, comparing classifiers, choosing a threshold.

Part D Beyond Two Balanced Classes

Problems 4.12–4.13: imbalance and multi-class strategies, and curved boundaries.

Module 5 Decision Trees

Contact Sessions 7 & 8
Impurity measures, information gain and ID3, Gini binary splits, continuous thresholds, gain ratio, pre- and post-pruning, MDL, overfitting.

Part A Growing the Tree

Problems 5.1–5.3: impurity, information gain and ID3 on the loan data, then continuing, predicting and evaluating.

Part B Other Ways to Split

Problems 5.4–5.6: Gini with binary groupings, continuous thresholds, and gain ratio.

Part C Keeping the Tree Honest

Problems 5.7–5.10: pre-pruning, post-pruning, MDL, and reading an overfitting table.

Part D Structure and Judgement

Problems 5.11–5.14: rules and algorithms, test conditions, multi-class impurity, and when a tree is the right tool.

BEFORE YOU START

How to Use This Set

Sixty problems, five modules, one method — and how to get the most out of them.

The five practice sets of *Machine Learning, Told Simply* in one volume, module by module, covering Contact Sessions 1 to 7 — the material reviewed in Session 8. Every solution comes *immediately* after its question.

Problem 3.7 means Module 3, Problem 7; inside a solution each step is tagged with the part it answers —

STEP 2 (B) — so you can check one part without reading the rest.

What is inside

Module	Topic	Sessions	Problems	Conceptual	Scenario	Numerical	Marks
1	Introduction to Machine Learning	Session 1	10	3	4	3	60
2	ML Workflow and Data Preprocessing	Session 2	10	2	3	5	65
3	Linear Models for Regression	Sessions 3 & 4	13	1	2	10	88
4	Logistic Regression and Classifier Evaluation	Sessions 5 & 6	13	2	3	8	90
5	Decision Trees	Sessions 7 & 8	14	2	3	9	95
	All modules	Sessions 1–8	60	10	15	35	398

Marks are indicative — they say how much working a question deserves, and roughly how many minutes to give it in an exam.

How each problem is laid out

Navy box — the question exactly as an exam would set it, with its type and marks.

What it tests — the idea the examiner is checking, so you know what the marks are for.

How to attempt — the plan, written *before* any working. Read it only after you have tried the question yourself.

Answers at a glance — the final answers in one line, so you can self-mark in ten seconds.

Steps — the working, one computation per line, each tagged with the part it answers; boxed green lines are the answers to write down.

Where marks are lost — the usual slips on this kind of question.

A way to work through it

Pass 1 — read and plan.

Cover the solution, read the question, write your own three-line plan, then compare with the gold box. Do a whole module this way before computing anything.

Pass 2 — work the numericals by hand.

Calculator, pen, paper, no peeking; check only against *Answers at a glance*. If you are off, the first line that differs from yours is the lesson.

Pass 3 — simulate the exam. One problem from each module, one minute per mark, full written answers; then mark yourself with the *Where marks are lost* notes.

Before the exam. Re-read the five **Formula Cards** and every *Answers at a glance* line. Any glance line that looks unfamiliar is your revision list.

REVISION SHEET

Formula Cards, Modules 1–5

Every formula and definition the sixty problems use, one card per module. Print these pages and keep them beside you while you work.

Module 1 in five lines

Learning: a program learns from experience E w.r.t. task T and performance measure P if its performance at T , measured by P , improves with E .

Families: supervised — pairs (x, y) , predict y (classification if y is a category, regression if a number); unsupervised — only x , find structure (clustering, dimensionality reduction); reinforcement — an agent, actions, delayed rewards; semi-supervised — few labels, many unlabelled.

By data use: batch (all data per update), mini-batch (a chunk), online (one instance). **By what is kept:** instance-based (store examples, compare) vs model-based (learn a rule).

Workflow: should I use ML? → gather → preprocess/clean/visualise → choose model, loss, regularisation → optimise → tune hyper-parameters → analyse mistakes, iterate.

Use ML when no human expertise exists, humans cannot explain theirs, the solution must be customised, or the relationships hide in large data — and there is a pattern, no formula, and enough data. **Generalisation** — performance on unseen data — is the goal; measure it on a held-out test set.

Module 2 formula card

Attribute types: nominal ($=, \neq$; permutation) $\subset$ ordinal ($<, >$; monotone map) $\subset$ interval ($+, -$; $v' = av + b$) $\subset$ ratio ($\times, \div$; $v' = av$). Separately: discrete vs continuous.

IQR fence: sort; Q_2 = median; Q_1 = median of lower half, Q_3 = median of upper half; $\text{IQR} = Q_3 - Q_1$; outlier if $x < Q_1 - 1.5 \text{ IQR}$ or $x > Q_3 + 1.5 \text{ IQR}$.

3-sigma: $\mu = \frac{1}{n} \sum x_i$, $\sigma = \sqrt{\frac{1}{n} \sum (x_i - \mu)^2}$ (or $n - 1$); $z = (x - \mu)/\sigma$; outlier if $|z| > 3$ (assumes roughly normal; 99.7% inside).

Scaling: min-max $v' = \frac{v - \min}{\max - \min}(\text{new_max} - \text{new_min}) + \text{new_min}$; z-score $v' = \frac{v - \mu}{\sigma}$; decimal $v' = v/10^j$, smallest j with $\max |v'| < 1$. Fit on training data only; do not scale the target.

Binning: equal-width $W = (B - A)/N$; equal-depth $\approx n/N$ per bin. **Encoding:** one-hot (one 0/1 column per category; nominal) vs label (integer codes; ordinal).

Sampling: representative $\iff$ same properties of interest as the whole; stratified keeps class proportions; sampling noise (too small) vs sampling bias (flawed process). Split: train (fit) / validation (choose) / test (once).

Inductive learning hypothesis: a hypothesis that approximates the target function well over a sufficiently large training set will also approximate it well on unobserved examples.

Module 3 formula card

Model: $h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \dots + \theta_d x_d = \boldsymbol{\theta}^T \mathbf{x}$ ($x_0 = 1$). **Cost:** $J = \frac{1}{2n} \sum_i (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2$.

Closed form (1 input): $\theta_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$, $\theta_0 = \bar{y} - \theta_1 \bar{x}$. **Normal equation:** $\boldsymbol{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$;

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

Gradient descent: $\theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum_i (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$ (all j at once); matrix $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\alpha}{n} \mathbf{X}^T (\mathbf{X} \boldsymbol{\theta} - \mathbf{y})$; **SGD:** one example, no $\frac{1}{n}$.

Ridge: $J + \frac{\lambda}{2} \sum_{j \geq 1} \theta_j^2$, $\theta_j \leftarrow \theta_j (1 - \alpha \lambda) - \alpha \text{grad}_j$. **Lasso:** $J + \lambda \sum_{j \geq 1} |\theta_j|$, $\theta_j \leftarrow \theta_j - \alpha \text{grad}_j - \alpha \lambda \text{sign}(\theta_j)$. Never penalise θ_0 .

Metrics: $\text{MAE} = \frac{1}{N} \sum |y - \hat{y}|$; $\text{MSE} = \frac{1}{N} \sum (y - \hat{y})^2$; $\text{RMSE} = \sqrt{\text{MSE}}$; $R^2 = 1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}$.

Diagnosis: J rising $\Rightarrow \alpha$ too big; crawling $\Rightarrow$ too small; zig-zag $\Rightarrow$ scale features. Train low / test high $\Rightarrow$ overfit (more data, simpler, regularise); both high $\Rightarrow$ underfit.

Module 4 formula card

Model: $z = \boldsymbol{\theta}^T \mathbf{x}$, $h_{\theta}(\mathbf{x}) = g(z) = \frac{1}{1 + e^{-z}} = P(y = 1 | \mathbf{x})$; predict 1 iff $z \geq 0$; boundary $z = 0$. $g(0) = 0.5$; $1 - g(z) = g(-z)$; $g' = g(1 - g)$; $\ln \frac{p}{1-p} = z$; odds ratio per unit of x_j : e^{θ_j} .

Cost: $J = -\frac{1}{m} \sum_i [y^{(i)} \log h^{(i)} + (1 - y^{(i)}) \log(1 - h^{(i)})]$. **Gradient:** $\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_i (h^{(i)} - y^{(i)}) x_j^{(i)}$. **Update:** $\theta_j \leftarrow \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$. **L2:** add $\frac{\lambda}{2m} \sum_{j \geq 1} \theta_j^2$, update $\theta_j \leftarrow \theta_j (1 - \frac{\alpha \lambda}{m}) - \alpha \text{grad}_j$ ($j \geq 1$). scikit-learn: $C = 1/\lambda$.

Metrics: $\text{Acc} = \frac{TP+TN}{\text{all}}$; $\text{Prec} = \frac{TP}{TP+FP}$; $\text{Rec} = \text{TPR} = \frac{TP}{TP+FN}$; $\text{Spec} = \text{TNR} = \frac{TN}{TN+FP}$; $\text{FPR} = \frac{FP}{FP+TN}$; $\text{FNR} = \frac{FN}{FN+TP}$; $F_1 = \frac{2PR}{P+R}$.

ROC: sort by score; at each distinct score (ties together) count TP, FP; plot (FPR, TPR); (0, 0) all-negative, (1, 1) all-positive, top-left ideal, diagonal random. **AUC** = $\sum_i (x_{i+1} - x_i) \frac{y_i + y_{i+1}}{2}$.

Multi-class: one-vs-all: K classifiers, $\hat{y} = \arg \max_i h^{(i)}$; one-vs-one: $K(K-1)/2$ classifiers, majority vote.

Imbalance: SMOTE / resample training only; use recall, precision, F_1 , AUC.

Module 5 formula card

Impurity: Entropy = $-\sum_i p_i \log_2 p_i$ (max $\log_2 C$); Gini = $1 - \sum_i p_i^2$ (max $1 - 1/C$); Error = $1 - \max_i p_i$. Handy: $H(1:1) = 1$, $H(1:2) = 0.918$, $H(1:3) = 0.811$, $H(2:3) = 0.971$, $H(1:4) = 0.722$, $H(1:5) = 0.650$.

Gain: $\text{Gain}(S, A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$. **Gini split:** $\sum_j \frac{N_j}{N} \text{Gini}(j)$, lower is better; nominal with k values has $2^{k-1} - 1$ binary groupings. **SplitInfo** = $-\sum_i \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$; **GainRatio** = Gain / SplitInfo.

Continuous: sort; candidate thresholds at midpoints (all, or where the class changes; plus one below the minimum); pick the lowest weighted impurity.

ID3: pure $\rightarrow$ leaf; no attributes / empty branch $\rightarrow$ majority class; else best-gain attribute, one branch per value, recurse without that attribute. **Stop:** pure, identical attributes, no records, or depth / min-samples / min-gain limits.

Pruning: pre — stop if best gain $\leq$ threshold or samples $<$ minimum (label by majority); post — grow fully, bottom-up replace a subtree by its majority leaf if prune-set error does not rise. **MDL:** minimise bits(tree) + bits(errors); node $\log_2 \# \text{attributes}$, leaf $\lceil \log_2 \# \text{classes} \rceil$, error $\log_2 n$.

MODULE 1 · CONTACT SESSION 1

Introduction to Machine Learning

Definitions of learning, the families, batch and online learning, instance- vs model-based, the workflow, generalisation.

Ten problems in three parts: what learning is, how it is measured and fed, and the workflow that turns data into a model that generalises. Most are conceptual or scenario-based — the exam asks you to *explain*, so practise writing crisp definitions and one-line justifications. The Module 1 formula card is in **Formula Cards** at the front.

The problem map

#	Title	Type	Topic on the slides	Marks
1.1	Three learning problems, three triples	Conceptual	Mitchell's definition; $\langle T, P, E \rangle$	6
1.2	Why the rule-book fails	Scenario	Traditional programming vs ML; spam filtering	5
1.3	Sort eight tasks into families	Scenario	Supervised / unsupervised / reinforcement / semi-supervised; classification vs regression	8
1.4	Should we use machine learning here?	Scenario	When ML is (not) appropriate; the three questions	6
1.5	Putting a number on P	Numerical	Performance measures; training vs test	6
1.6	Batch, mini-batch, online — count the updates	Numerical	Types by how training data is used	5
1.7	Two ways to quote a price	Numerical	Instance-based vs model-based	6
1.8	The workflow, scrambled	Scenario	The seven-step ML workflow; the car-price walk-through	6
1.9	Three models, one honest verdict	Conceptual	Generalisation; under/overfitting; issues in ML	6
1.10	Reading a dataset like a data scientist	Conceptual	Features, labels, instances, parameters vs hyper-parameters	6

MODULE 1 · PART A

What Learning Is

Problems 1.1–1.4: the definition of learning, why rules fail, the families, and when ML is the wrong tool.

Problem 1.1 · Three learning problems, three triples

CONCEPTUAL · 6 MARKS

State Tom Mitchell's definition of a well-posed learning problem. Then, for each of the following, write the task T , the performance measure P and the experience E , and name the family of learning it belongs to:

- A used-car marketplace wants to quote a fair selling price for any car listed on its site.
- A bank wants to flag credit-card transactions that are likely to be fraudulent.
- A program learns to play chess by playing against a copy of itself.

WHAT IT TESTS Whether you can reproduce the definition *and* apply it: a measurable P and a concrete E for each case, plus recognising regression vs classification vs reinforcement from the description.

HOW TO ATTEMPT

- Write the definition verbatim — it carries its own marks.
- For each case ask: what does the program *do* (T)? what *number* shows it is doing well (P)? what *data or practice* does it learn from (E)?
- Decide the family from the kind of feedback: labelled numbers $\rightarrow$ regression; labelled categories $\rightarrow$ classification; rewards from play $\rightarrow$ reinforcement.

ANSWERS AT A GLANCE (a) regression — P = average absolute pricing error on unseen cars; (b) classification — P = % of transactions correctly flagged (better: recall on frauds); (c) reinforcement learning — P = % of games won.

STEP 1 DEFINITION

A computer program is said to **learn** from experience E with respect to some task T and performance measure P if its performance at T , as measured by P , improves with experience E . A well-posed learning problem is the triple $\langle T, P, E \rangle$.

STEP 2 (A) CAR PRICES

T : predict the selling price of a car from its attributes (brand, year, engine size, kilometres, cab or not).

P : the average absolute difference (in rupees) between predicted and actual sale price, measured on listings the model has not seen.

E : the database of past listings together with the prices they actually sold for.

Family: **supervised learning — regression** (the label is a real number).

STEP 3 (B) FRAUD

T : classify each transaction as fraudulent or legitimate.

P : the percentage of transactions correctly classified on held-out data — and, because frauds are rare, preferably the percentage of true frauds that are caught (recall).

E : historical transactions labelled fraud / not-fraud by investigators.

Family: **supervised learning — binary classification**.

STEP 4 (C) CHESS

T : play chess (choose a move in any position).

P : percentage of games won against opponents (or a rating).

E : games played against itself — sequences of moves with a win/loss reward at the end.

Family: **reinforcement learning** — no labelled "correct move" is given; feedback is a delayed reward from the outcome of the game.

Definition as above; (a) regression, (b) classification, (c) reinforcement learning, with T , P , E as listed.

WHERE MARKS ARE LOST Writing P as "accuracy" without saying *on what data*; giving E as "data" instead of naming the dataset and its labels; calling (c) supervised because "it learns from games".

Problem 1.2 · Why the rule-book fails

SCENARIO · 5 MARKS

An e-commerce company blocks fake product reviews with a hand-written list of 340 rules ("contains 'best product ever' and fewer than 12 words → fake", ...). The list has grown for three years; two engineers now spend most of their time patching it, and it still misses about half the fakes.

- Explain, with a diagram, how the traditional-programming approach differs from the machine-learning approach in terms of what goes *in* and what comes *out*.
- Give three reasons the rule-book keeps failing and say how a learning approach addresses each.

WHAT IT TESTS The "rules in, answers out" vs "data + answers in, rules out" picture, and the spam-filter argument from the slides transferred to a new domain.

HOW TO ATTEMPT

- Draw the two block diagrams side by side and label the arrows.
- For (b), reuse the spam-filter reasons: rules are many and complex, hard to maintain, and the adversary adapts — then say what learning does instead.

ANSWERS AT A GLANCE (a) Traditional: data + rules → computer → answers. ML: data + answers → computer → rules (the model). (b) Too many fuzzy rules; costly maintenance; fakers change tactics — learning discovers the patterns from labelled examples, re-learns cheaply on new data, and adapts.

STEP 1 (A)

Traditional programming: inputs are the *data* (a review) and the *rules* written by a human; the computer applies the rules; the output is the *answer* (fake / genuine).

Machine learning: inputs are the *data* (many reviews) together with the *answers* (labels: fake / genuine); the computer runs a learning algorithm; the output is the *rules* — a model — which is then used on new reviews.

Sketch: two boxes labelled "Computer"; left box with arrows in from "Data" and "Rules" and out to "Answers"; right box with arrows in from "Data" and "Answers" and out to "Rules (model)".

STEP 2 (B) REASON 1 — COMPLEXITY

What makes a review fake is a fuzzy combination of many weak signals (wording, length, timing, reviewer history). Capturing it needs hundreds of interacting rules that contradict each other. A learner finds the combination automatically from labelled examples, so the program is much shorter.

STEP 3 (B) REASON 2 — MAINTENANCE

Every new pattern means a human edits the list and re-tests everything. With learning, new labelled reviews are simply added to the training set and the model is retrained — no rule editing.

STEP 4 (B) REASON 3 — ADAPTATION

Fakers read the rules' behaviour and change tactics ("b3st product"), exactly as spammers write "4U". Hand rules lag behind; a learner retrained on recent examples adapts, and typically ends up more accurate.

Traditional: rules are the input; ML: rules are the output. Learning wins because the rule is too fuzzy to write, too costly to maintain, and keeps changing.

WHERE MARKS ARE LOST Diagram with arrows unlabelled; saying "ML is more accurate" without a reason; forgetting that ML still needs *labelled* examples (someone must mark reviews as fake).

Problem 1.3 · Sort eight tasks into families

SCENARIO · 8 MARKS

For each task below, name the family (supervised / unsupervised / reinforcement / semi-supervised). For supervised tasks also say whether it is classification or regression, and for every task say *what the feedback is*.

- Predict how many days a listed car will take to sell.
- Group 40,000 customers by browsing behaviour, with no labels, to design marketing segments.
- Decide whether an incoming email is spam.
- A warehouse robot learns to park its trolley by being scored on each attempt.
- Predict tomorrow's electricity demand in megawatts.
- Google Photos clusters faces; you name one photo "Amma" and it labels the whole cluster.
- Compress a 300-column survey into 5 summary columns for plotting.
- Predict a loan applicant's grade (A / B / C / D).

WHAT IT TESTS The one question behind the families — what kind of feedback? — and the one question behind classification vs regression — is *y* a category or a number?

HOW TO ATTEMPT

1. Ask: is there a label y ? If yes, is it a number (regression) or a category (classification)?
2. If no label: is the goal groups/compression (unsupervised) or is there a reward from actions (reinforcement)?
3. A few labels plus many unlabelled records → semi-supervised.

ANSWERS AT A GLANCE (a) supervised regression; (b) unsupervised clustering; (c) supervised classification; (d) reinforcement; (e) supervised regression; (f) semi-supervised; (g) unsupervised dimensionality reduction; (h) supervised (multi-class) classification.

	Task	Family	Feedback / label
a	Days to sell	Supervised — regression	Past listings with their actual days-to-sell (a number)
b	Customer segments	Unsupervised — clustering	None; the groups are discovered from similarity
c	Spam or not	Supervised — classification	Emails labelled spam / ham
d	Parking robot	Reinforcement	A score (reward) after each attempt; no correct action is given
e	Electricity demand	Supervised — regression	Historical demand values (numbers)
f	Face labelling	Semi-supervised	Unlabelled clustering plus one human-given label propagated to the cluster
g	300 → 5 columns	Unsupervised — dimensionality reduction (e.g. PCA)	None; keep as much variation as possible in fewer columns
h	Loan grade A–D	Supervised — multi-class classification	Past applicants with their grades (categories)

Four supervised (a, e regression; c, h classification), two unsupervised (b, g), one reinforcement (d), one semi-supervised (f).

WHERE MARKS ARE LOST Calling (h) regression because grades "are ordered"; calling (d) supervised because "it gets feedback" — the feedback is a reward, not a correct answer; forgetting that (g) is unsupervised even though it is not clustering.

Problem 1.4 · Should we use machine learning here?

SCENARIO · 6 MARKS

The slides say ML is used when human expertise does not exist, when humans cannot explain their expertise, when models must be customised, or when relationships are hidden in large data — and that learning is not always useful. For each request below, decide **ML or not** and justify using the three screening questions (*is there a pattern? can it be solved analytically? do we have data?*):

- Compute the GST payable on each invoice.
- Recognise which of 200 employees is at the office door from a camera image.
- Recommend a different set of dishes to each of 2 million food-app users.
- Predict the exact winning lottery number next week from the last ten years of draws.
- Convert kilometres per litre to litres per 100 km.
- Identify early signs of kidney disease from 50,000 patient records with 300 lab measurements each.

WHAT IT TESTS Knowing that ML is a tool for *unknown* rules, and being able to spot when a rule is known (formula), when there is no pattern (randomness), and when there is no data.

HOW TO ATTEMPT

- For each case, answer the three questions in one line each.
- Then state the verdict and, if ML, which of the four "use when" situations applies.

ANSWERS AT A GLANCE ML for (b), (c), (f); not for (a), (d), (e).

STEP 1 (A)

Pattern: yes but it is a known rule. Analytically solvable: yes — $\text{GST} = \text{rate} \times \text{taxable value}$. Data: irrelevant. **Not ML** — like payroll on the slide, the rule is exact and cheap to code.

STEP 2 (B)

Pattern: yes. Analytical: no — nobody can write down what makes a face this employee's ("humans cannot explain their expertise"). Data: yes, a few labelled photos per employee. **ML** (classification with 200 classes).

STEP 3 (C)

Pattern: yes, tastes are predictable from history. Analytical: no. Data: yes, orders and ratings. **ML** — "models must be customised": one rulebook per user is impossible by hand.

STEP 4 (D)

Pattern: **no** — draws are designed to be random; past draws carry no information about the next. **Not ML**, however much data there is.

STEP 5 (E)

Analytical: yes — $L/100 \text{ km} = 100/\text{kmpL}$. **Not ML**.

STEP 6 (F)

Pattern: yes. Analytical: no — 300 interacting measurements are beyond a hand rule. Data: 50,000 labelled records. **ML** — "relationships hidden in large data" (medical diagnosis on the slide).

(a) no — formula; (b) yes — inexplicable expertise; (c) yes — customisation; (d) no — no pattern;
(e) no — formula; (f) yes — large hidden relationships.

WHERE MARKS ARE LOST Recommending ML for (d) because "there is a lot of data" — data without a pattern is noise; skipping the three questions and just guessing.

MODULE 1 · PART B

Measuring, Feeding and Storing

Problems 1.5–1.7: performance measures with numbers, how data is fed to a learner, and instance-based versus model-based prediction.

Problem 1.5 · Putting a number on P

NUMERICAL · 6 MARKS

Two models are evaluated on data they did not see during training.

Model A (spam classifier) — actual vs predicted labels for ten test emails (Y = spam, N = not spam):

Email	1	2	3	4	5	6	7	8	9	10
Actual	Y	N	Y	Y	N	N	Y	N	Y	Y
Predicted	Y	N	N	Y	N	Y	Y	N	Y	N

Model B (car-price regressor, prices in lakh ₹) — five test cars:

Car	1	2	3	4	5
Actual	3.2	4.5	2.8	5.0	3.9
Predicted	3.5	4.0	3.0	5.6	3.7

- Propose a suitable performance measure P for each model and compute it.
- Model A scored 100% on its 2,000 training emails. Which number — 100% or your answer to (a) — should be reported, and why?

WHAT IT TESTS Turning "performance measure" into arithmetic (accuracy for classification, mean absolute error for regression) and understanding that only performance on unseen data measures learning.

HOW TO ATTEMPT

- Classification: count correct predictions $\div$ total.
- Regression: compute each $|\text{actual} - \text{predicted}|$, then average.
- (b): the training score can be perfect by memorising; the test score estimates generalisation.

ANSWERS AT A GLANCE (a) Model A: accuracy = $7/10 = 70\%$. Model B: MAE = 0.36 lakh. (b) Report 70%.

STEP 1 (A) MODEL A

Compare each pair: emails 1, 2, 4, 5, 7, 8, 9 match (7 correct); emails 3, 6, 10 do not (3 wrong).

$$\text{Accuracy} = \frac{\text{correct}}{\text{total}} = \frac{7}{10} = 0.70 = 70\% \text{ (error rate } 30\%).$$

STEP 2 (A) MODEL B

Absolute errors: $|3.2 - 3.5| = 0.3$; $|4.5 - 4.0| = 0.5$; $|2.8 - 3.0| = 0.2$; $|5.0 - 5.6| = 0.6$; $|3.9 - 3.7| = 0.2$.

$$\text{Sum} = 0.3 + 0.5 + 0.2 + 0.6 + 0.2 = 1.8.$$

Mean absolute error = $1.8/5 = 0.36$ lakh ₹ (about ₹36,000 per car). (Mean squared error would be $(0.09 + 0.25 + 0.04 + 0.36 + 0.04)/5 = 0.156$; either is acceptable if named.)

STEP 3 (B)

Report **70%**. The 100% was measured on the emails the model was trained on; a model can achieve that by memorising, which says nothing about new emails. P must be measured on unseen data because the goal is **generalisation**. The gap (100% vs 70%) is itself a warning sign of overfitting.

A: accuracy 70%. B: MAE 0.36 lakh. Report the test figure, never the training figure.

WHERE MARKS ARE LOST Forgetting to name the measure ("70" without "accuracy"); signs in the regression errors (use absolute values); arguing that 100% is better "because it is higher".

Problem 1.6 · Batch, mini-batch, online — count the updates NUMERICAL · 5 MARKS

A training set has 12,000 records. Three learners are trained for 5 epochs each: learner X updates its parameters once per pass over all the data; learner Y uses mini-batches of 200 records; learner Z updates after every single record.

- Name the three types of learning and compute the number of parameter updates each performs in total.
- New records now arrive continuously, about 3 per second, and must influence the model within minutes. Which learner would you deploy and why? Name one drawback of it.

WHAT IT TESTS The three "how is the training data used" types, the meaning of an epoch, and matching online learning to streaming data.

HOW TO ATTEMPT

- Updates per epoch: batch 1; mini-batch $n/\text{batch size}$; online n .
- Multiply by the number of epochs.
- (b): streaming $\rightarrow$ online (incremental); drawback: noisy, sensitive to bad data.

ANSWERS AT A GLANCE X batch: 5 updates. Y mini-batch: 300 updates. Z online: 60,000 updates. Deploy Z (online).

STEP 1 (A) NAMES

X = **batch** learning (all available data per update); Y = **mini-batch** learning (a subset per update); Z = **online / incremental** learning (one instance per update).

STEP 2 (A) COUNTS

X: 1 update per epoch $\times$ 5 epochs = 5 updates.

Y: $12,000/200 = 60$ updates per epoch $\times$ 5 = 300 updates.

Z: 12,000 updates per epoch $\times$ 5 = 60,000 updates.

STEP 3 (B)

Deploy **Z, online learning**: each arriving record can update the model immediately, so new information is absorbed within seconds, and the full dataset never has to be reloaded or fit in memory. Drawback: each update is based on one record, so the model is **noisy** and can be pushed off course by a burst of bad

or unrepresentative records; the learning rate must be kept small and the input monitored. (Mini-batch is the usual compromise if a few minutes of delay is acceptable.)

5, 300 and 60,000 updates; online learning for the stream, at the cost of noisier, less stable updates.

WHERE MARKS ARE LOST Confusing "epoch" with "update"; forgetting to multiply by 5; recommending batch for streaming because it is "more accurate".

Problem 1.7 · Two ways to quote a price

NUMERICAL · 6 MARKS

A marketplace holds eight sold hatchbacks (kilometres driven in thousands, price in lakh ₹):

km (000s)	20	35	48	52	70	88	95	120
price	6.4	5.9	5.1	5.0	4.2	3.6	3.3	2.6

A new car with 60,000 km must be priced.

- Quote a price using an **instance-based** approach: average the prices of the three cars nearest in kilometres.
- Quote a price using a **model-based** approach: the learnt rule $\text{price} = 6.5 - 0.035 \text{ km}$.
- Explain the difference between the two approaches and give one advantage of each. Which needs the eight records at prediction time?

WHAT IT TESTS The instance-based vs model-based distinction made concrete: one stores examples and compares, the other stores a rule.

HOW TO ATTEMPT

- Compute $|\text{km} - 60|$ for each car, pick the three smallest, average their prices.
- Substitute 60 into the rule.
- Contrast: memorise-and-compare vs learn-a-pattern.

ANSWERS AT A GLANCE (a) ₹4.77 lakh; (b) ₹4.40 lakh; (c) instance-based needs the records at prediction time, model-based does not.

STEP 1 (A) DISTANCES

$|20 - 60| = 40$, $|35 - 60| = 25$, $|48 - 60| = 12$, $|52 - 60| = 8$, $|70 - 60| = 10$, $|88 - 60| = 28$, $|95 - 60| = 35$, $|120 - 60| = 60$.

Three nearest: 52 (8 away), 70 (10 away), 48 (12 away).

Average price = $\frac{5.0 + 4.2 + 5.1}{3} = \frac{14.3}{3} = 4.77$ lakh.

STEP 2 (B)

price = $6.5 - 0.035 \times 60 = 6.5 - 2.1 = 4.40$ lakh.

STEP 3 (C)

Instance-based learning compares the new point with stored known points and copies their answers (here: 3-nearest-neighbour averaging). No training step; all work happens at prediction time; it **needs the eight records** available every time. Advantage: no assumption about the shape of the relationship — it follows the data locally and adapts as records are added.

Model-based learning detects the pattern in the training data and builds a predictive model (the line). Training does the work once; prediction is a formula and the records can be discarded. Advantage: instant, compact, and can extrapolate a little; the coefficient (-0.035 lakh per 1,000 km) is interpretable.

3-NN quote ₹4.77 lakh; formula quote ₹4.40 lakh. Instance-based keeps the data; model-based keeps the rule.

WHERE MARKS ARE LOST Picking the three nearest by price instead of kilometres; averaging all eight; describing the difference without saying which one needs the data at prediction time.

MODULE 1 · PART C

Workflow and Generalisation

Problems 1.8–1.10: putting the workflow in order, judging models honestly, and reading a dataset with the right vocabulary.

Problem 1.8 · The workflow, scrambled

SCENARIO · 6 MARKS

The steps of a typical ML project are listed out of order:

(i) Search over hyper-parameters (ii) Gather and organise data (iii) Analyse the model's mistakes and iterate (iv) Decide whether ML is appropriate at all (v) Choose a model, a loss and any regularisation (vi) Preprocess, clean and visualise the data (vii) Optimise the model's parameters

- Put the steps in the correct order and, for the car-price-from-mileage project, say in one line what each step does.
- Three things went wrong in a real project. Name the step each failure belongs to and what should have been done: (1) the model was trained on listings from January only, when mostly hatchbacks sell; (2) the team compared candidate models by their error on the training data; (3) prices were stored as text ("3.5 lakh", "350000", "3,50,000").

WHAT IT TESTS The seven-step workflow, the car-price walk-through from the slides, and diagnosing which step a failure comes from.

HOW TO ATTEMPT

- Order: decide → gather → preprocess → choose → optimise → tune → analyse.
- Attach one concrete action per step for the example.
- For (b), map each failure to the step and state the fix.

ANSWERS AT A GLANCE (a) iv, ii, vi, v, vii, i, iii. (b) (1) step ii/vi — unrepresentative sample, gather across the year or stratify; (2) step i/iii — use a validation or test set; (3) step vi — clean and standardise to one numeric unit.

STEP 1 (A)

Order	Step	For car price from mileage
1.1	(iv) Should I use ML?	Is there a pattern (yes, price falls with km), a formula (no), data (past sales — yes)? Proceed.
1.2	(ii) Gather and organise	Collect past listings with mileage and final sale price; one row per car.
1.3	(vi) Preprocess, clean, visualise	Fix units, remove duplicates, split into training and test sets, scatter-plot price vs mileage.
1.4	(v) Choose model, loss, regularisation	Linear regression $\text{price} = \theta_0 + \theta_1 \text{ km}$; squared-error objective.
1.5	(vii) Optimise	Find θ_0, θ_1 minimising the squared error on the training set.
1.6	(i) Hyper-parameter search	Choose learning rate / polynomial degree / regularisation strength on a validation set.
1.7	(iii) Analyse mistakes, iterate	Evaluate on the sealed test set; look at the worst errors; go back to step 5 or step 3.

STEP 2 (B)(1)

A January-only file is a **non-representative training set** — a failure at **gathering / sampling** (steps ii–vi). Fix: gather across the whole year, or sample in a stratified way so every car type and season is represented.

STEP 3 (B)(2)

Comparing on training error confuses memorising with learning — a failure at **hyper-parameter search / evaluation** (steps i and iii). Fix: compare on a validation set; report the final number on a test set used once.

STEP 4 (B)(3)

Mixed text formats for the target are a **preprocessing / cleaning** failure (step vi). Fix: parse every price into one numeric unit (rupees, or lakh) before modelling; check for outliers created by wrong parsing.

Order iv–ii–vi–v–vii–i–iii; failures at sampling, evaluation, and cleaning respectively.

WHERE MARKS ARE LOST Putting "choose a model" before "look at the data"; leaving out the train/test split; not naming the step for each failure.

Problem 1.9 · Three models, one honest verdict

CONCEPTUAL · 6 MARKS

Three price models were built on the same training set and evaluated on the same test set (errors in lakh ₹):

Model	Training error	Test error
M1 (straight line)	0.90	0.95
M2 (curve)	0.40	0.45
M3 (very wiggly curve)	0.02	1.30

- Define generalisation. Which model generalises best?
- Diagnose each model as underfitting, overfitting, or a good fit, giving the evidence from the table.
- Explain, with the "two students" analogy, why M3's training error is misleading, and suggest two ways to improve M3.

WHAT IT TESTS The core idea of the module — performance on unseen data is the goal — and recognising under/overfitting from a training/test table.

HOW TO ATTEMPT

- Generalisation = doing well on data not seen during training; pick the lowest test error.
- Underfit: both errors high. Overfit: training tiny, test large. Good: both low and close.
- Memorising vs understanding; fixes: more data, simpler model, regularisation.

ANSWERS AT A GLANCE (a) M2. (b) M1 underfits, M2 good fit, M3 overfits. (c) memorising the question bank; more data, simpler/penalised model.

STEP 1 (A)

Generalisation is a model's ability to perform well on previously unseen data — the test set. Ranking by test error: $M2 (0.45) < M1 (0.95) < M3 (1.30)$. **M2 generalises best.**

STEP 2 (B)

M1 — underfitting: training error 0.90 and test error 0.95 are both high and close; the model is too simple to capture the pattern even on the data it trained on.

M2 — good fit: both errors are low and close (0.40 vs 0.45); it captures the pattern and transfers to new data.

M3 — overfitting: training error is nearly zero (0.02) but test error is the worst (1.30) — a huge gap; it has fitted the noise of the training set.

STEP 3 (C)

Two students prepare from the same question bank. One memorises every answer, the other learns the methods. On the bank itself both score full marks — M3's 0.02 is the memoriser's score. On the real exam (new questions = the test set) only the second does well. Training error therefore cannot tell memorising from understanding; only test error can.

Improving M3: (1) **more training data** — with many more cars, a flexible model cannot bend to every noise point; (2) **reduce complexity** — a less wiggly curve, or a penalty on complexity (regularisation); also stop training early when validation error starts rising.

M2 is best; M1 underfits, M3 overfits; cure M3 with more data or a simpler / regularised model.

WHERE MARKS ARE LOST Choosing M3 because its training error is lowest; diagnosing without quoting the numbers; giving only one remedy.

Problem 1.10 · Reading a dataset like a data scientist

CONCEPTUAL · 6 MARKS

A recruiter's table:

ID	CGPA	Communication	Aptitude	Programming	Job offered?
1	9.1	Average	Good	Excellent	Yes
2	8.4	Good	Good	Good	Yes
3	8.3	Poor	Average	Average	No
4	7.1	Average	Good	Average	No
5	8.2	Good	Excellent	Excellent	No

- Identify the features (attributes), the label (target), the number of instances, and the feature dimension. Which column should *not* be used as a feature, and why?
- Is the learning task classification or regression? What would change if the last column were "starting salary in lakh"?
- A learner fits $\hat{y} = \theta_0 + \theta_1 \text{CGPA} + \dots$ using a learning rate $\alpha = 0.1$. Which of θ_1 and α is a parameter and which is a hyper-parameter? Who sets each?
- Give three other names for "instance" and three for "feature".

WHAT IT TESTS The vocabulary that every later session assumes — and the identifier trap.

HOW TO ATTEMPT

- Features = input columns; label = the column to predict; instances = rows; dimension = number of features.
- Categorical label → classification; numeric → regression.
- Parameters are learnt from data; hyper-parameters are set by us.

ANSWERS AT A GLANCE (a) 4 features, label "Job offered?", 5 instances, dimension 4; drop ID. (b) classification; salary → regression. (c) θ_1 parameter (learnt), α hyper-parameter (set by us). (d) example / record / sample / point; attribute / predictor / input / dimension.

STEP 1 (A)

Features: CGPA, Communication, Aptitude, Programming — four of them, so the feature dimension is $d = 4$. **Label:** Job offered? **Instances:** 5 rows. **ID** is an identifier: it is unique per row and carries no information about the outcome; a learner could only memorise it (Sessions 7–8 show it produces perfect

but useless splits). Drop it.

STEP 2 (B)

The label takes values in $\{\text{Yes}, \text{No}\}$ — categorical — so this is **classification** (binary). If the label were starting salary in lakh, a real number, the same table would be a **regression** task; the features and rows would be unchanged.

STEP 3 (C)

θ_1 is a **parameter**: its value is chosen by the learning algorithm to fit the data. α is a **hyper-parameter**: it controls *how* learning proceeds and is set by the practitioner before training (and tuned on a validation set).

STEP 4 (D)

Instance = example, record, sample, point, case, object, row. Feature = attribute, predictor, input variable, dimension, field, column.

Four features (dimension 4), label Job offered?, five instances; drop ID; classification; θ_1 learnt, α chosen.

WHERE MARKS ARE LOST Counting ID or the label as a feature (dimension 5 or 6); calling α a parameter; forgetting that the dimension is the number of features, not rows.

MODULE 2 · CONTACT SESSION 2

ML Workflow and Data Preprocessing

Attribute types, data quality, outliers (IQR, 3-sigma), scaling, sampling and partitioning, binning, encoding, feature engineering, hypotheses.

Ten problems in three parts. The numerical core — IQR fences, 3-sigma, min-max and z-scores, binning — is where marks are won or lost by arithmetic, so work those by hand; the scenario problems test whether you can spot leakage, sampling bias and the wrong attribute type. The Module 2 formula card is in **Formula Cards** at the front.

The problem map

#	Title	Type	Topic on the slides	Marks
2.1	Type every column of a car listing	Conceptual	Nominal / ordinal / interval / ratio; discrete vs continuous	8
2.2	Twice as hot, twice as old	Scenario	Interval vs ratio; meaningful operations and transformations	5
2.3	Outliers by the IQR fence	Numerical	Quartiles, IQR, $1.5 \times \text{IQR}$ rule	6
2.4	Outliers by 3-sigma — and a disagreement	Numerical	z-scores, $\mu \pm 3\sigma$; robustness	7
2.5	Scale the age column three ways	Numerical	Min-max, z-score, decimal scaling; fit on training only	7
2.6	A fair split for an unfair dataset	Numerical	Random vs stratified sampling; train/validation/test; imbalance	6
2.7	Bin it, then encode it	Numerical	Equal-width vs equal-depth binning; one-hot vs label encoding	7
2.8	Diagnose a messy table	Scenario	Noise, outliers, missing, duplicates, inconsistency; cleaning plan	7
2.9	Engineer the features	Scenario	Extraction, selection, construction, transformation; curse of dimensionality	6
2.10	Hypotheses that survive the data	Conceptual	Hypothesis space, version space, inductive learning hypothesis	6

MODULE 2 · PART A

Knowing Your Attributes

Problems 2.1–2.2: attribute types and the operations that make sense.

Problem 2.1 · Type every column of a car listing

CONCEPTUAL · 8 MARKS

A used-car marketplace stores these columns for each listing: **Listing ID**; **Brand** (Maruti, Honda, ...); **Model year** (2015, 2019, ...); **Kilometres driven**; **Condition** (Poor / Fair / Good / Excellent); **Engine size** (cc); **Fuel** (Petrol / Diesel / CNG); **Number of previous owners**; **Insurance valid?** (Yes / No); **Listing date**; **Cabin temperature at inspection** (°C); **Asking price** (₹).

- Classify each column as nominal, ordinal, interval or ratio, with a one-line reason.
- Mark each numeric column as discrete or continuous.
- Which columns should not be used as predictive features, and why?

WHAT IT TESTS Stevens's four levels decided by "which arithmetic is meaningful", the separate discrete/continuous axis, and spotting identifiers.

HOW TO ATTEMPT

- For each column ask in turn: only same/different? ordered? meaningful differences? a true zero?
- Numeric columns: countable values (discrete) or any real value (continuous)?
- Identifiers and leak-prone columns are not features.

ANSWERS AT A GLANCE Nominal: ID, Brand, Fuel, Insurance valid. Ordinal: Condition. Interval: Model year, Listing date, Cabin temperature (°C). Ratio: Kilometres, Engine size, Previous owners, Asking price. Drop ID (and treat asking price as the target, not a feature).

Column	Type	Reason	Discrete / continuous
Listing ID	Nominal (identifier)	A label; only equality is meaningful.	discrete
Brand	Nominal	Names with no order.	—
Model year	Interval	Differences meaningful (2019 is 4 years after 2015) but year 0 is arbitrary — 2020 is not "twice" 1010.	discrete
Kilometres driven	Ratio	True zero (never driven); 80,000 km is twice 40,000 km.	continuous (measured)
Condition	Ordinal	Poor < Fair < Good < Excellent, but gaps undefined.	—
Engine size (cc)	Ratio	True zero; 2000 cc is twice 1000 cc.	continuous
Fuel	Nominal	Three unordered categories.	—
Previous owners	Ratio	A count with a true zero.	discrete
Insurance valid?	Nominal, binary (asymmetric)	Yes/No; "Yes" is the informative state.	discrete
Listing date	Interval	Date differences are meaningful; no true zero.	discrete (days)
Cabin temperature (°C)	Interval	Celsius zero is arbitrary; 30 °C is not twice 15 °C.	continuous
Asking price (₹)	Ratio	True zero; ratios meaningful.	continuous

STEP 2 (C)

Listing ID is an identifier — unique per row, no relationship to the outcome; a learner could only memorise it. **Asking price** is what we want to predict (the target), so it cannot also be an input; if the target were something else (days to sell), asking price would be a legitimate feature. **Cabin temperature** is almost certainly irrelevant noise and can be dropped by feature selection (Problem 2.9).

4 nominal, 1 ordinal, 3 interval, 4 ratio as tabled; drop the ID, keep price as target.

WHERE MARKS ARE LOST Calling Model year "ratio"; calling Condition "nominal"; classifying without a reason; confusing discrete/continuous with the four levels.

Problem 2.2 · Twice as hot, twice as old

SCENARIO · 5 MARKS

- A dealer says "the engine bay at 40 °C is twice as hot as the cabin at 20 °C". Is this statement physically meaningful? Convert both to kelvin and decide. Would the statement be meaningful on the Fahrenheit scale?
- Two cars are listed as "3 years old" and "6 years old". Is one twice as old as the other? What if, instead, the column stored "model year 2023" and "model year 2020"?
- State, for each of the four attribute types, the transformation that leaves its meaning unchanged, and give the transformation that links °C to °F.

WHAT IT TESTS The interval-vs-ratio distinction through the temperature and height-above-average puzzles from the slides, and the "allowed transformation" table.

HOW TO ATTEMPT

1. Ratios need a true zero. Kelvin has one; Celsius and Fahrenheit do not.
2. Age (from birth/manufacture) has a true zero; a calendar year does not.
3. Permutation / monotone / $av + b$ / av .

ANSWERS AT A GLANCE (a) Not meaningful: 313 K vs 293 K is a ratio of 1.07; not on Fahrenheit either. (b) Yes for ages; no for model years. (c) permutation; monotonic map; $v' = av + b$; $v' = av$; $^{\circ}\text{F} = 1.8^{\circ}\text{C} + 32$.

STEP 1 (A)

Celsius is an **interval** scale: its zero (water freezing) is arbitrary, so ratios are meaningless. Convert: $40^{\circ}\text{C} = 313.15\text{ K}$ and $20^{\circ}\text{C} = 293.15\text{ K}$.

Ratio in kelvin = $313.15/293.15 = 1.07$ — the engine bay is about 7% hotter in absolute terms, not 100%.

Fahrenheit (104°F vs 68°F , ratio 1.53) gives yet another number, confirming that "twice" depends on the arbitrary zero and is not meaningful. Only kelvin (a **ratio** scale, true zero) supports it.

STEP 2 (B)

Age in years counts time since manufacture — a true zero exists (a brand-new car), so age is a **ratio** attribute and 6 years *is* twice 3 years.

Model year is a calendar date — **interval**; $2023/2020 = 1.0015$ means nothing. Differences are fine (3 years apart); ratios are not. This is the slide's "height above average" puzzle: the same quantity is ratio or interval depending on where zero is placed.

STEP 3 (C)

Nominal: any one-to-one relabelling (permutation). Ordinal: any order-preserving (monotonic) map $v' = f(v)$. Interval: $v' = av + b$ — scale and shift. Ratio: $v' = av$ — scale only, zero stays.

$^{\circ}\text{C}$ to $^{\circ}\text{F}$: $F = 1.8C + 32$, an $av + b$ map with $a = 1.8$, $b = 32$: exactly the interval transformation, which is why both scales are interval and neither is ratio.

"Twice as hot" is meaningless in $^{\circ}\text{C}$ and $^{\circ}\text{F}$ (kelvin ratio 1.07); ages are ratio, model years interval; transformations as listed.

WHERE MARKS ARE LOST Answering (a) with "yes, 40 is twice 20"; forgetting to convert to kelvin; mixing up which transformation belongs to interval ($av + b$) and ratio (av).

MODULE 2 · PART B

Cleaning, Scaling, Sampling, Encoding

*Problems 2.3–2.7: the numerical core of Module 2.***Problem 2.3 · Outliers by the IQR fence**

NUMERICAL · 6 MARKS

Fourteen inspection times (minutes) recorded for used cars:

50, 47, 79, 42, 53, 45, 56, 47, 51, 60, 48, 52, 58, 55.

- Using the median-of-halves method, find Q_1 , Q_2 , Q_3 and the IQR.
- Compute the lower and upper fences and identify any outliers.
- The 79-minute inspection turns out to be genuine (a car with a hidden accident history). Should it be removed? Explain with the two "cases" of outliers from the slides.

WHAT IT TESTS The IQR procedure end to end, and judgement about whether an outlier is noise or signal.**HOW TO ATTEMPT**

- Sort. Median of 14 values = average of the 7th and 8th.
- Q_1 = median of the lower 7, Q_3 = median of the upper 7 (each is the 4th value of its half).
- Fences at $Q_1 - 1.5 \text{ IQR}$ and $Q_3 + 1.5 \text{ IQR}$.

ANSWERS AT A GLANCE $Q_1 = 47$, $Q_2 = 51.5$, $Q_3 = 56$, $\text{IQR} = 9$; fences 33.5 and 69.5; outlier = 79; keep it if the goal includes detecting problem cars.

STEP 1 (A) SORT42, 45, 47, 47, 48, 50, 51, | 52, 53, 55, 56, 58, 60, 79 ($n = 14$, so each half has 7 values).**STEP 2 (A) QUANTILES**

$$Q_2 = \frac{51 + 52}{2} = 51.5 \text{ (average of the 7th and 8th values).}$$

Lower half 42, 45, 47, **47**, 48, 50, 51: seven values, so its median is the 4th $\Rightarrow Q_1 = 47$.Upper half 52, 53, 55, **56**, 58, 60, 79: its median is the 4th $\Rightarrow Q_3 = 56$.

$$\text{IQR} = Q_3 - Q_1 = 56 - 47 = 9.$$

STEP 3 (B) FENCES

$$\text{Lower fence} = Q_1 - 1.5 \text{ IQR} = 47 - 1.5(9) = 47 - 13.5 = 33.5.$$

$$\text{Upper fence} = Q_3 + 1.5 \text{ IQR} = 56 + 13.5 = 69.5.$$

Values below 33.5: none. Values above 69.5: **79**. Outlier = 79.**STEP 4 (C)**

Case 1 (slide): the outlier is *noise* interfering with the analysis — remove or correct it. Case 2: the outlier is *the goal* — fraud, intrusion, defects. A genuine 79-minute inspection caused by a hidden accident is case 2 if the marketplace wants to **detect problem cars**: removing it would delete exactly the signal we want. If the task is merely to estimate the typical inspection duration for scheduling, it may be excluded or down-weighted — but the decision follows from the goal, not from the number.

$Q_1 = 47, Q_2 = 51.5, Q_3 = 56, \text{IQR} = 9$; fences 33.5 / 69.5; outlier 79; keep it when outliers are the target of analysis.

WHERE MARKS ARE LOST Using $\frac{n}{4}$ -th positions instead of the median-of-halves convention; miscounting the middle of a 7-value half; removing every outlier automatically.

Problem 2.4 · Outliers by 3-sigma — and a disagreement

NUMERICAL · 7 MARKS

- a. Engine-temperature readings from a fleet have mean $\mu = 72^\circ\text{C}$ and standard deviation $\sigma = 6$. Compute the 3-sigma bounds and the z-score of each of these readings: 55, 60, 70, 74, 91, 88, 52. Which are outliers? What assumption does the rule make?
- b. Nine service times (minutes): 5, 6, 6, 7, 7, 8, 8, 9, 30. Apply the 3-sigma rule (use the sample standard deviation) and the IQR rule. Do they agree about 30? Explain the disagreement.

WHAT IT TESTS z-scores and the $\mu \pm 3\sigma$ rule; the normality assumption; and the key insight that an extreme value inflates σ so the 3-sigma rule can miss it while the IQR rule (built from the middle) does not.

HOW TO ATTEMPT

1. Bounds $\mu \pm 3\sigma$; $z = (x - \mu)/\sigma$; flag $|z| > 3$.
2. For the nine values compute μ , squared deviations, σ (divide by $n - 1$), the upper bound; then quartiles and the IQR fence.

ANSWERS AT A GLANCE (a) bounds 54 and 90; outliers 91 ($z = 3.17$) and 52 ($z = -3.33$); assumes approximately normal data. (b) 3-sigma: upper bound ≈ 32.8 , so 30 is *not* flagged; IQR: fence = 12.25, so 30 is flagged. IQR is more robust.

STEP 1 (A) BOUNDS

Lower = $72 - 3(6) = 54$; upper = $72 + 18 = 90$.

STEP 2 (A) Z-SCORES

x	55	60	70	74	91	88	52
$z = (x - 72)/6$	-2.83	-2.00	-0.33	0.33	3.17	2.67	-3.33

Outliers: 91 ($z = 3.17 > 3$) and 52 ($z = -3.33 < -3$). Note 55 ($z = -2.83$) is close but inside.

Assumption: the readings are approximately **normally distributed**, so that about 99.7% lie within $\mu \pm 3\sigma$ and anything outside is genuinely rare.

STEP 3 (B) 3-SIGMA

$$\mu = \frac{5 + 6 + 6 + 7 + 7 + 8 + 8 + 9 + 30}{9} = \frac{86}{9} = 9.56.$$

Squared deviations: $(5 - 9.56)^2 = 20.75$; $(6 - 9.56)^2 = 12.64$ (twice); $(7 - 9.56)^2 = 6.53$ (twice); $(8 - 9.56)^2 = 2.42$ (twice); $(9 - 9.56)^2 = 0.31$; $(30 - 9.56)^2 = 418.0$.

Sum = $20.75 + 25.28 + 13.06 + 4.84 + 0.31 + 418.0 = 482.2$; sample variance = $482.2/8 = 60.3$; $\sigma = \sqrt{60.3} = 7.76$.

Upper bound = $9.56 + 3(7.76) = 32.8$. Since $30 < 32.8$, **30 is not an outlier** by 3-sigma ($z = 2.63$).

STEP 4 (B) IQR

Sorted: 5, 6, 6, 7, | 7 |, 8, 8, 9, 30; median = 7. Lower half 5, 6, 6, 7: $Q_1 = 6$. Upper half 8, 8, 9, 30: $Q_3 = 8.5$. IQR = 2.5.

Upper fence = $8.5 + 1.5(2.5) = 8.5 + 3.75 = 12.25$. Since $30 > 12.25$, **30 is an outlier** by IQR.

STEP 5 (B) WHY THEY DISAGREE

The value 30 itself dominates the standard deviation (418 of the 482 in the sum of squares), inflating σ and pushing the 3-sigma bound outward — the rule is fooled by the very point it should catch. Quartiles depend only on the middle of the data and are untouched by one extreme value, so the IQR fence stays tight. With small or skewed samples, prefer the IQR rule; 3-sigma needs a large, roughly normal sample.

(a) outliers 91 and 52. (b) 3-sigma misses 30 (bound 32.8), IQR catches it (fence 12.25); the outlier inflates σ .

WHERE MARKS ARE LOST Sign errors in z ; dividing by n instead of $n - 1$ without saying so (either is accepted if stated); asserting the methods "always agree".

Problem 2.5 · Scale the age column three ways

NUMERICAL · 7 MARKS

In a training set, customer age ranges from 18 to 62 with mean 35 and standard deviation 10.

- Min-max normalise the ages 18, 40, 62 to $[0, 1]$ and then to $[-1, 1]$.
- Z-score standardise the same three ages.
- Apply decimal scaling to the column (state j) and give the scaled value of 40.
- A test customer is 70. Compute the min-max and z-score values *using the training statistics*. Comment on the results and on why the test statistics must not be used.
- Which scaling would you choose if the algorithm is k -nearest neighbours and the column were monthly income (skewed, a few very large values)? Why?

WHAT IT TESTS The three scaling formulas, the fit-on-training rule, and when to prefer standardisation over normalisation.

HOW TO ATTEMPT

- Min-max fraction $\frac{v - \min}{\max - \min}$, then stretch to the new range.
- $z = (v - \mu) / \sigma$.
- Smallest j with $\max |v| / 10^j < 1$.
- Plug the test value into the *training* formulas.

ANSWERS AT A GLANCE (a) 0, 0.5, 1 and $-1, 0, 1$. (b) $-1.7, 0.5, 2.7$. (c) $j = 2; 0.40$. (d) 1.18 and 3.5 — outside the training range, correctly flagged. (e) z-score (or a log transform first): min-max would squash everyone into a sliver.

STEP 1 (A)

$\max - \min = 62 - 18 = 44$.

$$18 \rightarrow \frac{18 - 18}{44} = 0; 40 \rightarrow \frac{40 - 18}{44} = \frac{22}{44} = 0.5; 62 \rightarrow \frac{44}{44} = 1.$$

To $[-1, 1]$: $v' = \text{fraction} \times (1 - (-1)) + (-1) = 2 \text{ fraction} - 1$: $18 \rightarrow -1$; $40 \rightarrow 0$; $62 \rightarrow 1$.

STEP 2 (B)

$$18 \rightarrow \frac{18 - 35}{10} = -1.7; 40 \rightarrow \frac{5}{10} = 0.5; 62 \rightarrow \frac{27}{10} = 2.7.$$

STEP 3 (C)

Largest absolute value is 62. $62/10^1 = 6.2 \not< 1$; $62/10^2 = 0.62 < 1$. So $j = 2$ and $40 \rightarrow 40/100 = 0.40$.

STEP 4 (D)

Min-max: $\frac{70 - 18}{44} = \frac{52}{44} = 1.18$ — above 1. Z-score: $\frac{70 - 35}{10} = 3.5$ — 3.5 standard deviations above the training mean.

Both say the same useful thing: this customer is older than anyone the model trained on, so predictions for them deserve less trust. If we recomputed $\min / \max / \mu / \sigma$ on the test set, we would (i) hide this fact by forcing the value into range, (ii) leak test information into the model, and (iii) make training and test values inconsistent — the same age would map to different numbers in the two sets. Scalers are fit on the training data only and then applied unchanged.

STEP 5 (E)

k -NN is distance-based, so scaling is essential. Income is skewed with a few huge values: min-max would map the billionaire to 1 and squash everyone else near 0, destroying the differences that matter. **Z-score standardisation** is less affected by the extremes (and a log transform before scaling is even better). The slide's rule: normalisation when bounds are known and data is roughly uniform (age); standardisation for skewed attributes like income.

Min-max 0/0.5/1 (or $-1/0/1$); **z** $-1.7/0.5/2.7$; **decimal** $j = 2, 0.40$; **test** $70 \rightarrow 1.18 / 3.5$; **use z-score for income**.

WHERE MARKS ARE LOST Forgetting the " $\times (\text{new_max} - \text{new_min}) + \text{new_min}$ " part for $[-1, 1]$; clipping the test value to 1 "because it must be in range"; picking min-max for income.

Problem 2.6 · A fair split for an unfair dataset

NUMERICAL · 6 MARKS

A loan dataset has 200 applicants: 120 *repaid*, 60 *late*, 20 *defaulted*. It is to be split 75% training / 25% test.

- A plain random split produced a test set with 42 *repaid*, 6 *late* and 2 *defaulted*. What is wrong with it, and what could go wrong for the model?
- Perform a **stratified** split: how many of each class go to training and to test?
- The team then wants to tune a hyper-parameter. Explain why they must not use the test set for this and what they should do instead.
- The *defaulted* class is the one that matters and it is only 10% of the data. Name two ways of adjusting the **training** set for this, and say why the test set must be left alone.

WHAT IT TESTS Representativeness, stratified sampling arithmetic, the train/validation/test discipline, and imbalance handling.

HOW TO ATTEMPT

1. Compare test proportions with population proportions.
2. Stratified: 75% / 25% of *each* class.
3. Validation set; test set used once.
4. Under-sample majority / over-sample (SMOTE) minority — training only.

ANSWERS AT A GLANCE (a) test has 84/12/4 % vs population 60/30/10 % — late and defaulted under-represented. (b) train 90 / 45 / 15, test 30 / 15 / 5. (c) use a validation split (or cross-validation). (d) under-sample repaid, over-sample or SMOTE defaulted — training only.

STEP 1 (A)

Population proportions: $120/200 = 60\%$, $60/200 = 30\%$, $20/200 = 10\%$.

Random test set: $42/50 = 84\%$, $6/50 = 12\%$, $2/50 = 4\%$. Late and defaulted are badly under-represented (sampling noise from a small sample). Consequences: the training set is correspondingly over-supplied with late/defaulted cases, so the model's behaviour is learnt on one mix and tested on another; the test estimate for defaulters rests on only 2 cases and is meaningless.

STEP 2 (B)

Repaid: $0.75 \times 120 = 90$ train, 30 test. Late: $0.75 \times 60 = 45$ train, 15 test. Defaulted: $0.75 \times 20 = 15$ train, 5 test.

Totals: training 150, test 50; every split keeps the 60/30/10 proportions.

STEP 3 (C)

If the test set is used to choose a hyper-parameter, the choice has been fitted to the test set and its error is no longer an honest estimate of performance on unseen data. Carve a **validation set** out of the training data (e.g. 60/20/20 overall, stratified) for tuning, or use cross-validation; touch the test set once, at the end.

STEP 4 (D)

Under-sample the majority (keep fewer repaid rows) or **over-sample the rare class** (repeat defaulted rows, or synthesise new ones with SMOTE) so the learner sees enough defaulters. This changes the training distribution on purpose. The **test set must keep the true 10%** so that the reported metrics describe the real world the model will face.

Random split unrepresentative; stratified 90/45/15 vs 30/15/5; tune on validation; rebalance training only.

WHERE MARKS ARE LOST Splitting the total instead of each class; rounding so the totals no longer add to 200; rebalancing the test set.

Problem 2.7 · Bin it, then encode it

NUMERICAL · 7 MARKS

Monthly incomes (thousand ₹) of 12 customers: 22, 25, 27, 30, 33, 35, 38, 42, 50, 58, 75, 110.

- Discretise into $N = 3$ bins by **equal-width** partitioning: give the bin width, the edges, and the members of each bin.
- Discretise into 3 bins by **equal-depth** partitioning: give the members and reasonable cut points.
- Which method handles the skew (the 110) better, and why? Name two algorithms that prefer discretised inputs.
- The resulting bins are labelled Low / Mid / High. Encode this column (i) by one-hot encoding and (ii) by label encoding, for a customer in Mid. Which encoding is appropriate here and why? What would change if the categories were Petrol / Diesel / CNG?

WHAT IT TESTS Both binning rules with arithmetic, their behaviour on skewed data, and the ordinal/nominal reasoning behind label vs one-hot encoding.

HOW TO ATTEMPT

- $W = (B - A)/N$; edges $A, A + W, A + 2W, B$.
- n/N per bin after sorting; cut midway between neighbours.
- Equal-depth for skew.
- One-hot = one 0/1 column per category; label = one integer.

ANSWERS AT A GLANCE (a) $W = 29.33$; edges 22, 51.33, 80.67, 110; bins of 9 / 2 / 1. (b) 4 per bin: {22..30}, {33..42}, {50..110}; cuts 31.5 and 46. (c) equal-depth; naïve Bayes, decision trees. (d) one-hot (0, 1, 0), label 2; label encoding is fine (ordinal); one-hot for fuel (nominal).

STEP 1 (A)

$$A = 22, B = 110, W = \frac{110 - 22}{3} = \frac{88}{3} = 29.33.$$

Edges: 22, 51.33, 80.67, 110.

Bin 1 [22, 51.33): 22, 25, 27, 30, 33, 35, 38, 42, 50 — 9 customers. Bin 2 [51.33, 80.67): 58, 75 — 2. Bin 3 [80.67, 110]: 110 — 1.

STEP 2 (B)

$12/3 = 4$ per bin. Sorted data is already in order.

Bin 1: 22, 25, 27, 30; Bin 2: 33, 35, 38, 42; Bin 3: 50, 58, 75, 110.

Cut points midway between neighbours: $(30 + 33)/2 = 31.5$ and $(42 + 50)/2 = 46$.

STEP 3 (C)

Equal-width let the single outlier 110 stretch the range so that 9 of 12 customers landed in one bin and the top bin holds one person — "outliers may dominate the presentation; skewed data is not handled well".

Equal-depth guarantees 4 per bin whatever the extremes — "good data scaling". Algorithms that prefer discrete inputs (slide): naïve Bayes, decision trees and their ensembles (random forest), minimum-distance / k -NN classifiers.

STEP 4 (D)

One-hot for Mid: three binary columns (Low, Mid, High) = (0, 1, 0). Label encoding: Low = 1, Mid = 2, High = 3, so Mid $\rightarrow$ 2.

Low < Mid < High is a genuine order, so the **label encoding is appropriate** — the integers carry real information (Mid is between Low and High). Petrol / Diesel / CNG have **no order**; label codes 1, 2, 3 would invent one (is CNG "more" than Petrol?), so **one-hot** encoding is required for a nominal attribute, especially with linear or distance-based models.

Equal-width bins 9/2/1 (edges 51.33, 80.67); equal-depth 4/4/4 (cuts 31.5, 46); label-encode ordered bins, one-hot nominal fuel.

WHERE MARKS ARE LOST Using N bins of width $(B - A)/(N - 1)$; unequal counts in equal-depth; one-hot with a column for "Mid" only.

MODULE 2 · PART C

Quality, Features and Hypotheses

Problems 2.8–2.10: diagnosing dirty data, engineering features, and reasoning about hypotheses.

Problem 2.8 · Diagnose a messy table

SCENARIO · 7 MARKS

An extract from a dealer's listings export:

ID	Brand	Fuel	Year	Reg. year	km	Owner DOB	Price	Sold?
101	Honda	Petrol	2019	2019	41,000	1985-04-12	6.2 lakh	Yes
102	honda	petrol	2019	2019	41,000	1985-04-12	620000	Yes
103	Maruti	CNG	2016	2014	9,999,999	1990-01-01	3.1 lakh	No
104	Tata	Diesel	2021	2021	—	1990-01-01	7.8 lakh	—
105	Hyundai	Petrol	2018	2018	62,000	1990-01-01	–4.5 lakh	No

- List every data-quality problem you can find, naming its type (noise / outlier / missing / duplicate / inconsistent / disguised missing).
- For each problem, say how you would handle it.
- Which single row is unusable for supervised training even after cleaning, and why?

WHAT IT TESTS The catalogue of data-quality problems from the slides, applied to a realistic export; and knowing that a missing *label* is different from a missing feature.

HOW TO ATTEMPT

- Scan column by column and row by row; compare rows to each other (duplicates) and columns to each other (consistency).
- Attach a handling rule to each finding: standardise, merge, impute, cap/remove, investigate.

ANSWERS AT A GLANCE Eight findings: inconsistent codes (Brand/Fuel case), inconsistent units (Price), invalid value (negative price), duplicate (101/102), impossible value (km), cross-field inconsistency (Year vs Reg. year), disguised missing (DOB 1990-01-01), missing values (km, Sold?). Row 104 is unusable: no label.

Finding	Type	Handling
"Honda"/"honda", "Petrol"/"petrol"	Inconsistent codes	Standardise case and spelling before anything else, or "honda" becomes a new category.
Price as "6.2 lakh" and "620000"	Inconsistent units / format	Parse every price to one numeric unit (rupees) before anything else.
Price "-4.5 lakh" (row 105)	Invalid value (noise / error)	A negative price is impossible — correct from the source or set to missing and impute.
Rows 101 and 102	Near-duplicate (same car, differently formatted)	Merge into one record after standardising; otherwise the car is double-counted and may leak from training into test.
km = 9,999,999	Outlier / noise (typing error, "mistake or millionaire")	Physically impossible; set to missing and impute, or drop the row.
Row 103: Year 2016 but registered 2014	Inconsistent (cross-field)	A car cannot be registered before it is made; investigate which field is wrong.
Owner DOB = 1990-01-01 for three owners	Disguised missing (form default)	Treat 1 January 1990 as missing; impute or drop the attribute (it is probably irrelevant anyway).
km blank (104), Sold? blank (104)	Missing values	km: impute from similar cars (same brand/year) or drop the row. Sold? is the label — see (c).

STEP 2 (C)

Row 104: its label "Sold?" is missing. A feature can be imputed, but a label cannot be invented — a supervised learner needs (x, y) pairs. The row can only be used for unsupervised or semi-supervised work, or discarded from the training set.

Eight distinct problems as tabled; row 104 is unusable for supervised training because its label is missing.

WHERE MARKS ARE LOST Listing problems without naming the type; missing the disguised-missing DOB; treating the missing label like a missing feature and "imputing" it.

Problem 2.9 · Engineer the features

SCENARIO · 6 MARKS

After cleaning, the listings table has: Brand, Model, Fuel, Year, km, Engine cc, Owners, City, Listing date, Sold-within-30-days? (label). The dataset has 40,000 rows. A colleague proposes adding 250 more columns scraped from the listing photos (pixel statistics).

- Define the four kinds of feature engineering on the slides — extraction, selection, construction, transformation — and give one concrete example of each for this table.
- Explain the curse of dimensionality and why the 250 photo columns might hurt rather than help. What technique would you apply to them?
- Distinguish data engineering from feature engineering in one sentence each.

WHAT IT TESTS The four feature-engineering operations with domain-appropriate examples, and the sparsity argument behind dimensionality reduction.

HOW TO ATTEMPT

1. One definition + one example per kind.
2. Sparsity grows with dimensions; PCA to compress.
3. Raw → prepared (data engineering); prepared → model-ready features (feature engineering).

ANSWERS AT A GLANCE Extraction: PCA on the photo columns. Selection: drop City if irrelevant, drop redundant Model when Brand suffices (or vice versa). Construction: km per year = $\text{km} / (2026 - \text{Year})$; age. Transformation: bin km, one-hot Fuel, log-transform km.

STEP 1 (A)

Extraction — create a smaller number of new attributes that capture most of the information (dimensionality reduction). Example: reduce the 250 photo columns to 5 principal components.

Selection — keep a subset of existing features; drop irrelevant, redundant, or mostly-missing ones. Example: drop Listing date if it does not affect sales, or drop Model if Brand already carries the signal (redundant).

Construction — build new features from existing ones. Example: age = $2026 - \text{Year}$; km per year = km / age ; an interaction $\text{Engine cc} \times \text{Fuel}$.

Transformation — change the representation of a feature. Example: bin km into Low/Mid/High; one-hot encode Fuel; take $\log(\text{km})$ to tame skew; standardise Engine cc.

STEP 2 (B)

As the number of attributes grows, the data becomes increasingly **sparse** in the space it occupies: 40,000 points that fill a 10-dimensional space are hopelessly scattered in 260 dimensions. Every point is far from every other, "nearest neighbours" stop being meaningful, and the number of examples needed to cover the space grows exponentially. Extra columns also invite overfitting (the model finds spurious patterns among 250 noisy pixel statistics) and slow training.

Remedy: **feature extraction** with **Principal Component Analysis** — compress the 250 columns into a handful of components that retain most of the variance — and then **feature selection** on a validation set to keep only components that actually improve the sold-within-30-days prediction.

STEP 3 (C)

Data engineering converts raw data into prepared data — cleaning, deduplication, unit fixing, sampling, scaling. **Feature engineering** tunes the prepared data into the features the model expects — extraction, selection, construction, transformation.

Four operations with examples as above; 250 raw columns make the space sparse (curse of dimensionality) — compress with PCA and select.

WHERE MARKS ARE LOST Mixing up selection (subset of existing) with extraction (new combined columns); explaining the curse as "too slow" without the sparsity idea.

Problem 2.10 · Hypotheses that survive the data

CONCEPTUAL · 6 MARKS

A concept "customer will buy an extended warranty" is described by four attributes: Age (Young / Old), Car (New / Used), Fuel (Petrol / Diesel), Finance (Yes / No). Four training examples:

#	Age	Car	Fuel	Finance	Buys warranty
1	Young	Used	Petrol	Yes	Yes
2	Old	Used	Diesel	Yes	Yes
3	Young	New	Petrol	No	No
4	Old	Used	Petrol	Yes	Yes

Hypotheses are written as $\langle \text{Age, Car, Fuel, Finance} \rangle$ with "?" meaning any value and "Ø" meaning no value.

- Define *target function*, *hypothesis*, *hypothesis space* and *version space*.
- Write the most general and the most specific hypotheses.
- Check whether each of these is consistent with all four examples: $h_1 = \langle ?, \text{Used}, ?, ? \rangle$, $h_2 = \langle ?, ?, \text{Petrol}, ? \rangle$, $h_3 = \langle ?, ?, ?, \text{Yes} \rangle$.
- State the inductive learning hypothesis and say what a fifth example $\langle \text{Old, New, Diesel, Yes} \rangle \rightarrow \text{No}$ would do to the version space.

WHAT IT TESTS Mitchell's vocabulary and the mechanics of consistency — the foundation the decision-tree sessions assume.

HOW TO ATTEMPT

- Definitions in one line each.
- All "?" vs all "Ø".
- A hypothesis is consistent if it says Yes on exactly the Yes rows.
- A new example eliminates every hypothesis that gets it wrong.

ANSWERS AT A GLANCE (b) $\langle ?, ?, ?, ? \rangle$ and $\langle \text{Ø}, \text{Ø}, \text{Ø}, \text{Ø} \rangle$. (c) h_1 consistent; h_2 not (says Yes on row 3, No on row 2); h_3 consistent. (d) The fifth example rules out h_3 (it says Yes, truth is No); h_1 survives.

STEP 1 (A)

Target function f : the true, unknown rule that produced the labels. **Hypothesis** h : a proposed function believed to approximate f . **Hypothesis space** H : the set of all hypotheses the learner can output (here, all vectors of four slots each being a value, "?" or Ø). **Version space**: the subset of H consistent with every training example seen so far.

STEP 2 (B)

Most general: $\langle ?, ?, ?, ? \rangle$ — every customer buys. Most specific: $\langle \text{Ø}, \text{Ø}, \text{Ø}, \text{Ø} \rangle$ — nobody buys.

STEP 3 (C) H_1

h_1 says Yes iff Car = Used. Rows 1, 2, 4 are Used $\rightarrow$ Yes ✓ (labels Yes); row 3 is New $\rightarrow$ No ✓ (label No).

Consistent.

STEP 4 (C) H_2

h_2 says Yes iff Fuel = Petrol. Row 3 is Petrol $\rightarrow$ Yes $\times$ (label No); row 2 is Diesel $\rightarrow$ No $\times$ (label Yes). **Not consistent** — out of the version space.

STEP 5 (C) H_3

h_3 says Yes iff Finance = Yes. Rows 1, 2, 4 have Finance Yes $\rightarrow$ Yes $\checkmark$; row 3 has Finance No $\rightarrow$ No $\checkmark$.

Consistent. (Both h_1 and h_3 explain the data; the version space contains at least these two plus $\langle ?, \text{Used}, ?, \text{Yes} \rangle$ and the like.)

STEP 6 (D)

Inductive learning hypothesis: any hypothesis found to approximate the target function well over a sufficiently large set of training examples will also approximate it well over other unobserved examples.

Fifth example $\langle \text{Old}, \text{New}, \text{Diesel}, \text{Yes} \rangle \rightarrow \text{No}$: h_3 predicts Yes (Finance = Yes) — wrong, so h_3 is eliminated. h_1 predicts No (Car = New) — right, so it survives. Every example shrinks the version space; with enough examples, only hypotheses close to f remain — which is exactly why the inductive leap becomes safer as data grows.

Definitions: $\langle ?, ?, ?, ? \rangle / \langle \emptyset, \emptyset, \emptyset, \emptyset \rangle$; h_1, h_3 consistent, h_2 not; the fifth example removes h_3 .

WHERE MARKS ARE LOST Checking only the Yes rows for consistency (a hypothesis must also say No on the No rows); confusing hypothesis space with version space.

MODULE 3 · CONTACT SESSIONS 3 & 4

Linear Models for Regression

Closed form and the normal equation, gradient descent by hand, diagnosing runs, metrics, polynomial and basis functions, bias–variance, ridge and lasso.

Thirteen problems in four parts. The numbers are chosen to be clean enough for a calculator-free exam; the question shapes deliberately vary — fit, verify, recover a hyper-parameter, diagnose a run, compare models — so that you are ready for whatever form the exam question takes. The Module 3 formula card is in **Formula Cards** at the front.

The problem map

#	Title	Type	Topic on the slides	Marks
3.1	Fit a line by the two formulas	Numerical	Closed form, simple regression; residuals; SSE	7
3.2	The normal equation, by hand	Numerical	Vectorisation; $(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$	7
3.3	Sketch the bowl	Numerical	Cost function $J(\theta_1)$; convexity	5
3.4	One step downhill, one feature	Numerical	Gradient descent; plain and matrix form; loss before/after	7
3.5	Two predictors, three flavours	Numerical	Problem Type 3; ridge and lasso updates	10
3.6	Recover the learning rate	Numerical	The update rule read backwards	5
3.7	Diagnose three training logs	Scenario	Problem Type 2; learning rate; feature scaling	6
3.8	One epoch of stochastic descent	Numerical	Batch vs SGD vs mini-batch	7
3.9	Which model should ship?	Numerical	MAE, MSE, RMSE, R^2 on a test set	7
3.10	A curve that is still linear	Numerical	Polynomial regression; basis functions; design matrix	6
3.11	Pick the degree	Scenario	Problem Type 4; bias–variance; learning curves	6
3.12	The price of big weights	Numerical	Ridge cost; effect of λ ; L1 sparsity	7
3.13	Derive it, compare it	Conceptual	Gradient derivation; closed form vs GD; singular $\mathbf{X}^T \mathbf{X}$	8

MODULE 3 · PART A

Solving It: Closed Form and the Cost

Problems 3.1–3.3: the two formulas, the normal equation, and the shape of J .

Problem 3.1 · Fit a line by the two formulas

NUMERICAL · 7 MARKS

Hours of tutoring per week (x) and improvement in test score (y , marks) for five students:

x	1	2	3	4	5
y	2	4	5	4	5

- Fit $\hat{y} = \theta_0 + \theta_1 x$ by least squares using the closed-form formulas. Show the deviation table.
- Compute the five predictions, the residuals and the sum of squared errors.
- Verify that the residuals sum to zero and explain why this must always happen for a least-squares line with an intercept.
- Predict the improvement for 6 hours. What caution applies?

WHAT IT TESTS The closed-form recipe with a deviation table, and understanding one property of least squares ($\sum e_i = 0$).

HOW TO ATTEMPT

- Means; table of $x - \bar{x}$, $y - \bar{y}$, products, squares.
- θ_1 = sum of products / sum of squares; $\theta_0 = \bar{y} - \theta_1 \bar{x}$.
- Predictions and residuals; square and add.
- $\sum e_i = 0$ comes from $\partial J / \partial \theta_0 = 0$.

ANSWERS AT A GLANCE $\theta_1 = 0.6$, $\theta_0 = 2.2$; SSE = 2.4; residuals sum to 0 because $\partial J / \partial \theta_0 = 0$; prediction for 6 h is 5.8 (extrapolation).

STEP 1 (A) MEANS

$$\bar{x} = \frac{1 + 2 + 3 + 4 + 5}{5} = 3, \quad \bar{y} = \frac{2 + 4 + 5 + 4 + 5}{5} = \frac{20}{5} = 4.$$

STEP 2 (A) DEVIATION TABLE

x	y	$x - \bar{x}$	$y - \bar{y}$	$(x - \bar{x})(y - \bar{y})$	$(x - \bar{x})^2$
1	2	-2	-2	4	4
2	4	-1	0	0	1
3	5	0	1	0	0
4	4	1	0	0	1
5	5	2	1	2	4
Sums				6	10

$$\theta_1 = \frac{6}{10} = 0.6, \quad \theta_0 = \bar{y} - \theta_1 \bar{x} = 4 - 0.6 \times 3 = 4 - 1.8 = 2.2.$$

Fitted line: $\hat{y} = 2.2 + 0.6x$.

STEP 3 (B)

x	y	$\hat{y} = 2.2 + 0.6x$	$e = y - \hat{y}$	e^2
1	2	2.8	−0.8	0.64
2	4	3.4	0.6	0.36
3	5	4.0	1.0	1.00
4	4	4.6	−0.6	0.36
5	5	5.2	−0.2	0.04
Sums			0.0	2.40

SSE = 2.40 (so $J = \frac{1}{2 \cdot 5} \times 2.40 = 0.24$ if the $\frac{1}{2n}$ convention is used).

STEP 4 (C)

$-0.8 + 0.6 + 1.0 - 0.6 - 0.2 = 0 \checkmark$. Setting $\frac{\partial J}{\partial \theta_0} = \frac{1}{n} \sum_i (\hat{y}_i - y_i) = 0$ is exactly the condition that the residuals sum to zero — the intercept is chosen so that the line passes through $(\bar{x}, \bar{y})$, which forces positive and negative misses to cancel.

STEP 5 (D)

$\hat{y}(6) = 2.2 + 0.6 \times 6 = 5.8$ marks. Caution: $x = 6$ lies outside the range of the training data $[1, 5]$ — this is **extrapolation**; the linear trend may not continue (improvement probably saturates), so the prediction deserves less confidence.

$\hat{y} = 2.2 + 0.6x$; SSE 2.4; $\sum e = 0$; $\hat{y}(6) = 5.8$ (extrapolated).

WHERE MARKS ARE LOST Using $\sum xy$ and $\sum x^2$ without subtracting the means (that formula needs the intercept-free version); sign slips in residuals; not noting extrapolation.

Problem 3.2 · The normal equation, by hand

NUMERICAL · 7 MARKS

Four cars: age in years $x = 0, 1, 2, 3$ and annual maintenance cost in thousand rupees $y = 1, 3, 4, 6$.

- Write the design matrix $\mathbf{X}$ (with the column of ones) and the target vector $\mathbf{y}$.
- Compute $\mathbf{X}^T \mathbf{X}$, its inverse, and $\mathbf{X}^T \mathbf{y}$.
- Solve the normal equation for $\boldsymbol{\theta}$ and write the fitted model.
- Check your answer with the two closed-form formulas of Problem 3.1.
- Under what condition would $\mathbf{X}^T \mathbf{X}$ have no inverse? Give an example with these columns.

WHAT IT TESTS Vectorised least squares end to end, including the 2×2 inverse, and recognising singularity.

HOW TO ATTEMPT

- $\mathbf{X}^T \mathbf{X} = \begin{bmatrix} n & \sum x \\ \sum x & \sum x^2 \end{bmatrix}$, $\mathbf{X}^T \mathbf{y} = \begin{bmatrix} \sum y \\ \sum xy \end{bmatrix}$.
- Invert with the 2×2 formula; multiply.
- Cross-check with $\theta_1 = S_{xy} / S_{xx}$.

ANSWERS AT A GLANCE $\theta = (1.1, 1.6)$: cost = $1.1 + 1.6 \text{ age}$. Singular if a column is a multiple/combination of another (e.g. adding "age in months").

STEP 1 (A)

$$\mathbf{X} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 1 \\ 3 \\ 4 \\ 6 \end{bmatrix}.$$

STEP 2 (B)

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} n & \sum x \\ \sum x & \sum x^2 \end{bmatrix} = \begin{bmatrix} 4 & 6 \\ 6 & 14 \end{bmatrix} \text{ (since } \sum x = 0 + 1 + 2 + 3 = 6, \sum x^2 = 0 + 1 + 4 + 9 = 14 \text{)}.$$

$$\text{Determinant} = 4 \times 14 - 6 \times 6 = 56 - 36 = 20.$$

$$(\mathbf{X}^T \mathbf{X})^{-1} = \frac{1}{20} \begin{bmatrix} 14 & -6 \\ -6 & 4 \end{bmatrix} = \begin{bmatrix} 0.7 & -0.3 \\ -0.3 & 0.2 \end{bmatrix}.$$

$$\mathbf{X}^T \mathbf{y} = \begin{bmatrix} \sum y \\ \sum xy \end{bmatrix} = \begin{bmatrix} 1 + 3 + 4 + 6 \\ 0(1) + 1(3) + 2(4) + 3(6) \end{bmatrix} = \begin{bmatrix} 14 \\ 29 \end{bmatrix}.$$

STEP 3 (C)

$$\theta = \begin{bmatrix} 0.7 & -0.3 \\ -0.3 & 0.2 \end{bmatrix} \begin{bmatrix} 14 \\ 29 \end{bmatrix} = \begin{bmatrix} 0.7(14) - 0.3(29) \\ -0.3(14) + 0.2(29) \end{bmatrix} = \begin{bmatrix} 9.8 - 8.7 \\ -4.2 + 5.8 \end{bmatrix} = \begin{bmatrix} 1.1 \\ 1.6 \end{bmatrix}.$$

$$\text{Model: } \widehat{\text{cost}} = 1.1 + 1.6 \times \text{age (thousand ₹)}.$$

STEP 4 (D)

$$\bar{x} = 1.5, \bar{y} = 3.5. S_{xy} = (-1.5)(-2.5) + (-0.5)(-0.5) + (0.5)(0.5) + (1.5)(2.5) = 3.75 + 0.25 + 0.25 + 3.75 = 8. S_{xx} = 2.25 + 0.25 + 0.25 + 2.25 = 5. \theta_1 = 8/5 = 1.6 \checkmark; \theta_0 = 3.5 - 1.6(1.5) = 3.5 - 2.4 = 1.1 \checkmark.$$

STEP 5 (E)

$\mathbf{X}^T \mathbf{X}$ is singular (determinant zero, no inverse) when the columns of $\mathbf{X}$ are linearly dependent — one feature is a multiple or linear combination of others — or when there are more features than examples. Example: add a third column "age in months" = $12 \times$ age; it is an exact multiple of the age column, so the three columns are dependent and the determinant is 0. Fixes: drop the redundant column, use the pseudo-inverse, or regularise (ridge adds $\lambda \mathbf{I}$).

$\theta = (1.1, 1.6)$; verified; singular when features are linearly dependent.

WHERE MARKS ARE LOST Swapping the off-diagonal signs in the inverse; forgetting the $\frac{1}{\det}$; computing $\sum xy$ as $\sum x \sum y$.

Problem 3.3 · Sketch the bowl

NUMERICAL · 5 MARKS

Three points $(1, 2)$, $(2, 4)$, $(3, 6)$ are fitted by a line through the origin, $h_\theta(x) = \theta_1 x$, with cost $J(\theta_1) = \frac{1}{2n} \sum_i (\theta_1 x_i - y_i)^2$.

- Compute $J(\theta_1)$ for $\theta_1 = 0, 1, 2, 3$ and sketch J against θ_1 .
- Find the minimising θ_1 analytically by setting $dJ/d\theta_1 = 0$.
- What property of J guarantees that gradient descent from any starting θ_1 reaches this minimum?

WHAT IT TESTS Evaluating a cost, seeing it as a function of the parameter, and the convexity argument.

HOW TO ATTEMPT

- Plug each θ_1 into the three predictions, square the errors, add, divide by $2n = 6$.
- Differentiate the sum, set to zero, solve.
- Convex $\Rightarrow$ single global minimum.

ANSWERS AT A GLANCE $J(0) = 9.33, J(1) = 2.33, J(2) = 0, J(3) = 2.33$; minimum at $\theta_1 = \sum x_i y_i / \sum x_i^2 = 28/14 = 2$; convexity.

STEP 1 (A)

θ_1	Predictions $\theta_1 x$	Errors $\theta_1 x_i - y_i$	Sum of squares	$J = \text{sum}/6$
0	0, 0, 0	-2, -4, -6	$4 + 16 + 36 = 56$	9.33
1	1, 2, 3	-1, -2, -3	$1 + 4 + 9 = 14$	2.33
2	2, 4, 6	0, 0, 0	0	0
3	3, 6, 9	1, 2, 3	14	2.33

Sketch: a U-shaped curve, symmetric about $\theta_1 = 2$, touching zero there, rising to 2.33 one unit away on either side and to 9.33 two units away on the left.

STEP 2 (B)

$$\frac{dJ}{d\theta_1} = \frac{1}{2n} \sum_i 2(\theta_1 x_i - y_i) x_i = \frac{1}{n} \left(\theta_1 \sum_i x_i^2 - \sum_i x_i y_i \right).$$

$$\text{Set to zero: } \theta_1 = \frac{\sum x_i y_i}{\sum x_i^2} = \frac{1(2) + 2(4) + 3(6)}{1 + 4 + 9} = \frac{28}{14} = 2. \checkmark \text{ matches the table.}$$

STEP 3 (C)

J is a **convex** function of θ_1 (a quadratic with positive leading coefficient $\frac{1}{2n} \sum x_i^2$): the segment between any two points on its graph lies on or above the graph, so there is exactly one minimum and no other stationary point. Gradient descent always moves downhill, so from any start it ends at $\theta_1 = 2$ (given a sensible learning rate).

Bowl with bottom at $\theta_1 = 2$; J values 9.33, 2.33, 0, 2.33; convexity guarantees convergence to the global minimum.

WHERE MARKS ARE LOST Dividing by n instead of $2n$ when the question defines J with $\frac{1}{2n}$; not labelling the sketch's axes; saying "it is a parabola" without the word convex.

MODULE 3 · PART B

Walking Down: Gradient Descent

Problems 3.4–3.8: one-feature and two-feature updates, regularised updates, recovering α , diagnosing logs, and stochastic descent.

Problem 3.4 · One step downhill, one feature

NUMERICAL · 7 MARKS

Data: $(x, y) = (1, 3), (2, 5), (3, 7)$. Model $h_{\theta}(x) = \theta_0 + \theta_1 x$, cost $J = \frac{1}{2n} \sum (h_{\theta}(x_i) - y_i)^2$, learning rate $\alpha = 0.1$, start $\theta^{(0)} = (0, 0)$.

- Compute the initial cost $J(\theta^{(0)})$.
- Write the two update equations and perform one iteration of batch gradient descent (plain, one sum at a time).
- Repeat (b) in matrix form: $\mathbf{e} = \mathbf{X}\theta - \mathbf{y}$, $\nabla J = \frac{1}{n} \mathbf{X}^T \mathbf{e}$.
- Compute the new cost and comment. What is the exact least-squares solution?

WHAT IT TESTS The core Problem-Type-3 mechanics in both notations, with a loss check.

HOW TO ATTEMPT

- Predictions $\rightarrow$ errors $\rightarrow$ squares $\rightarrow J$.
- $\text{grad}_0 = \frac{1}{n} \sum e_i$, $\text{grad}_1 = \frac{1}{n} \sum e_i x_i$; update both with old values.
- Same numbers via $\mathbf{X}^T \mathbf{e}$.

ANSWERS AT A GLANCE $J_0 = 13.83$; $\theta^{(1)} = (0.5, 1.133)$; $J_1 = 2.74$; exact solution $\theta^* = (1, 2)$ with $J = 0$.

STEP 1 (A)

Predictions with $\theta = (0, 0)$: 0, 0, 0. Errors $e_i = \hat{y}_i - y_i$: -3, -5, -7.

$$J_0 = \frac{1}{6} (9 + 25 + 49) = \frac{83}{6} = 13.83.$$

STEP 2 (B) UPDATE EQUATIONS

$$\theta_0 \leftarrow \theta_0 - \alpha \cdot \frac{1}{3} \sum_{i=1}^3 e_i, \quad \theta_1 \leftarrow \theta_1 - \alpha \cdot \frac{1}{3} \sum_{i=1}^3 e_i x_i.$$

STEP 3 (B) SUMS

$$\sum e_i = -3 - 5 - 7 = -15 \Rightarrow \text{grad}_0 = -15/3 = -5.$$

$$\sum e_i x_i = (-3)(1) + (-5)(2) + (-7)(3) = -3 - 10 - 21 = -34 \Rightarrow \text{grad}_1 = -34/3 = -11.333.$$

STEP 4 (B) UPDATE

$$\theta_0^{(1)} = 0 - 0.1(-5) = 0.5; \quad \theta_1^{(1)} = 0 - 0.1(-11.333) = 1.133.$$

STEP 5 (C) MATRIX FORM

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 3 \\ 5 \\ 7 \end{bmatrix}, \mathbf{e} = \mathbf{X}\theta - \mathbf{y} = \begin{bmatrix} -3 \\ -5 \\ -7 \end{bmatrix}.$$

$$\nabla J = \frac{1}{3} \mathbf{X}^T \mathbf{e} = \frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} -3 \\ -5 \\ -7 \end{bmatrix} = \frac{1}{3} \begin{bmatrix} -15 \\ -34 \end{bmatrix} = \begin{bmatrix} -5 \\ -11.333 \end{bmatrix}.$$

$$\boldsymbol{\theta}^{(1)} = \boldsymbol{\theta}^{(0)} - 0.1 \nabla J = \begin{bmatrix} 0.5 \\ 1.133 \end{bmatrix} \text{ — the same numbers, three lines instead of three sums.}$$

STEP 6 (D)

New predictions: $0.5 + 1.133 = 1.633$; $0.5 + 2.267 = 2.767$; $0.5 + 3.4 = 3.900$. Errors: $-1.367, -2.233, -3.100$.

$J_1 = \frac{1}{6}(1.868 + 4.988 + 9.610) = \frac{16.47}{6} = 2.74$ — down from 13.83, an 80% drop in one step, as expected for a good α .

The exact solution: the points lie exactly on $y = 1 + 2x$, so $\boldsymbol{\theta}^* = (1, 2)$ and $J^* = 0$. Descent will approach it over further iterations.

$$J_0 = 13.83 \rightarrow \boldsymbol{\theta}^{(1)} = (0.5, 1.133) \rightarrow J_1 = 2.74; \text{ limit } (1, 2).$$

WHERE MARKS ARE LOST Updating θ_0 first and then using the new θ_0 inside the θ_1 gradient; forgetting $\frac{1}{n}$; sign of the error (use $\hat{y} - y$ consistently with the minus in the update).

Problem 3.5 · Two predictors, three flavours

NUMERICAL · 10 MARKS

A company models weekly sales y (thousand units) from advertising spend x_1 and discount level x_2 :

x_1	1	2	3
x_2	2	1	0
y	6	7	8

Model $\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2$, initial $\boldsymbol{\theta} = (1, 2, -0.5)$, learning rate $\alpha = 0.1$, $n = 3$.

- Write the three update equations and perform one iteration of batch gradient descent.
- Repeat with **ridge** regularisation, $\lambda = 2$ (penalty $\frac{\lambda}{2} \sum_{j \geq 1} \theta_j^2$).
- Repeat with **lasso** regularisation, $\lambda = 2$ (penalty $\lambda \sum_{j \geq 1} |\theta_j|$).
- Tabulate the three results and interpret the differences. Why is θ_0 the same in all three?

WHAT IT TESTS Problem Type 3 with two predictors, plus the modified update rules — the exact structure of the slide's BMI problem, with fresh numbers.

HOW TO ATTEMPT

- Table: $\hat{y}$, e , $e x_1$, $e x_2$; three sums; three gradients.
- Plain update; ridge: multiply old weight by $(1 - \alpha \lambda)$ first; lasso: subtract $\alpha \lambda \text{sign}(\theta_j)$.
- Intercept never regularised.

ANSWERS AT A GLANCE Plain $(1.25, 2.40, -0.15)$; ridge $(1.25, 2.00, -0.05)$; lasso $(1.25, 2.20, 0.05)$.

STEP 1 (A) EQUATIONS

$$\theta_0 \leftarrow \theta_0 - \alpha \frac{1}{3} \sum e_i, \quad \theta_1 \leftarrow \theta_1 - \alpha \frac{1}{3} \sum e_i x_{1i}, \quad \theta_2 \leftarrow \theta_2 - \alpha \frac{1}{3} \sum e_i x_{2i}, \text{ with } e_i = \hat{y}_i - y_i.$$

STEP 2 (A) TABLE

i	x_1	x_2	y	$\hat{y} = 1 + 2x_1 - 0.5x_2$	e	$e x_1$	$e x_2$
1	1	2	6	$1 + 2 - 1 = 2$	-4	-4	-8
2	2	1	7	$1 + 4 - 0.5 = 4.5$	-2.5	-5	-2.5
3	3	0	8	$1 + 6 - 0 = 7$	-1	-3	0
Sums					-7.5	-12	-10.5

Gradients: $\text{grad}_0 = -7.5/3 = -2.5$; $\text{grad}_1 = -12/3 = -4$; $\text{grad}_2 = -10.5/3 = -3.5$.

STEP 3 (A) PLAIN UPDATE

$\theta_0 = 1 - 0.1(-2.5) = 1.25$; $\theta_1 = 2 - 0.1(-4) = 2.40$; $\theta_2 = -0.5 - 0.1(-3.5) = -0.15$.

STEP 4 (B) RIDGE

Rule: $\theta_j \leftarrow \theta_j(1 - \alpha\lambda) - \alpha \text{grad}_j$ for $j \geq 1$; $\alpha\lambda = 0.1 \times 2 = 0.2$, shrink factor 0.8.

$\theta_0 = 1.25$ (not regularised).

$\theta_1 = 2(0.8) - 0.1(-4) = 1.6 + 0.4 = 2.00$.

$\theta_2 = -0.5(0.8) - 0.1(-3.5) = -0.4 + 0.35 = -0.05$.

STEP 5 (C) LASSO

Rule: $\theta_j \leftarrow \theta_j - \alpha \text{grad}_j - \alpha\lambda \text{sign}(\theta_j)$; $\text{sign}(\theta_1) = +1$, $\text{sign}(\theta_2) = -1$.

$\theta_0 = 1.25$.

$\theta_1 = 2 + 0.4 - 0.2(+1) = 2.20$.

$\theta_2 = -0.5 + 0.35 - 0.2(-1) = -0.15 + 0.2 = 0.05$.

STEP 6 (D)

Model	θ_0	θ_1	θ_2
Linear	1.25	2.40	-0.15
Ridge ($\lambda = 2$)	1.25	2.00	-0.05
Lasso ($\lambda = 2$)	1.25	2.20	0.05

Interpretation. Ridge first shrinks each weight proportionally ($2 \rightarrow 1.6$, $-0.5 \rightarrow -0.4$) and then takes the same gradient step, so its weights end up smaller in magnitude than the plain ones. Lasso subtracts a *fixed* $\alpha\lambda = 0.2$ toward zero: for θ_1 (large, positive) that is a small dent; for θ_2 (small, negative) the fixed push overshoots through zero to $+0.05$ — in a proper lasso solver it would be clipped at exactly 0 (soft-thresholding), which is how lasso produces sparse models. θ_0 is identical in all three because the intercept is never penalised — shifting the whole plane up or down is not "complexity".

Plain (1.25, 2.40, -0.15); **ridge** (1.25, 2.00, -0.05); **lasso** (1.25, 2.20, 0.05); **intercept unregularised**.

WHERE MARKS ARE LOST Regularising θ_0 ; applying the shrink factor *after* the gradient step instead of to the old weight; wrong sign for $\text{sign}(\theta_2)$.

Problem 3.6 · Recover the learning rate

NUMERICAL · 5 MARKS

A colleague's notebook shows part of a gradient-descent run on a two-parameter model but the learning rate was not recorded:

Iteration	θ	∇J at that θ
0	(0.5, 1.0)	(2, -4)
1		
2	?	

- Recover the learning rate α from iterations 0 and 1. Check it using both components.
- Compute θ at iteration 2.
- The gradient shrank from (2, -4) to (1.5, -3). Is the run converging? What would you conclude if instead the gradient at iteration 1 were (-3, 6)?

WHAT IT TESTS Reading the update rule backwards — a shape of question that checks you understand $\theta \leftarrow \theta - \alpha \nabla J$ rather than just execute it.

HOW TO ATTEMPT

- $\alpha = (\theta_{\text{old}} - \theta_{\text{new}}) / \text{grad}$, per component.
- Apply the rule forward.
- Shrinking gradient $\Rightarrow$ approaching the bottom; sign-flipped, larger gradient $\Rightarrow$ overshooting.

ANSWERS AT A GLANCE $\alpha = 0.05$; $\theta^{(2)} = (0.325, 1.35)$; converging; a sign-flipped, bigger gradient would mean α is too large.

STEP 1 (A)

From $\theta_j^{(1)} = \theta_j^{(0)} - \alpha g_j$: $\alpha = \frac{\theta_j^{(0)} - \theta_j^{(1)}}{g_j}$.

Component 1: $\alpha = \frac{0.5 - 0.4}{2} = 0.05$. Component 2: $\alpha = \frac{1.0 - 1.2}{-4} = \frac{-0.2}{-4} = 0.05$ ✓ consistent.

STEP 2 (B)

$\theta_1^{(2)} = 0.4 - 0.05(1.5) = 0.4 - 0.075 = 0.325$; $\theta_2^{(2)} = 1.2 - 0.05(-3) = 1.2 + 0.15 = 1.35$.

STEP 3 (C)

The gradient kept its direction and shrank by 25% — the parameters are moving steadily toward the point where the gradient is zero: **converging**. If the new gradient were (-3, 6) — opposite sign and larger — the step would have jumped *across* the valley to a steeper point on the other side: **overshooting**, the signature of a learning rate that is too large; reduce α .

$\alpha = 0.05$; $\theta^{(2)} = (0.325, 1.35)$; **converging**; sign flip = **overshoot**.

WHERE MARKS ARE LOST Sign error giving $\alpha = -0.05$; checking only one component; describing overshoot without linking it to α .

Problem 3.7 · Diagnose three training logs

SCENARIO · 6 MARKS

Three runs of gradient descent on the same regression problem printed J after each iteration:

Iteration	0	1	2	3	4	5
Run A	10.0	9.8	9.6	9.4	9.2	9.0
Run B	10.0	4.1	1.6	0.7	0.4	0.3
Run C	10.0	12.5	19.0	41.0	120.0	390.0

- Diagnose each run (learning rate too small / about right / too large) with the evidence.
- A fourth run, D, decreases but zig-zags: 10.0, 6.5, 7.9, 5.2, 6.1, 4.4. Its two features are "engine cc" (800–2500) and "owners" (1–4). Diagnose and prescribe.
- State the practical rule for choosing α from the slides, and the general stopping criterion.

WHAT IT TESTS Problem Type 2 — interpreting convergence — and the feature-scaling story behind zig-zag.**HOW TO ATTEMPT**

- Look at the direction and speed of change in J .
- Zig-zag with wildly different feature scales $\Rightarrow$ elongated contours.
- Try α on a $\times 3$ grid; stop when J or θ stops changing.

ANSWERS AT A GLANCE A: too small. B: good. C: too large (diverging). D: unscaled features $\rightarrow$ standardise; then α can be raised.

STEP 1 (A)

Run A: J falls by only 0.2 per iteration — monotonic but a crawl; at this rate hundreds of iterations are needed. Learning rate **too small**; increase it (try 3α).

Run B: J falls fast and smoothly ($10 \rightarrow 4.1 \rightarrow 1.6 \rightarrow 0.7$) and flattens near a floor — **about right**.

Run C: J increases every iteration and explodes (12.5, 19, 41, 120, 390) — each step overshoots to a higher point on the opposite wall. Learning rate **too large; diverging**. Reduce α (try $\alpha/3$ or smaller).

STEP 2 (B)

Run D decreases overall but bounces ($6.5 \rightarrow 7.9 \rightarrow 5.2 \rightarrow 6.1$). With one feature in the thousands and the other in single digits, the cost surface is a long narrow valley (elongated contours): the step that suits the "owners" direction overshoots in the "engine cc" direction, giving the **zig-zag**. Prescription:

standardise both features (z-score, fitted on the training set) so the contours become roughly circular; then descent goes straight to the minimum and a larger α becomes safe. (Alternatively lower α — but that only slows the zig-zag.)

STEP 3 (C)

Rule: plot J against the iteration number; it must decrease every iteration. If it rises or oscillates, decrease α ; if it decreases very slowly, increase α . Try values on a grid like

0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1. Stop when the change in J (or $\|\theta_{\text{new}} - \theta_{\text{old}}\|$, or $\|\nabla J\|$) falls below a small threshold ϵ , or after a maximum number of iterations.

A small, B right, C diverging; D needs feature scaling; grid-search α and stop on a threshold.

WHERE MARKS ARE LOST Calling C "converging slowly"; prescribing a smaller α for D without mentioning scaling; not quoting the numbers as evidence.

Problem 3.8 · One epoch of stochastic descent

NUMERICAL · 7 MARKS

Data $(1, 2), (2, 4), (3, 5)$; model $\theta_0 + \theta_1 x$; $\alpha = 0.1$; start $(0, 0)$.

- Write the stochastic gradient descent (SGD) update rule for one example and explain how it differs from the batch rule.
- Run one epoch of SGD visiting the examples in the order given. Show θ after each example.
- The batch step from the same start gives $\theta = (0.367, 0.833)$ with $J = 1.51$. Compute J after your SGD epoch and compare the two — which did more work, and which got further?
- How many updates per epoch would mini-batch descent with batch size 2 perform on a dataset of 1,000 examples? Name one advantage of each of the three variants.

WHAT IT TESTS The mechanics and trade-offs of SGD vs batch vs mini-batch, with a real one-epoch computation.

HOW TO ATTEMPT

- SGD: $\theta_j \leftarrow \theta_j - \alpha(h_{\theta}(x^{(i)}) - y^{(i)})x_j^{(i)}$ — one example, no $\frac{1}{n}$.
- Three sequential updates, each using the latest θ .
- Evaluate $J = \frac{1}{2n} \sum e^2$ at the end.

ANSWERS AT A GLANCE After examples 1, 2, 3: $(0.2, 0.2) \rightarrow (0.54, 0.88) \rightarrow (0.722, 1.426)$; $J = 0.034$ vs batch 1.51. Mini-batch of 2 on 1,000 examples: 500 updates per epoch.

STEP 1 (A)

SGD: pick one example i ; $\theta_0 \leftarrow \theta_0 - \alpha e_i$, $\theta_1 \leftarrow \theta_1 - \alpha e_i x_i$ with $e_i = h_{\theta}(x_i) - y_i$. Batch uses the *average* of $e_i x_i$ over all n examples per update; SGD uses a single example's value, updates immediately, and therefore makes n updates per epoch instead of one.

STEP 2 (B) EXAMPLE 1: $(1, 2)$

$\hat{y} = 0 + 0(1) = 0$; $e = 0 - 2 = -2$. $\theta_0 = 0 - 0.1(-2) = 0.2$; $\theta_1 = 0 - 0.1(-2)(1) = 0.2$. $\theta = (0.2, 0.2)$.

STEP 3 (B) EXAMPLE 2: $(2, 4)$

$\hat{y} = 0.2 + 0.2(2) = 0.6$; $e = 0.6 - 4 = -3.4$. $\theta_0 = 0.2 - 0.1(-3.4) = 0.54$; $\theta_1 = 0.2 - 0.1(-3.4)(2) = 0.2 + 0.68 = 0.88$. $\theta = (0.54, 0.88)$.

STEP 4 (B) EXAMPLE 3: $(3, 5)$

$\hat{y} = 0.54 + 0.88(3) = 3.18$; $e = 3.18 - 5 = -1.82$. $\theta_0 = 0.54 - 0.1(-1.82) = 0.722$; $\theta_1 = 0.88 - 0.1(-1.82)(3) = 0.88 + 0.546 = 1.426$. $\theta = (0.722, 1.426)$.

STEP 5 (C)

Predictions with $(0.722, 1.426)$: 2.148, 3.574, 5.000. Errors: 0.148, -0.426 , 0.000. $J = \frac{1}{6}(0.0219 + 0.1815 + 0) = \frac{0.2034}{6} = 0.034$.

Batch made one update and reached $J = 1.51$; SGD made three updates (the same amount of gradient arithmetic — each example's error was computed once) and reached $J = 0.034$. On this tiny, well-behaved dataset SGD got much further per epoch because it corrected course three times. In general its steps are noisier and it hovers around the minimum rather than settling exactly; the exact least-squares solution here is $(0.667, 1.5)$ with $J = 0.028$.

STEP 6 (D)

Mini-batch with size 2: $1000/2 = 500$ updates per epoch. Advantages — batch: smooth, stable convergence, exact gradient; SGD: cheapest per update, fast early progress, works online on streaming data; mini-batch: a balance of speed and stability that uses vectorised hardware efficiently (the deep-learning default).

SGD epoch $\rightarrow (0.722, 1.426)$, $J = 0.034$ vs batch $J = 1.51$; 500 mini-batch updates per epoch.

WHERE MARKS ARE LOST Including a $\frac{1}{n}$ in the SGD step; using the original θ for every example instead of the updated one; comparing J at different θ without recomputing predictions.

MODULE 3 · PART C

Judging and Bending the Line

Problems 3.9–3.11: evaluation metrics, polynomial regression, and choosing complexity.

Problem 3.9 · Which model should ship?

NUMERICAL · 7 MARKS

Two regression models predict delivery times (minutes) for five unseen orders:

Order	1	2	3	4	5
Actual y	10	12	15	18	20
Model A	11	11	16	17	21
Model B $\hat{y}$	9	13	14	20	19

- For each model compute MAE, MSE and RMSE.
- Compute R^2 for each model.
- Which model should ship? Could the ranking differ between MAE and RMSE in general — when?
- Model B had a lower error than A on the *training* set. Does that change your answer?

WHAT IT TESTS The four regression metrics computed by hand, their interpretation, and the test-set principle.**HOW TO ATTEMPT**

- Residuals; absolute values; squares; averages; square root.
- $\bar{y}$, $\sum(y - \bar{y})^2$, then $1 - SS_{\text{res}}/SS_{\text{tot}}$.
- RMSE punishes big misses more than MAE.

ANSWERS AT A GLANCE A: MAE 1.0, MSE 1.0, RMSE 1.0, $R^2 = 0.926$. B: MAE 1.2, MSE 1.6, RMSE 1.26, $R^2 = 0.882$. Ship A. Training error is irrelevant to the choice.

STEP 1 (A) MODEL AResiduals $y - \hat{y}$: $-1, +1, -1, +1, -1$. Absolute: $1, 1, 1, 1, 1 \Rightarrow \text{MAE} = 5/5 = 1.0$.Squares: $1, 1, 1, 1, 1 \Rightarrow \text{MSE} = 5/5 = 1.0$; $\text{RMSE} = \sqrt{1.0} = 1.0$.**STEP 2 (A) MODEL B**Residuals: $+1, -1, +1, -2, +1$. Absolute: $1, 1, 1, 2, 1 \Rightarrow \text{MAE} = 6/5 = 1.2$.Squares: $1, 1, 1, 4, 1 \Rightarrow \text{MSE} = 8/5 = 1.6$; $\text{RMSE} = \sqrt{1.6} = 1.26$.**STEP 3 (B)**

$$\bar{y} = \frac{10 + 12 + 15 + 18 + 20}{5} = 15. \quad SS_{\text{tot}} = 25 + 9 + 0 + 9 + 25 = 68.$$

$$\text{A: } R^2 = 1 - \frac{5}{68} = 1 - 0.074 = 0.926. \quad \text{B: } R^2 = 1 - \frac{8}{68} = 1 - 0.118 = 0.882.$$

STEP 4 (C)

Model A wins on every metric: smaller typical error and it explains 92.6% of the variation in delivery time against 88.2%. The rankings could disagree in general: MAE counts every miss equally, RMSE squares them, so a model with many small errors and one large one can have the better MAE but the worse RMSE. Here B's single 2-minute miss is what pushes its RMSE (1.26) further above its MAE (1.2) than A's (1.0 vs 1.0).

STEP 5 (D)

No. Training error measures fit to data the model has already seen and can be made arbitrarily small by a flexible model; the decision must rest on unseen data. B's better training error with worse test error is, if anything, a sign that B is overfitting.

A: 1.0/1.0/1.0/0.926; B: 1.2/1.6/1.26/0.882; ship A; ignore training error.

WHERE MARKS ARE LOST Forgetting to square-root for RMSE; computing R^2 with $SS_{\text{res}}/SS_{\text{tot}}$ without the "1–"; letting training error influence the choice.

Problem 3.10 · A curve that is still linear

NUMERICAL · 6 MARKS

Braking distance y (m) at speed x (tens of km/h): $(x, y) = (-1, 3), (0, 1), (1, 3), (2, 9)$ (negative x is a coded speed below the reference).

- A straight line fits badly. Write the degree-2 polynomial model $y = w_0 + w_1x + w_2x^2$ as a linear basis-function model: give ϕ_0, ϕ_1, ϕ_2 and the 4×3 design matrix Φ .
- Verify that $\mathbf{w} = (1, 0, 2)$ fits all four points exactly, and explain why the normal equation would therefore return exactly this $\mathbf{w}$.
- Explain in what sense this curved model is a "linear" model, and why that matters for training.
- Predict y at $x = 3$. Name one risk with this prediction.

WHAT IT TESTS The basis-function view — feature construction turns a curve into ordinary linear regression — and the phrase "linear in $\mathbf{w}$ ".

HOW TO ATTEMPT

- $\phi_j(x) = x^j$; one row per example.
- Plug each x into $1 + 0x + 2x^2$.
- Linear in the weights; convex cost; same solvers.

ANSWERS AT A GLANCE Φ has rows $(1, x, x^2)$; $\mathbf{w} = (1, 0, 2)$ gives $\text{SSE} = 0$, hence it is the least-squares solution; linear in $\mathbf{w}$; $\hat{y}(3) = 19$ (extrapolation).

STEP 1 (A)

$$\phi_0(x) = 1, \phi_1(x) = x, \phi_2(x) = x^2; \quad y = \sum_{j=0}^2 w_j \phi_j(x) = \mathbf{w}^T \boldsymbol{\phi}(x).$$

$$\Phi = \begin{bmatrix} 1 & -1 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 3 \\ 1 \\ 3 \\ 9 \end{bmatrix}.$$

STEP 2 (B)

$x = -1: 1 + 0 + 2(1) = 3 \checkmark; x = 0: 1 \checkmark; x = 1: 1 + 0 + 2 = 3 \checkmark; x = 2: 1 + 0 + 8 = 9 \checkmark$. All residuals are 0, so $SSE = 0$.

The least-squares cost is never negative, so a $\mathbf{w}$ with cost 0 is a global minimiser; and because $\Phi^T \Phi$ is invertible here (three independent columns, four rows), the minimiser is unique — the normal equation $\mathbf{w} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y}$ must return $(1, 0, 2)$.

STEP 3 (C)

The prediction is a **linear combination of the weights**: $y = w_0 \cdot 1 + w_1 \cdot x + w_2 \cdot x^2$. The non-linearity in x is frozen inside the fixed basis functions; the algorithm only ever sees three columns of numbers. Consequently the cost is still a convex quadratic in $\mathbf{w}$ — the closed form and gradient descent apply unchanged, with $\mathbf{X}$ replaced by Φ — and there is a single global minimum.

STEP 4 (D)

$\hat{y}(3) = 1 + 0 + 2(9) = 19$ m. Risk: $x = 3$ is outside the fitted range $[-1, 2]$; polynomials are global and grow fast, so the extrapolated curve may leave the physically sensible region (and with only four points, the fit is exact but not validated).

Φ rows $(1, x, x^2)$; $\mathbf{w} = (1, 0, 2)$ exact; linear in $\mathbf{w}$; $\hat{y}(3) = 19$.

WHERE MARKS ARE LOST Saying the model is "non-linear" full stop; omitting the column of ones from Φ ; not connecting $SSE = 0$ to "must be the least-squares answer".

Problem 3.11 · Pick the degree

SCENARIO · 6 MARKS

Polynomial models of increasing degree were fitted to 60 training cars and evaluated on 40 test cars (RMSE in thousand ₹):

Degree	1	2	3	4	5
Training RMSE	2.1	1.2	0.9	0.5	0.1
Test RMSE	2.2	1.3	1.1	1.9	3.4

- Which degree should be chosen, and why?
- Classify degrees 1 and 5 as high-bias or high-variance, with evidence.
- The team can collect 600 more training cars. Which degree's test error would improve most, and which barely at all? Explain with learning curves.
- Sketch (describe) the two curves of training and test error against degree, and mark the sweet spot.

WHAT IT TESTS Problem Type 4: reading complexity and learning curves, and the bias–variance vocabulary.

HOW TO ATTEMPT

- Choose the minimum *test* RMSE.
- Both errors high $\Rightarrow$ bias; train $\ll$ test $\Rightarrow$ variance.
- More data narrows the variance gap; does nothing for bias.

ANSWERS AT A GLANCE Degree 3 (test RMSE 1.1). Degree 1 high bias; degree 5 high variance. More data helps degree 5 most, degree 1 hardly at all.

STEP 1 (A)

Test RMSE is the honest measure. It falls $2.2 \rightarrow 1.3 \rightarrow 1.1$ and then rises $1.9 \rightarrow 3.4$. Minimum at **degree 3**: choose it.

STEP 2 (B)

Degree 1 — high bias (underfitting): training RMSE 2.1 and test RMSE 2.2 are both high and almost equal. The straight line cannot follow the price curve even on the data it trained on.

Degree 5 — high variance (overfitting): training RMSE 0.1 is near zero while test RMSE 3.4 is the worst of all — a huge gap. The wiggly polynomial has fitted the noise of the 60 training cars.

STEP 3 (C)

Learning curves plot error against training-set size. For a high-variance model, training error rises slowly and test error falls as data grows, and the gap between them **narrows**: with 660 cars the degree-5 polynomial can no longer bend to every point, so its test RMSE would drop substantially (and its training RMSE would rise from 0.1). For a high-bias model both curves have already flattened at a high level: more data does not change what a straight line can represent, so degree 1's test error would barely move. Degree 3 would improve modestly.

STEP 4 (D)

Horizontal axis: degree (model complexity) 1 to 5. Training-error curve: falls monotonically $2.1 \rightarrow 0.1$. Test-error curve: U-shaped, falling to 1.1 at degree 3 and rising to 3.4. Left of degree 3 label "high bias / underfitting"; right of it "high variance / overfitting"; the sweet spot is the bottom of the U at degree 3.

Choose degree 3; 1 = bias, 5 = variance; extra data helps 5 most and 1 least; U-shaped test curve with minimum at 3.

WHERE MARKS ARE LOST Picking degree 5 for its training error; describing learning curves without saying what the axes are; claiming more data cures underfitting.

MODULE 3 · PART D

Taming and Understanding

Problems 3.12–3.13: regularisation with numbers, and the derivations and comparisons the exam asks for.

Problem 3.12 · The price of big weights

NUMERICAL · 7 MARKS

Two candidate parameter vectors for a model with $n = 4$ training examples and two features:

Candidate	θ_0	θ_1	θ_2	SSE on training data
A	1	3	4	2
B	1	1	1	6

The ridge cost is $J(\boldsymbol{\theta}) = \frac{1}{2n}\text{SSE} + \frac{\lambda}{2} \sum_{j=1}^2 \theta_j^2$.

- With $\lambda = 0$, which candidate has the lower cost?
- With $\lambda = 0.5$, compute both costs. Which is preferred now?
- Find the value of λ at which the two candidates cost the same.
- Explain what happens to the fitted model as $\lambda \rightarrow \infty$, and why lasso ($\lambda \sum |\theta_j|$) can drive weights to exactly zero while ridge cannot.

WHAT IT TESTS Regularisation as a trade-off you can compute, the role of λ , and the L1/L2 sparsity distinction.

HOW TO ATTEMPT

- Data term = SSE/8; penalty = $\frac{\lambda}{2}(\theta_1^2 + \theta_2^2)$ — θ_0 excluded.
- Set the two costs equal and solve for λ .
- Large λ : penalty dominates; only θ_0 survives.

ANSWERS AT A GLANCE (a) A ($0.25 < 0.75$). (b) A = 6.5, B = 1.25: B. (c) $\lambda = 1/23 \approx 0.0435$. (d) Flat line $\hat{y} = \theta_0$; L1's fixed pull reaches zero, L2's proportional shrink never does.

STEP 1 (A)

$\lambda = 0$: $J_A = \frac{2}{2 \times 4} = 0.25$; $J_B = \frac{6}{8} = 0.75$. **A** — it fits the training data better and nothing penalises its size.

STEP 2 (B)

Penalty for A: $\frac{0.5}{2}(3^2 + 4^2) = 0.25 \times 25 = 6.25$. $J_A = 0.25 + 6.25 = 6.50$.

Penalty for B: $0.25 \times (1^2 + 1^2) = 0.5$. $J_B = 0.75 + 0.5 = 1.25$.

B is now preferred: its worse fit (+0.5 in the data term) is far cheaper than A's big weights (+6.25 in the penalty). θ_0 is not penalised, which is why only θ_1, θ_2 enter.

STEP 3 (C)

Set $J_A = J_B$: $0.25 + \frac{\lambda}{2}(25) = 0.75 + \frac{\lambda}{2}(2)$.

$$12.5\lambda - \lambda = 0.75 - 0.25 \Rightarrow 11.5\lambda = 0.5 \Rightarrow \lambda = \frac{0.5}{11.5} = \frac{1}{23} \approx 0.0435.$$

For $\lambda < 0.0435$ the better-fitting A wins; above it the smaller B wins. λ is the exchange rate between fit and simplicity — chosen on a validation set.

STEP 4 (D)

As $\lambda \rightarrow \infty$ the penalty dominates: the only way to keep J finite is $\theta_1, \theta_2 \rightarrow 0$, leaving $\hat{y} \approx \theta_0$ — a horizontal line through the mean of y . Massive **underfitting** (the slide's " $\lambda = 10^{10}$ " picture).

Ridge's gradient step multiplies the weight by $(1 - \alpha\lambda)$: a *proportional* shrink that makes weights smaller and smaller but never exactly zero. Lasso subtracts a *fixed* amount $\alpha\lambda \text{sign}(\theta_j)$ each step, which reaches zero in finitely many steps and (with soft-thresholding) stays there. Geometrically, the L1 constraint region is a diamond whose corners lie on the axes, and the cost contours usually touch a corner — where one coordinate is exactly zero — while the L2 circle has no corners. Hence L1 gives sparse models and does feature selection; L2 gives small but non-zero weights.

$\lambda = 0$: A; $\lambda = 0.5$: B (1.25 vs 6.5); **break-even** $\lambda = 1/23$; $\lambda \rightarrow \infty$ **flattens the model**; **L1 zeroes weights**, L2 only shrinks them.

WHERE MARKS ARE LOST Penalising θ_0 (adds 0.25 to both, which cancels here but is wrong); forgetting the $\frac{1}{2}$ in the penalty; explaining sparsity as "L1 is stronger" instead of fixed-vs-proportional shrink.

Problem 3.13 · Derive it, compare it

CONCEPTUAL · 8 MARKS

- Starting from $J(\theta) = \frac{1}{2n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2$ with $h_{\theta}(\mathbf{x}) = \sum_{k=0}^d \theta_k x_k$, derive $\frac{\partial J}{\partial \theta_j}$ and hence the batch gradient-descent update. State clearly where the $\frac{1}{2}$ goes.
- Write the same gradient in matrix form and show how the normal equation follows from setting it to zero.
- Compare the closed-form solution and gradient descent under four headings: iterations, learning rate, cost with large d , and applicability beyond least squares. When would you choose each?
- Explain why gradient descent needs feature scaling but the normal equation does not.

WHAT IT TESTS The derivation the slides show in pictures, written out; and the comparison table, understood rather than memorised.

HOW TO ATTEMPT

- Chain rule on the square; only the $k = j$ term survives inside.
- $\nabla J = \frac{1}{n} \mathbf{X}^T (\mathbf{X}\theta - \mathbf{y})$; set to zero.
- Four-row table; then two "choose when" sentences.
- Scaling affects the *path* of an iterative method, not the algebraic solution.

ANSWERS AT A GLANCE $\partial J / \partial \theta_j = \frac{1}{n} \sum_i (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$; $\nabla J = \frac{1}{n} \mathbf{X}^T (\mathbf{X}\theta - \mathbf{y}) = \mathbf{0} \Rightarrow \theta = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$; GD for large n, d , online, non-least-squares costs; closed form for small d and one-off exact fits; scaling matters only for the iterative path.

STEP 1 (A)

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{2n} \sum_{i=1}^n \frac{\partial}{\partial \theta_j} \left(\sum_k \theta_k x_k^{(i)} - y^{(i)} \right)^2.$$

Chain rule: $\frac{\partial}{\partial \theta_j} (u^2) = 2u \frac{\partial u}{\partial \theta_j}$ with $u = \sum_k \theta_k x_k^{(i)} - y^{(i)}$. The 2 cancels the $\frac{1}{2}$:

$$= \frac{1}{n} \sum_{i=1}^n \left(\sum_k \theta_k x_k^{(i)} - y^{(i)} \right) \frac{\partial}{\partial \theta_j} \left(\sum_k \theta_k x_k^{(i)} - y^{(i)} \right).$$

Inside the last bracket only the $k = j$ term depends on θ_j , and its derivative is $x_j^{(i)}$; $y^{(i)}$ is a constant:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{n} \sum_{i=1}^n \left(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}.$$

Update: $\theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$, for all $j = 0, \dots, d$ simultaneously.

STEP 2 (B)

Stack the examples: $\mathbf{X}\boldsymbol{\theta} - \mathbf{y}$ is the vector of errors; "sum of error $\times$ feature j " for all j at once is

$$\mathbf{X}^T(\mathbf{X}\boldsymbol{\theta} - \mathbf{y}). \text{ So } \nabla J = \frac{1}{n} \mathbf{X}^T(\mathbf{X}\boldsymbol{\theta} - \mathbf{y}).$$

At the minimum $\nabla J = \mathbf{0}$: $\mathbf{X}^T \mathbf{X} \boldsymbol{\theta} - \mathbf{X}^T \mathbf{y} = \mathbf{0} \Rightarrow \mathbf{X}^T \mathbf{X} \boldsymbol{\theta} = \mathbf{X}^T \mathbf{y} \Rightarrow \boldsymbol{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$, provided the inverse exists.

STEP 3 (C)

Heading	Gradient descent	Closed form (normal equation)
Iterations	Many; must check convergence	None — one formula
Learning rate	Must choose α (and scale features)	No α
Cost with large d	Each iteration $O(nd)$; fine for very large d	Inverting a $(d+1) \times (d+1)$ matrix, about $O(d^3)$ — slow beyond a few thousand features
Beyond least squares	Works for any differentiable cost (logistic regression, neural nets); supports online/streaming updates	Exists only for the squared-error cost; recompute from scratch when data changes

Choose the closed form for small-to-moderate d when an exact one-off fit is wanted. Choose gradient descent when n or d is very large, when data streams in, or when the model is not least squares.

STEP 4 (D)

The normal equation solves the zero-gradient equations algebraically; rescaling a feature just rescales the corresponding coefficient, and the solution (and predictions) are the same. Gradient descent, however, *walks* on the cost surface; with unequal feature scales the surface is a long narrow valley, the single step size α is too big across the valley and too small along it, and the walk zig-zags slowly. Scaling makes the contours round so the walk goes straight. It changes the path, not the destination.

Derivation with the $\frac{1}{2}$ cancelling the 2; $\nabla J = \frac{1}{n} \mathbf{X}^T(\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$; normal equation; comparison table; scaling affects the iterative path only.

WHERE MARKS ARE LOST Dropping the $x_j^{(i)}$ factor; writing $\mathbf{X}\mathbf{X}^T$ instead of $\mathbf{X}^T \mathbf{X}$; saying the closed form "needs scaling too".

MODULE 4 · CONTACT SESSIONS 5 & 6

Logistic Regression and Classifier Evaluation

Sigmoid, boundaries and odds, cross-entropy and its gradient, regularised updates, confusion-matrix metrics, ROC and AUC, imbalance, multi-class.

Thirteen problems in four parts. Numbers are chosen so the sigmoids are not all saturated — you will see genuine probabilities between 0.1 and 0.9 — and the evaluation problems come in every shape the exam uses: counts given, pairs given, scores given, curves given. The Module 4 formula card is in **Formula Cards** at the front.

The problem map

#	Title	Type	Topic on the slides	Marks
4.1	Sigmoids, a boundary, and odds	Numerical	Sigmoid; decision boundary; odds ratio	7
4.2	One honest step of logistic descent	Numerical	Gradient descent for logistic regression; prediction	9
4.3	The job-offer data, regularised	Numerical	Regularised logistic regression; two iterations; interpretation	10
4.4	Scoring probabilities with cross-entropy	Numerical	Log-loss; why not squared error	6
4.5	The gradient that matches	Conceptual	Derivation of $\partial J / \partial \theta_j$; sigmoid derivative	7
4.6	Four counts, six metrics	Numerical	Confusion matrix; precision, recall, specificity, F_1	6
4.7	The 99% model that catches nothing	Scenario	Accuracy paradox; skewed classes; metric choice	6
4.8	From twelve predictions to a matrix	Numerical	Building the confusion matrix from labels	6
4.9	Draw the ROC from eight scores	Numerical	ROC construction with a tie; AUC by trapezoids	9
4.10	Two curves, one operating point	Numerical	AUC from points; comparing classifiers	6
4.11	Set the threshold for the job	Scenario	Threshold selection; costs of FP vs FN	6
4.12	Rare classes, many classes	Conceptual	Imbalance remedies; one-vs-all, one-vs-one	6
4.13	A boundary that is not a line	Scenario	Non-linear decision boundaries with polynomial features	6

MODULE 4 · PART A

The Model and How It Learns

Problems 4.1–4.5: sigmoids and boundaries, gradient descent by hand, regularisation, log-loss, and the derivation.

Problem 4.1 · Sigmoids, a boundary, and odds

NUMERICAL · 7 MARKS

A logistic model for "customer renews subscription" uses x_1 = months active and x_2 = support tickets resolved, with $\theta = (\theta_0, \theta_1, \theta_2) = (-4, 1, 0.5)$.

- Compute z , $h_\theta(\mathbf{x})$ and the predicted class for customers $(x_1, x_2) = (2, 3)$, $(5, 1)$ and $(1, 6)$.
- Write the equation of the decision boundary and describe it geometrically. Which side is "renews"?
- Interpret $\theta_1 = 1$ and $\theta_2 = 0.5$ as odds ratios.
- Show that $P(y = 0 \mid \mathbf{x}) = g(-z)$ and verify it numerically for the first customer.

WHAT IT TESTS Computing with the sigmoid, reading a boundary off θ , and the odds-ratio interpretation.

HOW TO ATTEMPT

- $z = \theta_0 + \theta_1 x_1 + \theta_2 x_2$; $h = 1/(1 + e^{-z})$; class by the sign of z .
- Boundary: $z = 0$ — a line in the (x_1, x_2) plane.
- Odds multiply by e^{θ_j} per unit of x_j .

ANSWERS AT A GLANCE (a) $z = -0.5, 1.5, 0 \Rightarrow h = 0.378, 0.818, 0.5 \Rightarrow$ classes 0, 1, 1 (on the boundary; ≥ 0.5 rule). (b) $x_1 + 0.5x_2 = 4$; above-right renews. (c) $e^1 = 2.72$, $e^{0.5} = 1.65$. (d) $g(0.5) = 0.622 = 1 - 0.378 \checkmark$.

STEP 1 (A)

$(2, 3)$: $z = -4 + 1(2) + 0.5(3) = -4 + 2 + 1.5 = -0.5$; $h = \frac{1}{1 + e^{0.5}} = \frac{1}{1 + 1.6487} = 0.378$; $z < 0 \Rightarrow$ predict **0** (does not renew), with 38% chance of renewing.

$(5, 1)$: $z = -4 + 5 + 0.5 = 1.5$; $h = \frac{1}{1 + e^{-1.5}} = \frac{1}{1 + 0.2231} = 0.818$; predict **1**.

$(1, 6)$: $z = -4 + 1 + 3 = 0$; $h = 0.5$ exactly — on the boundary; with the rule "predict 1 if $h \geq 0.5$ ", class **1**.

STEP 2 (B)

Boundary: $-4 + x_1 + 0.5x_2 = 0 \iff x_1 + 0.5x_2 = 4 \iff x_2 = 8 - 2x_1$: a straight line through $(4, 0)$ and $(0, 8)$. Points with $x_1 + 0.5x_2 > 4$ (above-right, more months and more resolved tickets) have $z > 0$: **renews**.

STEP 3 (C)

Each extra month active multiplies the odds of renewing by $e^1 = 2.72$ (holding tickets fixed). Each extra resolved ticket multiplies the odds by $e^{0.5} = 1.65$. Because $\ln \frac{p}{1-p} = z$ is linear, these factors are the same at every starting point.

STEP 4 (D)

$1 - g(z) = 1 - \frac{1}{1 + e^{-z}} = \frac{e^{-z}}{1 + e^{-z}}$; multiply numerator and denominator by e^z : $\frac{1}{e^z + 1} = g(-z)$.
 Check: customer 1 has $z = -0.5$; $g(0.5) = \frac{1}{1 + e^{-0.5}} = \frac{1}{1 + 0.6065} = 0.622$, and $1 - 0.378 = 0.622$ ✓.

Classes 0, 1, 1; boundary $x_1 + 0.5x_2 = 4$; odds ratios 2.72 and 1.65; $P(y = 0) = g(-z)$.

WHERE MARKS ARE LOST Using e^{+z} in the sigmoid; calling the boundary " $h = 0$ " instead of $z = 0$ / $h = 0.5$; interpreting θ_1 as "probability increases by 1".

Problem 4.2 · One honest step of logistic descent

NUMERICAL · 9 MARKS

Four students; x_1 = weekly study hours (scaled), $x_2 = 1$ if attendance $\geq 75\%$, $y = 1$ if passed.

i	x_1	x_2	y
1	1	0	0
2	2	1	0
3	3	1	1
4	4	0	1

Initial $\theta = (-2, 0.5, 0.5)$, learning rate $\alpha = 0.5$, $m = 4$.

- Compute the initial cross-entropy cost $J(\theta)$.
- Perform one iteration of batch gradient descent: table of z , h , $e = h - y$, ex_1 , ex_2 ; the three sums; the three updates.
- Compute the cost after the update and comment.
- Predict pass/fail for a student with $x_1 = 2.5$, $x_2 = 1$ using the updated weights.

WHAT IT TESTS The full logistic-regression fitting recipe with genuinely intermediate sigmoid values, plus the loss check and a prediction.

HOW TO ATTEMPT

- $z = \theta_0 + \theta_1 x_1 + \theta_2 x_2$; $h = g(z)$; per-row cost $-\log h$ (if $y = 1$) or $-\log(1 - h)$ (if $y = 0$).
- Errors $e = h - y$; sums of e , ex_1 , ex_2 ; divide by m ; update with α .
- Recompute h and the cost.

ANSWERS AT A GLANCE $J_0 = 0.515$; $\theta^{(1)} = (-1.945, 0.820, 0.515)$; $J_1 = 0.406$; new student: $z = 0.62$, $h = 0.65 \Rightarrow$ pass.

STEP 1 (A) AND (B) — THE TABLE

i	$z = -2 + 0.5x_1 + 0.5x_2$	$h = 1/(1 + e^{-z})$	y	cost row	$e = h - y$	$e x_1$	$e x_2$
1	$-2 + 0.5 + 0 = -1.5$	0.1824	0	$-\log(0.8176) = 0.2014$	0.1824	0.1824	0
2	$-2 + 1 + 0.5 = -0.5$	0.3775	0	$-\log(0.6225) = 0.4741$	0.3775	0.7551	0.3775
3	$-2 + 1.5 + 0.5 = 0$	0.5000	1	$-\log(0.5) = 0.6931$	-0.5	-1.5	-0.5
4	$-2 + 2 + 0 = 0$	0.5000	1	0.6931	-0.5	-2.0	0
Sums				2.0617	-0.4401	-2.5625	-0.1225

$$(a) J_0 = \frac{2.0617}{4} = 0.515.$$

STEP 2 (B) GRADIENTS AND UPDATES

$\text{grad}_0 = -0.4401/4 = -0.1100$; $\text{grad}_1 = -2.5625/4 = -0.6406$; $\text{grad}_2 = -0.1225/4 = -0.0306$.
 $\theta_0 = -2 - 0.5(-0.1100) = -1.945$; $\theta_1 = 0.5 - 0.5(-0.6406) = 0.820$; $\theta_2 = 0.5 - 0.5(-0.0306) = 0.515$.

STEP 3 (C)

New z : $-1.945 + 0.820 = -1.125$; $-1.945 + 1.640 + 0.515 = 0.210$; $-1.945 + 2.460 + 0.515 = 1.030$; $-1.945 + 3.281 = 1.336$.

New h : 0.245, 0.552, 0.737, 0.792. Row costs: $-\log 0.755 = 0.281$; $-\log 0.448 = 0.803$;

$-\log 0.737 = 0.305$; $-\log 0.792 = 0.233$. $J_1 = \frac{1.622}{4} = 0.406$.

The cost fell from 0.515 to 0.406 — the step worked. Row 2 got worse (its h rose to 0.55 although $y = 0$) because the update mostly increased θ_1 ; further iterations will push θ_2 down and fix it.

STEP 4 (D)

$z = -1.945 + 0.820(2.5) + 0.515(1) = -1.945 + 2.051 + 0.515 = 0.621$; $h = \frac{1}{1 + e^{-0.621}} = \frac{1}{1 + 0.537} = 0.650$. $z > 0 \Rightarrow$ predict **pass**, with about 65% confidence.

$J : 0.515 \rightarrow 0.406$; $\theta^{(1)} = (-1.945, 0.820, 0.515)$; **new student passes** ($h = 0.65$).

WHERE MARKS ARE LOST Using $y - h$ with a minus in the update (double sign flip); forgetting $\frac{1}{m}$; computing the cost with $\log_{10}$ instead of the natural log (state the base if you must).

Problem 4.3 · The job-offer data, regularised

NUMERICAL · 10 MARKS

The class example (CGPA, scaled IQ, job offered):

i	1	2	3	4	5	6
CGPA x_1	5.5	5	8	9	6	7.5
IQ	6.7	7	6	7	8	7.3
y	1	0	1	1	0	0

Initial weights $(\theta_0, \theta_1, \theta_2) = (-0.1, 0.2, -0.5)$, learning rate $\alpha = 0.1$, L2 regularisation constant $\lambda = 1$ with cost $J + \frac{\lambda}{2m} \sum_{j \geq 1} \theta_j^2$, $m = 6$.

- Formulate the regularised gradient-descent update equations for this problem.
- Perform two iterations. Report the loss (data term and regularised total) at each stage and interpret.
- Using the final weights: (i) interpret the influence of CGPA on the odds of an offer; (ii) predict whether a candidate with CGPA = 9, IQ = 5 is offered a job.
- The slide's practice version uses $\alpha = 0.8$ and $\lambda = 10$. Without computing, predict what happens to the loss and say why.

WHAT IT TESTS Regularised logistic regression exactly in the slide's practice format, with loss tracking, interpretation and prediction — and judgement about hyper-parameters.

HOW TO ATTEMPT

- Update: $\theta_0 \leftarrow \theta_0 - \alpha \text{grad}_0$; $\theta_j \leftarrow \theta_j(1 - \alpha\lambda/m) - \alpha \text{grad}_j$ for $j = 1, 2$.
- Each iteration: table of z, h, e, ex_1, ex_2 ; sums; updates; loss.
- Odds ratio e^{θ_1} ; prediction from the sign of z .

ANSWERS AT A GLANCE Iteration 1: $(-0.061, 0.495, -0.235)$, iteration 2: $(-0.092, 0.289, -0.473)$; regularised loss $1.078 \rightarrow 0.858 \rightarrow 0.780$. Odds ratio $e^{0.289} = 1.34$ per CGPA point; candidate $(9, 5)$: $z = 0.146$, $h = 0.54 \Rightarrow$ offered (marginally). With $\alpha = 0.8$, $\lambda = 10$ the loss blows up.

STEP 1 (A)

$h = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$, $e_i = h_i - y_i$, $\text{grad}_j = \frac{1}{6} \sum_i e_i x_{ji}$ (with $x_{0i} = 1$). Shrink factor $1 - \alpha\lambda/m = 1 - 0.1/6 = 0.9833$.

$$\theta_0 \leftarrow \theta_0 - 0.1 \text{grad}_0; \quad \theta_1 \leftarrow 0.9833 \theta_1 - 0.1 \text{grad}_1; \quad \theta_2 \leftarrow 0.9833 \theta_2 - 0.1 \text{grad}_2.$$

STEP 2 (B) ITERATION 1

i	$z = -0.1 + 0.2x_1 - 0.5x_2$	h	y	e	$e x_1$	$e x_2$
1	-2.35	0.0871	1	-0.9129	-5.021	-6.117
2	-2.60	0.0691	0	0.0691	0.346	0.484
3	-1.50	0.1824	1	-0.8176	-6.541	-4.905
4	-1.80	0.1419	1	-0.8581	-7.723	-6.007
5	-2.90	0.0522	0	0.0522	0.313	0.417
6	-2.25	0.0953	0	0.0953	0.715	0.696
Sums				-2.372	-17.911	-15.432

Gradients: $-0.395, -2.985, -2.572$. Loss before: data term 1.054; penalty $\frac{1}{12}(0.2^2 + 0.5^2) = 0.024$; total 1.078.

$\theta_0 = -0.1 - 0.1(-0.395) = -0.061$; $\theta_1 = 0.9833(0.2) - 0.1(-2.985) = 0.197 + 0.299 = 0.495$;
 $\theta_2 = 0.9833(-0.5) - 0.1(-2.572) = -0.492 + 0.257 = -0.235$.

STEP 3 (B) ITERATION 2

i	$z = -0.061 + 0.495x_1 - 0.235x_2$	h	y	e	$e x_1$	$e x_2$
1	1.092	0.749	1	-0.251	-1.382	-1.683
2	0.774	0.684	0	0.684	3.422	4.791
3	2.494	0.924	1	-0.076	-0.610	-0.458
4	2.755	0.940	1	-0.060	-0.538	-0.419
5	1.035	0.738	0	0.738	4.427	5.903
6	1.942	0.875	0	0.875	6.559	6.384
Sums				1.910	11.878	14.519

Gradients: 0.318, 1.980, 2.420. Loss before this step: data term 0.833; penalty $\frac{1}{12}(0.495^2 + 0.235^2) = 0.025$; total 0.858.

$\theta_0 = -0.061 - 0.1(0.318) = -0.092$; $\theta_1 = 0.9833(0.495) - 0.1(1.980) = 0.487 - 0.198 = 0.289$;
 $\theta_2 = 0.9833(-0.235) - 0.1(2.420) = -0.231 - 0.242 = -0.473$.

Loss after iteration 2: data term 0.755, total 0.780.

Interpretation. Loss: $1.078 \rightarrow 0.858 \rightarrow 0.780$, decreasing each time — the learning rate is appropriate. Iteration 1 overshoot on θ_1 (all h jumped above 0.68, even for the "no" rows), and iteration 2 corrected by pulling θ_1 down and θ_2 further negative: the model is learning "high CGPA helps, high IQ (as coded here) hurts". The L2 term shaves about 1.7% off the weights each step, keeping them small.

STEP 4 (C)

(i) $\theta_1 = 0.289$: each additional CGPA point multiplies the odds of an offer by $e^{0.289} = 1.34$, other things equal — a positive influence.

(ii) $z = -0.092 + 0.289(9) - 0.473(5) = -0.092 + 2.601 - 2.365 = 0.144$; $h = \frac{1}{1 + e^{-0.144}} = 0.536$. $z > 0 \Rightarrow$ **offered**, but only just (54%) — after two iterations the model is far from converged.

STEP 5 (D)

With $\alpha = 0.8$ the first step multiplies the gradient by eight times as much: the weights leap to about $(0.2, 2.3, 2.2)$, every z becomes ≈ 30 , every h saturates at 1, and the cross-entropy on the three "no" rows explodes ($-\log(1 - h) \rightarrow$ very large): the loss *increases* sharply and then oscillates. And $\lambda = 10$ with $m = 6$ makes the shrink factor $1 - 0.8 \times 10/6 = -0.33$ — negative — which flips the sign of every weight each step. Diagnosis: learning rate far too large for unscaled features, and an unreasonably large λ ; standardise the features and use $\alpha \approx 0.1$, $\lambda \lesssim 1$.

Two iterations: $(-0.061, 0.495, -0.235)$ then $(-0.092, 0.289, -0.473)$; **loss** $1.078 \rightarrow 0.858 \rightarrow 0.780$; **odds ratio** 1.34; **candidate** (9, 5) **offered** at 0.54; **the slide's** $\alpha = 0.8$, $\lambda = 10$ **diverges**.

WHERE MARKS ARE LOST Regularising θ_0 ; using the shrink factor $(1 - \alpha\lambda)$ when the cost has $\lambda/2m$ (state your convention); reporting the loss without the penalty term when the question says "regularised".

Problem 4.4 · Scoring probabilities with cross-entropy

NUMERICAL · 6 MARKS

Two classifiers output probabilities of "positive" for the same four test cases with true labels $y = (1, 0, 1, 0)$:

	case 1 ($y = 1$)	case 2 ($y = 0$)	case 3 ($y = 1$)	case 4 ($y = 0$)
Model P	0.90	0.20	0.70	0.40
Model Q	0.95	0.05	0.90	0.90

- Compute the cross-entropy (log-loss) J of each model.
- Compute the accuracy of each model at threshold 0.5. Which metric prefers which model, and why do they disagree?
- Compute the mean squared error $\frac{1}{m} \sum (h - y)^2$ for both. Why is squared error nevertheless a poor training objective for logistic regression?

WHAT IT TESTS Log-loss arithmetic, its punishment of confident mistakes, and the argument against squared error.

HOW TO ATTEMPT

- Per case: $-\ln h$ if $y = 1$, $-\ln(1 - h)$ if $y = 0$; average.
- Threshold, count correct.
- Squared error per case; the convexity argument.

ANSWERS AT A GLANCE $J_P = 0.299$, $J_Q = 0.628$; accuracy $P = 100\%$, $Q = 75\%$; MSE $P = 0.075$, $Q = 0.206$; log-loss punishes Q's confident mistake ($-\ln 0.1 = 2.30$).

STEP 1 (A) MODEL P

Case 1: $-\ln 0.90 = 0.105$; case 2: $-\ln(1 - 0.20) = -\ln 0.80 = 0.223$; case 3: $-\ln 0.70 = 0.357$; case 4: $-\ln 0.60 = 0.511$.

$$J_P = \frac{0.105 + 0.223 + 0.357 + 0.511}{4} = \frac{1.196}{4} = 0.299.$$

STEP 2 (A) MODEL Q

Case 1: $-\ln 0.95 = 0.051$; case 2: $-\ln 0.95 = 0.051$; case 3: $-\ln 0.90 = 0.105$; case 4: $-\ln(1 - 0.90) = -\ln 0.10 = 2.303$.

$$J_Q = \frac{0.051 + 0.051 + 0.105 + 2.303}{4} = \frac{2.510}{4} = 0.628.$$

STEP 3 (B)

Threshold 0.5: P predicts (1, 0, 1, 0) — all four correct, accuracy 100%. Q predicts (1, 0, 1, 1) — case 4 wrong, accuracy 75%. Both metrics prefer P here; but note that Q is *more confident and right* on three cases and P's accuracy hides how close case 4 (0.40) was. The general disagreement: accuracy only sees which side of 0.5 a probability falls, log-loss sees *how* confident it was. A model that is right on every case but always says 0.51 has perfect accuracy and a mediocre log-loss; a model wrong once with total confidence is destroyed by log-loss ($-\ln 0.1 = 2.3$ dwarfs everything else in J_Q) while accuracy docks it just one case.

STEP 4 (C)

$$\text{MSE}_P = \frac{0.01 + 0.04 + 0.09 + 0.16}{4} = \frac{0.30}{4} = 0.075; \text{MSE}_Q = \frac{0.0025 + 0.0025 + 0.01 + 0.81}{4} = \frac{0.825}{4} = 0.206.$$

Squared error still ranks the models the same way, but as a *training objective* for a sigmoid model it is non-convex in θ (local minima and flat plateaus where the sigmoid saturates), so gradient descent can stall or stop in the wrong valley; and its maximum penalty per case is 1, so a confident mistake is punished far too gently compared with the $-\ln$ cost that goes to infinity. Cross-entropy is convex and gives the clean gradient $(h - y)x_j$.

$J_P = 0.299 < J_Q = 0.628$; **accuracy 1.0 vs 0.75; MSE 0.075 vs 0.206; log-loss for training because it is convex and punishes confident errors.**

WHERE MARKS ARE LOST Using $-\ln h$ for every case regardless of y ; base-10 logs without saying so; explaining "why not squared error" only by "it is worse", without convexity.

Problem 4.5 · The gradient that matches

CONCEPTUAL · 7 MARKS

- Show that the sigmoid $g(z) = (1 + e^{-z})^{-1}$ satisfies $g'(z) = g(z)(1 - g(z))$.
- For one training example with cost $c = -y \ln h - (1 - y) \ln(1 - h)$, $h = g(\theta^T \mathbf{x})$, derive $\partial c / \partial \theta_j$ and simplify it to $(h - y)x_j$.
- Write the resulting batch update for m examples and explain in one sentence why it has the same form as linear regression's update even though the cost functions are different.
- Why does logistic regression have no closed-form solution?

WHAT IT TESTS The two derivations behind the update rule and the "same form" insight.

HOW TO ATTEMPT

- Differentiate $(1 + e^{-z})^{-1}$ by the chain rule, then rewrite in terms of g .
- Chain: $\partial c / \partial h \cdot \partial h / \partial z \cdot \partial z / \partial \theta_j$; simplify.
- Average over examples.
- The zero-gradient equations are non-linear in θ .

ANSWERS AT A GLANCE $g' = g(1 - g)$; $\partial c / \partial \theta_j = (h - y)x_j$; update $\theta_j \leftarrow \theta_j - \frac{\alpha}{m} \sum (h^{(i)} - y^{(i)})x_j^{(i)}$; no closed form because $\sum (g(\theta^T \mathbf{x}^{(i)}) - y^{(i)})\mathbf{x}^{(i)} = \mathbf{0}$ is non-linear in θ .

STEP 1 (A)

$$g'(z) = -(1 + e^{-z})^{-2} \cdot \frac{d}{dz}(1 + e^{-z}) = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2}.$$

Split the fraction: $\frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}}$. The first factor is $g(z)$; the second is $\frac{(1 + e^{-z}) - 1}{1 + e^{-z}} = 1 - g(z)$. Hence $g' = g(1 - g)$.

STEP 2 (B)

$$\frac{\partial c}{\partial h} = -\frac{y}{h} + \frac{1-y}{1-h}; \quad \frac{\partial h}{\partial z} = h(1-h); \quad \frac{\partial z}{\partial \theta_j} = x_j.$$

Chain: $\frac{\partial c}{\partial \theta_j} = \left(-\frac{y}{h} + \frac{1-y}{1-h}\right) h(1-h) x_j = (-y(1-h) + (1-y)h) x_j.$

Expand the bracket: $-y + yh + h - yh = h - y$. Therefore $\frac{\partial c}{\partial \theta_j} = (h - y) x_j.$

STEP 3 (C)

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}, \text{ all } j \text{ simultaneously, with } h_{\theta} = g(\boldsymbol{\theta}^T \mathbf{x}).$$

It matches linear regression's update because the cross-entropy cost was chosen so that its derivative with respect to h , $\frac{h-y}{h(1-h)}$, exactly cancels the sigmoid's derivative $h(1-h)$ — leaving "error times feature" in both cases; the only difference is what h_{θ} is.

STEP 4 (D)

Setting the gradient to zero gives $\sum_i (g(\boldsymbol{\theta}^T \mathbf{x}^{(i)}) - y^{(i)}) \mathbf{x}^{(i)} = \mathbf{0}$. Because g is non-linear, these are non-linear (transcendental) equations in $\boldsymbol{\theta}$ with no algebraic solution — unlike least squares, where $\mathbf{X}^T \mathbf{X} \boldsymbol{\theta} = \mathbf{X}^T \mathbf{y}$ is linear. Hence iterative methods: gradient descent or Newton's method.

$g' = g(1-g); (h-y)x_j$; same form because the cost's derivative cancels the sigmoid's; no closed form because the normal equations are non-linear.

WHERE MARKS ARE LOST Losing the minus sign in $\partial c / \partial h$; not showing the cancellation line; claiming "no closed form" without a reason.

MODULE 4 · PART B

Counting Right and Wrong

Problems 4.6–4.8: confusion-matrix metrics from counts, from a scenario, and from raw predictions.

Problem 4.6 · Four counts, six metrics

NUMERICAL · 6 MARKS

A model screens 1,000 insurance claims for fraud (fraud = positive). Results: $TP = 40$, $FN = 10$, $FP = 30$, $TN = 920$.

- Draw the confusion matrix.
- Compute accuracy, error rate, precision, recall (sensitivity), specificity, FPR and F_1 .
- In words: what does the precision of this model tell the investigations team, and what does the recall tell them?
- If the company can only investigate 50 claims a month, which metric should the model be tuned for, and in which direction should the threshold move?

WHAT IT TESTS Every formula on the metrics slide, plus the ability to explain two of them to a non-technical reader.

HOW TO ATTEMPT

- Rows actual, columns predicted; totals.
- Apply the formulas one per line.
- Precision = "of those flagged"; recall = "of the real frauds".

ANSWERS AT A GLANCE Accuracy 0.96, error 0.04, precision 0.571, recall 0.80, specificity 0.968, FPR 0.032, $F_1 = 0.667$. Investigation budget $\Rightarrow$ raise precision by raising the threshold.

STEP 1 (A)

Actual \ Predicted	Fraud	Not fraud	Total
Fraud	40 (TP)	10 (FN)	50
Not fraud	30 (FP)	920 (TN)	950
Total	70	930	1000

STEP 2 (B)

$$\text{Accuracy} = \frac{40 + 920}{1000} = 0.960; \text{error rate} = 1 - 0.960 = 0.040.$$

$$\text{Precision} = \frac{40}{40 + 30} = \frac{40}{70} = 0.571.$$

$$\text{Recall} = \text{sensitivity} = \frac{40}{40 + 10} = \frac{40}{50} = 0.800.$$

$$\text{Specificity} = \frac{920}{920 + 30} = \frac{920}{950} = 0.968; \text{FPR} = \frac{30}{950} = 0.032 = 1 - 0.968.$$

$$F_1 = \frac{2(0.571)(0.800)}{0.571 + 0.800} = \frac{0.914}{1.371} = 0.667.$$

STEP 3 (C)

Precision 57%: of the 70 claims the model flags, about 4 in 7 really are fraud — 30 investigations a month will be wasted on honest claims. **Recall 80%:** of the 50 real frauds, the model catches 40 and lets 10 through unflagged.

STEP 4 (D)

With capacity for only 50 investigations, the 70 flags already exceed the budget, so the model should be tuned for **precision** (make each flag count). **Raise the threshold:** fewer claims cross it, the ones that do are more certainly fraud, precision rises — at the cost of recall (more real frauds slip through). That trade-off is the ROC / precision–recall curve of Part C.

0.96/0.04/0.571/0.80/0.968/0.032/0.667; tune for precision by raising the threshold.

WHERE MARKS ARE LOST Swapping precision and recall denominators; F_1 as the arithmetic mean; leaving (c) as formulas instead of sentences.

Problem 4.7 · The 99% model that catches nothing

SCENARIO · 6 MARKS

A payment platform processes 5,000 transactions a day, of which about 50 are fraudulent. Two models are evaluated on one day's data. **Model A** predicts "genuine" for every transaction. **Model B** produces $TP = 45$, $FN = 5$, $FP = 200$, $TN = 4750$.

- Compute the accuracy of both models. Which looks better?
- Compute precision, recall and F_1 for both. Which is better?
- Explain why accuracy is misleading here, and state when accuracy is a reasonable metric.
- The fraud team says a missed fraud costs ₹20,000 and a false alarm costs ₹200 in review time. Compute the daily cost of each model.

WHAT IT TESTS The accuracy paradox with numbers, the right metrics for a rare class, and turning a confusion matrix into money.

HOW TO ATTEMPT

- Model A: $TP = 0$, $FN = 50$, $FP = 0$, $TN = 4950$.
- Formulas; note precision undefined for A.
- Cost = $20,000 FN + 200 FP$.

ANSWERS AT A GLANCE Accuracy A 99.0%, B 95.9%. Recall A 0, B 0.90; precision B 0.184; F_1 B 0.305 (A: 0). B is better. Cost A ₹10,00,000/day; B ₹1,40,000/day.

STEP 1 (A)

Model A: $TP = 0$, $FN = 50$, $FP = 0$, $TN = 4950$. Accuracy = $4950/5000 = 0.990$.

Model B: accuracy = $(45 + 4750)/5000 = 4795/5000 = 0.959$. By accuracy, A "looks" better.

STEP 2 (B)

A: recall = $0/50 = 0$; precision = $0/0$ — undefined (it flags nothing); $F_1 = 0$.

$$\text{B: precision} = \frac{45}{45 + 200} = \frac{45}{245} = 0.184; \text{recall} = \frac{45}{50} = 0.900; F_1 = \frac{2(0.184)(0.900)}{1.084} = 0.305.$$

B is better: it catches 90% of frauds; A catches none. B's low precision (many false alarms) is the price of that recall and can be worked on.

STEP 3 (C)

With 99% of transactions genuine, a model that never says "fraud" is automatically 99% accurate — accuracy is dominated by the majority class and is blind to the rare class we actually care about. Accuracy is reasonable only when the classes are roughly **balanced** and the two kinds of mistake cost about the same. For a rare, important class use recall, precision, F_1 , or ROC/AUC.

STEP 4 (D)

Cost A = $20,000 \times 50 + 200 \times 0 = 10,00,000$ rupees per day.

Cost B = $20,000 \times 5 + 200 \times 200 = 1,00,000 + 40,000 = 1,40,000$ rupees per day.

The "less accurate" model saves about ₹8.6 lakh a day. The costs of the two mistakes, not accuracy, decide.

A: acc 0.99, recall 0; B: acc 0.959, precision 0.184, recall 0.90, $F_1 = 0.305$; B wins and costs 7× less per day.

WHERE MARKS ARE LOST Reporting A's precision as 0 or 1 instead of undefined; forgetting to include FP review costs; concluding from accuracy alone.

Problem 4.8 · From twelve predictions to a matrix

NUMERICAL · 6 MARKS

A classifier's output on twelve test emails (P = spam, N = not spam):

Email	1	2	3	4	5	6	7	8	9	10	11	12
Actual	P	P	N	P	N	N	P	N	P	N	N	P
Predicted	P	N	N	P	P	N	P	N	P	N	P	P

- Build the confusion matrix (spam = positive).
- Compute accuracy, precision, recall, specificity and F_1 .
- For a spam filter, which of FP and FN is the more serious mistake? Which metric should therefore be high, and is this model good at it?

WHAT IT TESTS Building the matrix from raw labels — the most error-prone step in exams — then the application-specific judgement.

HOW TO ATTEMPT

- Go email by email and tally into TP / FN / FP / TN. Check the four cells add to 12.
- Formulas.
- Spam: FP (real mail lost) is worse $\Rightarrow$ want high precision / low FPR.

ANSWERS AT A GLANCE $TP = 5, FN = 1, FP = 2, TN = 4$; accuracy 0.75, precision 0.714, recall 0.833, specificity 0.667, $F_1 = 0.769$. Spam filters need low FP — precision 0.71 is not good enough.

STEP 1 (A) TALLY

Actual P, predicted P (TP): emails 1, 4, 7, 9, 12 → 5.

Actual P, predicted N (FN): email 2 → 1.

Actual N, predicted P (FP): emails 5, 11 → 2.

Actual N, predicted N (TN): emails 3, 6, 8, 10 → 4. Check: $5 + 1 + 2 + 4 = 12$ ✓.

Actual \ Predicted	P	N
P	5 (TP)	1 (FN)
N	2 (FP)	4 (TN)

STEP 2 (B)

$$\text{Accuracy} = \frac{5 + 4}{12} = 0.750; \text{precision} = \frac{5}{5 + 2} = 0.714; \text{recall} = \frac{5}{5 + 1} = 0.833; \text{specificity} = \frac{4}{4 + 2} = 0.667; F_1 = \frac{2(0.714)(0.833)}{1.548} = 0.769.$$

STEP 3 (C)

A false positive sends a genuine email (perhaps an offer letter) to the spam folder — usually far worse than a false negative, which merely lets one spam into the inbox. So the filter should have **high precision** (equivalently a low FPR / high specificity). This model's precision is 0.71 — two of seven flagged emails were genuine — and its specificity is only 0.67: **not good enough** for a spam filter; raise the threshold to trade some recall for precision.

5/1/2/4; 0.75/0.714/0.833/0.667/0.769; spam needs low FP, so this precision is inadequate.

WHERE MARKS ARE LOST Miscounting one email (always check the total); putting FP and FN in the wrong cells; arguing that FN is worse for spam.

MODULE 4 · PART C

Every Threshold at Once: ROC and AUC

Problems 4.9–4.11: constructing the curve, computing the area, comparing classifiers, choosing a threshold.

Problem 4.9 · Draw the ROC from eight scores

NUMERICAL · 9 MARKS

A classifier scores eight test instances (higher = more likely positive):

Instance	1	2	3	4	5	6	7	8
Score	0.92	0.85	0.80	0.70	0.70	0.55	0.45	0.30
True class	P	P	N	P	N	N	P	N

- Construct the ROC table: for each distinct threshold t ("positive if score $\geq t$ ") give TP, FP, TN, FN, TPR and FPR. Treat tied scores correctly.
- Sketch the ROC curve (list the points in order).
- Compute the AUC using the trapezoidal rule.
- What TPR and FPR does the default threshold 0.5 give? Which threshold would you pick if a false positive is very costly?

WHAT IT TESTS The ROC recipe including the tie, the trapezoid AUC, and reading operating points off the curve.

HOW TO ATTEMPT

- $P = 4$, $N = 4$; sort (already sorted); thresholds at distinct scores; the two 0.70s enter together.
- $TPR = TP/4$, $FPR = FP/4$.
- Sum $(x_{i+1} - x_i)(y_i + y_{i+1})/2$ over consecutive points.

ANSWERS AT A GLANCE Points

$(0, 0)$, $(0, 0.25)$, $(0, 0.5)$, $(0.25, 0.5)$, $(0.5, 0.75)$, $(0.75, 0.75)$, $(0.75, 1)$, $(1, 1)$; $AUC = 0.719$.

Threshold 0.5: TPR 0.75, FPR 0.75; for costly FP choose $t = 0.85$ (TPR 0.5, FPR 0).

STEP 1 (A)

$P = 4$ positives (instances 1, 2, 4, 7), $N = 4$ negatives (3, 5, 6, 8).

Threshold t	Called positive	TP	FP	TN	FN	$TPR = TP/4$	$FPR = FP/4$
≥ 0.92	{1}	1	0	4	3	0.25	0
≥ 0.85	{1, 2}	2	0	4	2	0.50	0
≥ 0.80	{1, 2, 3}	2	1	3	2	0.50	0.25
≥ 0.70 (tie: 4 and 5 together)	{1, ..., 5}	3	2	2	1	0.75	0.50
≥ 0.55	{1, ..., 6}	3	3	1	1	0.75	0.75
≥ 0.45	{1, ..., 7}	4	3	1	0	1.00	0.75
≥ 0.30	all	4	4	0	0	1.00	1.00

At $t = 0.70$ both instances 4 (P) and 5 (N) cross the threshold at once, so the curve moves diagonally from $(0.25, 0.5)$ to $(0.5, 0.75)$ — one cannot admit instance 4 without instance 5.

STEP 2 (B)

Starting at $(0, 0)$ (threshold above every score): $(0, 0) \rightarrow (0, 0.25) \rightarrow (0, 0.5) \rightarrow (0.25, 0.5) \rightarrow (0.5, 0.75) \rightarrow (0.75, 0.75) \rightarrow (0.75, 1.0) \rightarrow (1, 1)$. Plot FPR on the horizontal axis, TPR on the vertical, join the points, and add the diagonal for reference.

STEP 3 (C)

Only segments with a change in FPR contribute: $(x_{i+1} - x_i) \cdot \frac{y_i + y_{i+1}}{2}$.

$(0, 0.5) \rightarrow (0.25, 0.5)$: $0.25 \times \frac{0.5+0.5}{2} = 0.25 \times 0.5 = 0.125$.

$(0.25, 0.5) \rightarrow (0.5, 0.75)$: $0.25 \times \frac{0.5+0.75}{2} = 0.25 \times 0.625 = 0.15625$ (the tie's diagonal — a genuine trapezoid).

$(0.5, 0.75) \rightarrow (0.75, 0.75)$: $0.25 \times 0.75 = 0.1875$.

$(0.75, 1.0) \rightarrow (1, 1)$: $0.25 \times 1.0 = 0.25$.

$AUC = 0.125 + 0.15625 + 0.1875 + 0.25 = \mathbf{0.719}$ — clearly better than random (0.5), well short of perfect.

STEP 4 (D)

Threshold 0.5 admits instances with score ≥ 0.5 , i.e. the same set as $t = 0.55$: $TPR = 0.75$, $FPR = 0.75$ — a point on the diagonal, no better than guessing at that operating point. If false positives are very costly, pick $t = 0.85$: $FPR = 0$ with $TPR = 0.5$ — half the positives caught with no false alarms.

ROC table as above; AUC 0.719; $t = 0.5 \Rightarrow (0.75, 0.75)$; costly FP $\Rightarrow t = 0.85$.

WHERE MARKS ARE LOST Splitting the tie into two steps; forgetting the $(0, 0)$ start; adding rectangles for vertical moves; using $(y_i + y_{i+1})/2$ but the wrong Δx .

Problem 4.10 · Two curves, one operating point

NUMERICAL · 6 MARKS

Two loan-default models were evaluated at several thresholds; their ROC points (FPR, TPR) are:

Model A	(0, 0)	(0.1, 0.50)	(0.2, 0.70)	(0.4, 0.85)	(0.7, 0.95)	(1, 1)
Model B	(0, 0)	(0.1, 0.30)	(0.2, 0.50)	(0.4, 0.80)	(0.7, 0.97)	(1, 1)

- Compute the AUC of each model by the trapezoidal rule.
- The bank will operate at $FPR \leq 0.1$ (it refuses to reject more than 10% of good customers). Which model is better *there*?
- Is there a region where B is better than A? What does that tell you about comparing models by AUC alone?
- Explain what the points $(0, 0)$ and $(1, 1)$ represent, and what an AUC of 0.5 would mean.

WHAT IT TESTS AUC arithmetic from given points and the "operating point vs global AUC" judgement from the model-comparison slide.

HOW TO ATTEMPT

1. Five trapezoids per model.
2. Compare TPR at FPR = 0.1.
3. Look for a crossing.

ANSWERS AT A GLANCE $AUC_A = 0.803$, $AUC_B = 0.746$; at FPR 0.1 A (TPR 0.50 vs 0.30); B overtakes A between FPR 0.4 and 0.7; AUC alone hides where each model wins.

STEP 1 (A) MODEL A

$$0.1 \cdot \frac{0+0.5}{2} = 0.025; 0.1 \cdot \frac{0.5+0.7}{2} = 0.060; 0.2 \cdot \frac{0.7+0.85}{2} = 0.155; 0.3 \cdot \frac{0.85+0.95}{2} = 0.270; 0.3 \cdot \frac{0.95+1}{2} = 0.2925.$$

$$AUC_A = 0.025 + 0.060 + 0.155 + 0.270 + 0.2925 = 0.8025.$$

STEP 2 (A) MODEL B

$$0.1 \cdot \frac{0+0.3}{2} = 0.015; 0.1 \cdot \frac{0.3+0.5}{2} = 0.040; 0.2 \cdot \frac{0.5+0.8}{2} = 0.130; 0.3 \cdot \frac{0.8+0.97}{2} = 0.2655; 0.3 \cdot \frac{0.97+1}{2} = 0.2955.$$

$$AUC_B = 0.015 + 0.040 + 0.130 + 0.2655 + 0.2955 = 0.746.$$

STEP 3 (B)

At FPR = 0.1: A detects 50% of defaulters, B only 30%. **Model A** is better at the bank's operating point.

STEP 4 (C)

At FPR = 0.7, B's TPR (0.97) exceeds A's (0.95): the curves cross somewhere between FPR 0.4 and 0.7, and B is (slightly) better for large FPR. So no model dominates everywhere; AUC summarises the whole curve and can hide that one model wins in the region you actually use. Compare at the operating point when it is known; use AUC when it is not.

STEP 5 (D)

(0, 0): the threshold is so high that nothing is called positive — no detections, no false alarms. (1, 1): everything is called positive. $AUC = 0.5$ is the diagonal: the classifier's scores carry no information — a randomly chosen defaulter is no more likely to score above a randomly chosen good customer than below.

AUC 0.803 vs 0.746; A wins at $FPR \leq 0.1$; B wins at high FPR; AUC alone can mislead.

WHERE MARKS ARE LOST Using rectangles instead of trapezoids for sloped segments; forgetting the last segment to (1, 1); answering (b) with the AUC.

Problem 4.11 · Set the threshold for the job

SCENARIO · 6 MARKS

One logistic model, evaluated on 200 test patients (40 with the disease), at several thresholds on its output probability:

Threshold t	TP	FN	FP	TN
0.9	12	28	1	159
0.7	24	16	5	155
0.5	32	8	16	144
0.3	38	2	40	120
0.1	40	0	96	64

- Compute TPR and FPR at each threshold, and precision at $t = 0.3$ and $t = 0.7$.
- The model is to be used as a **screening test** (a positive result leads to a cheap confirmatory test). Choose a threshold and justify it.
- The same model is later used to decide who gets an expensive, risky treatment *without* further tests. Choose a threshold now and justify it.
- What does this pair of answers illustrate about "the best classifier"?

WHAT IT TESTS Threshold selection from a table — the ROC "usage" slide — and the costs-of-mistakes reasoning (screening wants low FN; treatment wants low FP).

HOW TO ATTEMPT

- TPR = $TP/40$, FPR = $FP/160$.
- Screening: minimise FN → low threshold.
- Treatment: minimise FP → high threshold.

ANSWERS AT A GLANCE TPR/FPR: 0.30/0.006, 0.60/0.031, 0.80/0.10, 0.95/0.25, 1.00/0.60. Screening: $t = 0.3$ (or 0.1). Treatment: $t = 0.9$ (or 0.7). One model, many classifiers.

STEP 1 (A)

t	TPR = $TP/40$	FPR = $FP/160$	Precision = $TP/(TP + FP)$
0.9	$12/40 = 0.30$	$1/160 = 0.006$	$12/13 = 0.92$
0.7	$24/40 = 0.60$	$5/160 = 0.031$	$24/29 = 0.83$
0.5	$32/40 = 0.80$	$16/160 = 0.10$	$32/48 = 0.67$
0.3	$38/40 = 0.95$	$40/160 = 0.25$	$38/78 = 0.49$
0.1	$40/40 = 1.00$	$96/160 = 0.60$	$40/136 = 0.29$

STEP 2 (B)

Screening: a missed case (FN) is dangerous, a false alarm (FP) only costs a cheap confirmatory test. Choose a **low threshold** so recall is high: $t = 0.3$ catches 95% of cases with 40 false alarms out of 160 (precision 0.49 — half of the referrals will be cleared by the confirmatory test, which is acceptable). $t = 0.1$ catches everyone but sends 96 healthy people for confirmation; choose it only if the confirmatory test is essentially free.

STEP 3 (C)

Treating on the model's word alone: a false positive means an expensive, risky treatment given to a healthy person. Choose a **high threshold**: $t = 0.9$ treats 13 people of whom 12 truly have the disease (precision 0.92, FPR 0.006), at the cost of missing 70% of cases — so it must be paired with another route (e.g. the screening pathway) for those missed. $t = 0.7$ (precision 0.83, recall 0.60) is the compromise if missing 70% is unacceptable.

STEP 4 (D)

The same trained model is a whole family of classifiers, one per threshold — the ROC curve. "Best" depends on the relative cost of FP and FN, which is a property of the application, not of the model. Conservative and aggressive settings can coexist in one deployment.

Table as above; screening $t = 0.3$ (high recall); treatment $t = 0.9$ (high precision); the threshold, not the model, sets the trade-off.

WHERE MARKS ARE LOST FPR with the wrong denominator (use the 160 negatives); choosing a threshold without naming the mistake it avoids; claiming $t = 0.5$ is "standard".

MODULE 4 · PART D

Beyond Two Balanced Classes

Problems 4.12–4.13: imbalance and multi-class strategies, and curved boundaries.

Problem 4.12 · Rare classes, many classes

CONCEPTUAL · 6 MARKS

- A defect detector is trained on 20,000 images of which 150 show defects. Name three remedies for the class imbalance from the slides and say, for each, what changes and what must stay unchanged.
- A logistic-regression system must classify support tickets into six categories. How many binary classifiers are trained under one-vs-all and under one-vs-one? Explain how each makes its final prediction.
- Under one-vs-all with classes {Billing, Delivery, Returns}, the three classifiers output $h^{(1)} = 0.30$, $h^{(2)} = 0.72$, $h^{(3)} = 0.55$ for a new ticket. What is the prediction, and why is it not a problem that the three numbers do not sum to 1?
- Why does scikit-learn use one-vs-one for SVMs but one-vs-all for logistic regression?

WHAT IT TESTS The imbalance slide and the multi-class slides, including the arithmetic and the argmax rule.

HOW TO ATTEMPT

- SMOTE / resampling (training only), metric change, algorithm change.
- K vs $K(K-1)/2$; argmax vs vote.
- Pick the largest.

ANSWERS AT A GLANCE (a) resample the training set (SMOTE / under-sample), switch metrics to recall / precision / F_1 / AUC, try SVMs or trees — test set untouched. (b) OvA 6, OvO 15. (c) Delivery. (d) SVM training cost grows fast with n , so many small problems are cheaper.

STEP 1 (A)

Generate synthetic samples — oversample the defect class with SMOTE (new points interpolated between a defect image and its nearest defect neighbours), or under-sample the 19,850 normals.

Changes: the *training* class balance. Unchanged: the validation and test sets, which must keep the real 0.75% rate.

Change the performance metric — report recall, precision, F_1 or ROC/AUC instead of accuracy (which would be 99.25% for "never a defect"). Changes: how models are selected and reported. Unchanged: the data.

Try different algorithms — support vector machines and tree-based methods (with class weights) cope better with imbalance. Changes: the learner. Unchanged: the evaluation protocol.

STEP 2 (B)

One-vs-all: $K = 6$ classifiers, each trained on all tickets with "its" class as 1 and the other five as 0.

Prediction: run all six, output the class whose classifier gives the highest probability, $\hat{y} = \arg \max_i h^{(i)}(\mathbf{x})$.

One-vs-one: $\frac{K(K-1)}{2} = \frac{6 \times 5}{2} = 15$ classifiers, one per pair of classes, each trained only on tickets of its two classes. Prediction: each classifier votes for one of its two classes; the class with the most votes wins.

STEP 3 (C)

$\arg \max(0.30, 0.72, 0.55) = \text{classifier 2} \Rightarrow$ **Delivery**. The three classifiers were trained separately, each answering "is it my class vs everything else?", so each is a probability of a *different* event; they are not a distribution over the three classes and need not sum to 1. Only their ranking is used.

STEP 4 (D)

SVM training time grows badly with the number of training examples, so training 15 small classifiers on pairs of classes is cheaper than 6 classifiers on the full dataset; logistic regression trains cheaply on large sets, so the simpler one-vs-all with K models is preferred.

Three remedies (training-set only); OvA 6 / OvO 15; Delivery; SVM scales poorly with n .

WHERE MARKS ARE LOST Resampling the test set; OvO count as K^2 or $K(K-1)$; treating OvA outputs as needing to sum to 1.

Problem 4.13 · A boundary that is not a line

SCENARIO · 6 MARKS

A sensor flags a machine as faulty ($y = 1$) from two standardised readings x_1, x_2 . Engineers found that normal machines cluster near the origin, so the model uses squared features: $h_{\theta}(\mathbf{x}) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2)$ with $\theta = (-4, 0, 0, 1, 1)$.

- Write the decision boundary and describe its shape. Which region is "faulty"?
- Classify the readings $(1, 1)$, $(2, 2)$ and $(0, 2)$, giving z and the predicted class.
- The engineers change θ_4 to 4. What is the new boundary's shape? Classify $(1, 1)$ and $(2, 0)$ under it.
- Is this still a "linear" model? Is its cost still convex? Explain briefly.

WHAT IT TESTS The non-linear-boundary slide: reading a curve off θ , classifying, and the "linear in θ " idea carried over from regression.

HOW TO ATTEMPT

- $z = 0$; recognise a circle $x_1^2 + x_2^2 = 4$.
- Compute z ; sign gives the class.
- $x_1^2 + 4x_2^2 = 4$ is an ellipse.
- Linear in θ ; cross-entropy convex in θ .

ANSWERS AT A GLANCE (a) circle of radius 2 centred at the origin; outside = faulty. (b) $z = -2 \rightarrow 0$; $z = 4 \rightarrow 1$; $z = 0 \rightarrow 1$ (on the boundary). (c) ellipse $x_1^2 + 4x_2^2 = 4$; $(1, 1) \rightarrow z = 1 \rightarrow 1$; $(2, 0) \rightarrow z = 0 \rightarrow 1$ (boundary). (d) yes, linear in θ ; yes, convex.

STEP 1 (A)

$z = -4 + x_1^2 + x_2^2$. Boundary $z = 0 \iff x_1^2 + x_2^2 = 4$: a **circle of radius 2** centred at the origin. $z > 0$ outside the circle $\Rightarrow$ **faulty outside**, normal inside — matching the engineers' observation that normal machines sit near the origin.

STEP 2 (B)

$(1, 1)$: $z = -4 + 1 + 1 = -2 < 0 \Rightarrow$ class 0 (normal), $h = g(-2) = 0.12$.

$(2, 2)$: $z = -4 + 4 + 4 = 4 > 0 \Rightarrow$ class 1 (faulty), $h = g(4) = 0.98$.

$(0, 2)$: $z = -4 + 0 + 4 = 0 \Rightarrow$ exactly on the boundary, $h = 0.5$; by the " ≥ 0.5 " convention, class 1.

STEP 3 (C)

$z = -4 + x_1^2 + 4x_2^2$; boundary $x_1^2 + 4x_2^2 = 4$, i.e. $\frac{x_1^2}{4} + \frac{x_2^2}{1} = 1$: an **ellipse** with semi-axes 2 (along x_1) and 1 (along x_2) — the x_2 reading is now judged more strictly.

$(1, 1)$: $z = -4 + 1 + 4 = 1 > 0 \Rightarrow$ faulty (it was normal under the circle). $(2, 0)$: $z = -4 + 4 + 0 = 0 \Rightarrow$ on the boundary $\Rightarrow$ class 1.

STEP 4 (D)

Yes: the argument of the sigmoid is $\theta^T \phi(\mathbf{x})$ with fixed basis functions $\phi = (1, x_1, x_2, x_1^2, x_2^2)$ — **linear in θ** ; the curvature lives in the features, exactly as in polynomial regression. Hence the cross-entropy cost is still convex in θ , gradient descent has the same update $(\frac{1}{m} \sum (h - y)\phi_j)$ and finds the global minimum. Adding many polynomial terms buys arbitrarily complex boundaries, at the usual risk of overfitting — regularise.

Circle radius 2 (faulty outside); classes 0, 1, 1; ellipse $x_1^2 + 4x_2^2 = 4$ with $(1, 1)$ now faulty and $(2, 0)$ on the boundary; linear in θ , convex cost.

WHERE MARKS ARE LOST Describing the boundary as "a parabola"; wrong side for "faulty"; saying the model is non-linear "so the cost is non-convex".

MODULE 5 · CONTACT SESSIONS 7 & 8

Decision Trees

Impurity measures, information gain and ID3, Gini binary splits, continuous thresholds, gain ratio, pre- and post-pruning, MDL, overfitting.

Fourteen problems in four parts. One small loan-approval dataset (right) runs through most of the numerical problems, so building, continuing, predicting, evaluating, pruning and MDL-scoring all happen on a tree you have watched grow. All ten problem types on the slides are covered, with fresh data and a few new question shapes. The Module 5 formula card is in **Formula Cards** at the front. **The dataset:** Eight applications, 4 approved and 4 refused; used in Problems 5.2–5.4 and 5.6–5.8 (Problem 5.9 uses a hypothetical variant with eight binary attributes).

The running dataset — loan approvals

#	Income	Credit	Employed	Approve?
1	High	Good	Yes	Yes
2	High	Bad	Yes	Yes
3	Medium	Good	Yes	Yes
4	Medium	Bad	No	No
5	Low	Good	Yes	No
6	Low	Bad	No	No
7	Medium	Good	No	Yes
8	Low	Good	No	No

The problem map

#	Title	Type	Topic on the slides	Marks
5.1	Three impurities, five nodes	Numerical	Entropy, Gini, classification error	6
5.2	Grow the loan tree (ID3)	Numerical	Problem Type 1: information gain, root selection, full tree	10
5.3	Continue, predict, evaluate	Numerical	Problem Types 2, 3, 4	8
5.4	Gini and the three ways to cut a three-valued attribute	Numerical	Gini index; binary splits of nominal attributes (CART)	7
5.5	Where to cut "years at job"	Numerical	Problem Type 5: continuous attributes, threshold scan	8
5.6	The ID column and gain ratio	Numerical	Problem Type 6: split information, gain ratio	7
5.7	Stop early: pre-pruning	Scenario	Problem Type 8: gain threshold and minimum samples	6
5.8	Cut back: post-pruning with a prune set	Numerical	Problem Type 9: reduced-error pruning	7
5.9	Count the bits: MDL	Numerical	Problem Type 10: minimum description length	7
5.10	Reading an overfitting table	Scenario	Problem Type 7: model overfitting, tree size, remedies	6
5.11	Trees as rules, rules as trees	Conceptual	Hunt's algorithm; representation; stopping criteria	6
5.12	Test conditions for every attribute type	Scenario	Binary / nominal / ordinal / continuous splits	5
5.13	Three classes in one node	Numerical	Multi-class entropy and Gini; gain with three classes	6
5.14	When to pick a tree	Conceptual	Advantages, disadvantages, axis-parallel boundaries, applications	6

MODULE 5 · PART A

Growing the Tree

Problems 5.1–5.3: impurity, information gain and ID3 on the loan data, then continuing, predicting and evaluating.

Problem 5.1 · Three impurities, five nodes

NUMERICAL · 6 MARKS

Five nodes of a binary tree hold the following (class 0, class 1) counts: $N_1 = (8, 0)$, $N_2 = (6, 2)$, $N_3 = (5, 3)$, $N_4 = (4, 4)$, $N_5 = (1, 7)$.

- Compute the entropy, Gini index and classification error of each node.
- Which node is purest, which is most mixed? Why do all three measures agree on those two?
- N_3 and N_5 : which is purer by entropy? by error? Explain the one case where the measures rank differently in general and why that makes error a poor criterion for growing trees.

WHAT IT TESTS Fluency with the three impurity formulas and the reason entropy/Gini are preferred.

HOW TO ATTEMPT

- Convert counts to proportions; apply the three formulas.
- All are 0 when pure and maximal at 50/50.
- Error is piecewise linear — it cannot see a split that leaves the majority class unchanged.

ANSWERS AT A GLANCE N_1 : 0/0/0; N_2 : 0.811/0.375/0.25; N_3 : 0.954/0.469/0.375; N_4 : 1/0.5/0.5; N_5 : 0.544/0.219/0.125. Purest N_1 , most mixed N_4 .

Node	p_0, p_1	Entropy $-\sum p \log_2 p$	Gini $1 - \sum p^2$	Error $1 - \max p$
$N_1 (8, 0)$	1, 0	0	$1 - 1 = 0$	0
$N_2 (6, 2)$	0.75, 0.25	$0.311 + 0.5 = 0.811$	$1 - 0.5625 - 0.0625 = 0.375$	0.25
$N_3 (5, 3)$	0.625, 0.375	$0.424 + 0.531 = 0.954$	$1 - 0.391 - 0.141 = 0.469$	0.375
$N_4 (4, 4)$	0.5, 0.5	$0.5 + 0.5 = 1$	0.5	0.5
$N_5 (1, 7)$	0.125, 0.875	$0.375 + 0.169 = 0.544$	$1 - 0.016 - 0.766 = 0.219$	0.125

STEP 2 (B)

Purest: N_1 (all three measures 0). Most mixed: N_4 (entropy 1, Gini 0.5, error 0.5 — the maxima for two classes). All three measures are zero exactly when one class has all the records and largest at $p = 0.5$, so they must agree at the two extremes.

STEP 3 (C)

N_5 is purer than N_3 by every measure: entropy $0.544 < 0.954$, Gini $0.219 < 0.469$, error $0.125 < 0.375$. The measures differ when scoring a *split*. Take a parent (6, 2) — error 0.25, entropy 0.811, Gini 0.375 — and split it into children (4, 0) and (2, 2).

Error of the children: $\frac{4}{8}(0) + \frac{2}{8}(0.5) = 0.125$ — exactly the parent's value: **no improvement** recorded, although one child became pure.

Entropy of the children: $\frac{4}{8}(0) + \frac{4}{8}(1) = 0.5$ — a gain of $0.811 - 0.5 = 0.311$. Gini: $\frac{4}{8}(0) + \frac{4}{8}(0.5) = 0.25$ — a gain of 0.125.

Error is piecewise linear, so it only registers a change when a child's *majority class* changes; entropy and Gini are strictly concave and reward every move toward purity. That is why the slide says error is "less sensitive to changes in class proportions" and Gini or entropy are preferred for growing trees.

Table as above; purest N_1 , most mixed N_4 ; error is piecewise linear and blind to many proportion changes.

WHERE MARKS ARE LOST Forgetting the minus sign in entropy; Gini as $\sum p^2$; claiming error "always agrees" with entropy.

Problem 5.2 · Grow the loan tree (ID3)

NUMERICAL · 10 MARKS

Using the eight-row loan-approval dataset (opening page):

- Compute the entropy of the class column.
- Compute the information gain of Income, Credit and Employed. Which is the root?
- Continue the ID3 algorithm on any impure child until every leaf is pure. Draw the final tree.
- Write the tree as a disjunction of conjunctions (rules for Approve = Yes).

WHAT IT TESTS Problem Type 1 from scratch on unseen data — the full root computation and the recursion.

HOW TO ATTEMPT

- Class counts $\rightarrow$ entropy.
- For each attribute: table of (Yes, No) per value $\rightarrow$ child entropies $\rightarrow$ weighted sum $\rightarrow$ gain.
- Recurse on the impure child with the remaining attributes.

ANSWERS AT A GLANCE $H(S) = 1$; gains: Income 0.656, Employed 0.189, Credit 0.049 $\Rightarrow$ Income root; Medium branch split on Credit; rules: $(\text{Income} = \text{High}) \vee (\text{Income} = \text{Medium} \wedge \text{Credit} = \text{Good})$.

STEP 1 (A)

4 Yes, 4 No: $H(S) = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = 1$ bit.

STEP 2 (B) INCOME

High: rows 1, 2 $\rightarrow$ (2 Yes, 0 No), $H = 0$. Medium: rows 3, 4, 7 $\rightarrow$ (2, 1), $H = 0.918$. Low: rows 5, 6, 8 $\rightarrow$ (0, 3), $H = 0$.

Weighted = $\frac{2}{8}(0) + \frac{3}{8}(0.918) + \frac{3}{8}(0) = 0.344$. Gain(Income) = $1 - 0.344 = \mathbf{0.656}$.

STEP 3 (B) CREDIT

Good: rows 1, 3, 5, 7, 8 $\rightarrow$ (3 Yes, 2 No), $H = 0.971$. Bad: rows 2, 4, 6 $\rightarrow$ (1, 2), $H = 0.918$.

Weighted = $\frac{5}{8}(0.971) + \frac{3}{8}(0.918) = 0.607 + 0.344 = 0.951$. Gain(Credit) = $1 - 0.951 = \mathbf{0.049}$.

STEP 4 (B) EMPLOYED

Yes: rows 1, 2, 3, 5 $\rightarrow$ (3, 1), $H = 0.811$. No: rows 4, 6, 7, 8 $\rightarrow$ (1, 3), $H = 0.811$.

Weighted = $\frac{4}{8}(0.811) + \frac{4}{8}(0.811) = 0.811$. Gain(Employed) = $1 - 0.811 = \mathbf{0.189}$.

Root = Income ($0.656 > 0.189 > 0.049$).

STEP 5 (C) THE MEDIUM BRANCH

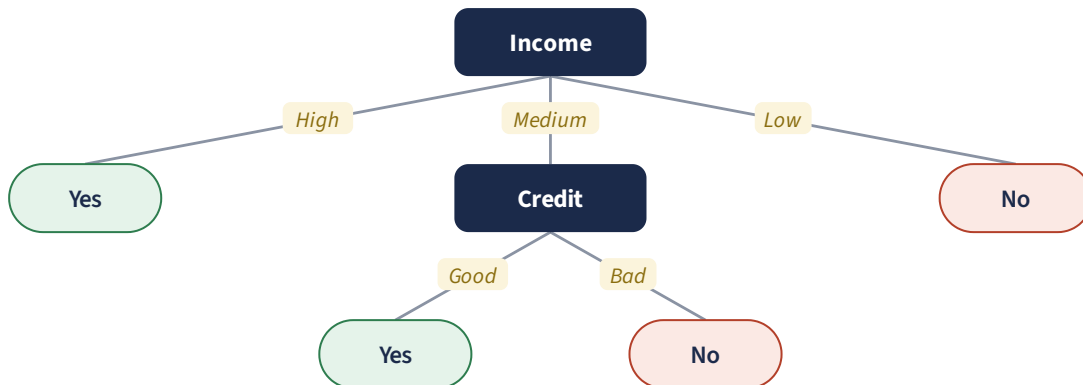
High → all Yes → leaf **Yes**. Low → all No → leaf **No**. Medium holds rows 3 (Good, Yes, Yes), 4 (Bad, No, No), 7 (Good, No, Yes): (2 Yes, 1 No), $H = 0.918$; remaining attributes Credit, Employed.

Credit: Good → rows 3, 7 (2, 0) $H = 0$; Bad → row 4 (0, 1) $H = 1$. Gain = $0.918 - 0 = \mathbf{0.918}$.

Employed: Yes → row 3 (1, 0) $H = 0$; No → rows 4, 7 (1, 1) $H = 1$. Weighted = $\frac{1}{3}(0) + \frac{2}{3}(1) = 0.667$.

Gain = $0.918 - 0.667 = 0.251$.

Split Medium on **Credit**: Good → Yes, Bad → No. Every leaf is pure; stop.



The finished ID3 tree.

STEP 6 (D)

Approve = Yes $\iff$ (Income = High) $\vee$ (Income = Medium $\wedge$ Credit = Good). Employed never appears — the tree found it unnecessary.

Root Income (gain 0.656); Medium → Credit; two rules for Yes.

WHERE MARKS ARE LOST Using 0.94 for a 2:1 split (it is 0.918); forgetting that Income is no longer a candidate inside its own branch; not stating the gains for the losing attributes.

Problem 5.3 · Continue, predict, evaluate

NUMERICAL · 8 MARKS

- (Type 2) A colleague started the tree with **Employed** at the root instead. Show the two children with their class counts, and state which one(s) must be split further and which attribute would be chosen next in the "Employed = No" branch.
- (Type 3) Using the correct tree from Problem 5.2, predict Approve for the applications (Medium, Bad, Yes), (High, Bad, No) and (Low, Good, Yes). Which attributes were consulted for each?
- (Type 4) Four new applications with known outcomes form a test set: (High, Bad, No → Yes), (Low, Good, Yes → No), (Medium, Good, No → No), (Low, Bad, Yes → No). Build the confusion matrix (Yes = positive), and compute test accuracy, precision and recall. Compare with training accuracy.

WHAT IT TESTS Problem Types 2–4: recognising an impure node, walking the tree, and scoring it on unseen data.

HOW TO ATTEMPT

1. Partition rows by Employed; compute gains inside the impure branch with the remaining attributes.
2. Follow root → leaf; note the path.
3. Tally TP/FN/FP/TN over the four test rows.

ANSWERS AT A GLANCE (a) Both children impure (3:1 and 1:3); in the No branch Income and Credit tie at gain 0.311. (b) No, Yes, No. (c) $TP = 1, FN = 0, FP = 1, TN = 2$: accuracy 0.75, precision 0.5, recall 1; training accuracy 1.0.

STEP 1 (A)

Employed = Yes: rows 1, 2, 3, 5 → (3 Yes, 1 No), impure. Employed = No: rows 4, 6, 7, 8 → (1 Yes, 3 No), impure. **Both** must be split further.

No branch (rows 4: Medium/Bad/No; 6: Low/Bad/No; 7: Medium/Good/Yes; 8: Low/Good/No), $H(1, 3) = 0.811$. Income: Medium (rows 4, 7: 1 Yes, 1 No) $H = 1$; Low (rows 6, 8: 0, 2) $H = 0$; weighted $\frac{2}{4}(1) = 0.5$; gain 0.311. Credit: Good (rows 7, 8: 1, 1) $H = 1$; Bad (rows 4, 6: 0, 2) $H = 0$; weighted 0.5; gain 0.311. A tie at 0.311 — ID3 may choose either Income or Credit; each leaves one impure (1, 1) child, which the other attribute then resolves.

Lesson: a poor root does not stop the algorithm — it just needs more levels. Employed at the root gives a deeper tree that still fits the training data perfectly.

STEP 2 (B)

(Medium, Bad, Yes): Income = Medium → Credit = Bad → **No**. Consulted: Income, Credit.

(High, Bad, No): Income = High → **Yes**. Consulted: Income only.

(Low, Good, Yes): Income = Low → **No**. Consulted: Income only.

STEP 3 (C)

Test row	True	Path	Predicted	Cell
(High, Bad, No)	Yes	High → Yes	Yes	TP
(Low, Good, Yes)	No	Low → No	No	TN
(Medium, Good, No)	No	Medium → Good → Yes	Yes	FP
(Low, Bad, Yes)	No	Low → No	No	TN

$TP = 1, FN = 0, FP = 1, TN = 2$. Accuracy = $3/4 = 0.75$; precision = $1/(1 + 1) = 0.5$; recall = $1/1 = 1.0$.

Training accuracy is $8/8 = 100\%$ (every leaf pure). The drop to 75% on new data — with the error on a Medium/Good applicant, the very rule learnt from only two training rows — is the usual gap between fit and generalisation; with eight training rows some overfitting is inevitable.

Both Employed-children impure; predictions No / Yes / No; test $TP=1, FN=0, FP=1, TN=2$, accuracy 0.75 vs training 1.0.

WHERE MARKS ARE LOST Splitting a pure child; walking the wrong branch; computing accuracy on the training rows when the test set is given.

MODULE 5 · PART B

Other Ways to Split

Problems 5.4–5.6: Gini with binary groupings, continuous thresholds, and gain ratio.

Problem 5.4 · Gini and the three ways to cut a three-valued attribute

NUMERICAL · 7
MARKS

A CART-style tree (binary splits, Gini index) is grown on the loan data. Consider the attribute **Income** $\in$ {High, Medium, Low}.

- Compute the Gini index of the parent node.
- List every binary grouping of Income's three values and compute the weighted Gini of each split.
- Which grouping is best? What is its Gini gain?
- For a nominal attribute with k values, how many binary groupings exist? Evaluate for $k = 3$ and $k = 5$.

WHAT IT TESTS Gini arithmetic and the CART binary-grouping idea from the marital-status slide.

HOW TO ATTEMPT

- Parent (4, 4).
- Groupings: {H} | {M, L}; {M} | {H, L}; {L} | {H, M}. Child counts from the table.
- Lowest weighted Gini wins.
- $2^{k-1} - 1$.

ANSWERS AT A GLANCE Parent Gini 0.5; splits 0.333, 0.467, 0.200; best {Low} vs {High, Medium}, gain 0.300; $2^{k-1} - 1$: 3 and 15.

STEP 1 (A)

Parent (4 Yes, 4 No): $\text{Gini} = 1 - 0.5^2 - 0.5^2 = 0.5$.

STEP 2 (B) {HIGH} VS {MEDIUM, LOW}

High: (2, 0), $\text{Gini} = 0$. Medium+Low: rows 3, 4, 5, 6, 7, 8 $\rightarrow$ (2 Yes, 4 No), $\text{Gini} = 1 - (\frac{1}{3})^2 - (\frac{2}{3})^2 = 1 - \frac{1}{9} - \frac{4}{9} = 0.444$.

Weighted $= \frac{2}{8}(0) + \frac{6}{8}(0.444) = 0.333$.

STEP 3 (B) {MEDIUM} VS {HIGH, LOW}

Medium: (2, 1), $\text{Gini} = 1 - \frac{4}{9} - \frac{1}{9} = 0.444$. High+Low: (2, 3), $\text{Gini} = 1 - 0.16 - 0.36 = 0.48$.

Weighted $= \frac{3}{8}(0.444) + \frac{5}{8}(0.48) = 0.167 + 0.300 = 0.467$.

STEP 4 (B) {LOW} VS {HIGH, MEDIUM}

Low: (0, 3), $\text{Gini} = 0$. High+Medium: (4, 1), $\text{Gini} = 1 - 0.64 - 0.04 = 0.32$.

Weighted $= \frac{3}{8}(0) + \frac{5}{8}(0.32) = 0.200$.

STEP 5 (C)

Best grouping: **{Low} vs {High, Medium}** with weighted Gini 0.200; $\text{gain} = 0.5 - 0.200 = 0.300$. It isolates the three sure refusals; the (4, 1) child would be split next (on Income again — High vs Medium — or on Credit).

STEP 6 (D)

Each value goes to one of two sides, minus the two all-on-one-side assignments, divided by two for symmetry: $\frac{2^k - 2}{2} = 2^{k-1} - 1$. For $k = 3$: 3 (as listed). For $k = 5$: 15. This is why CART's exhaustive search is affordable for small k only.

Parent 0.5; {H}{M,L} 0.333; {M}{H,L} 0.467; {L}{H,M} 0.200 (best, gain 0.3); $2^{k-1} - 1$ groupings.

WHERE MARKS ARE LOST Missing one of the three groupings; weighting by the wrong node sizes; reporting the highest weighted Gini as best.

Problem 5.5 · Where to cut "years at job"

NUMERICAL · 8 MARKS

A continuous attribute is added to the loan data — years at current job — with the class:

Years	0.5	1	2	3	4	5	7	10
Approve	No	No	No	Yes	Yes	No	Yes	Yes

- List the candidate thresholds (midpoints between consecutive sorted values).
- For each candidate compute the weighted Gini of the split "Years < c" vs "Years ≥ c".
- Which threshold is best? Compute its information gain as well.
- Why is it enough to consider only the midpoints where the class changes? Which candidates does that rule eliminate here?

WHAT IT TESTS Problem Type 5 — the scan over thresholds — with both impurity measures and the class-change shortcut.

HOW TO ATTEMPT

- Data is already sorted; midpoints of neighbours.
- Count (Yes, No) left and right; Gini each; weight by size.
- Lowest Gini; then entropy gain for that split.

ANSWERS AT A GLANCE Candidates 0.75, 1.5, 2.5, 3.5, 4.5, 6, 8.5; best $c = 2.5$ with weighted Gini 0.200 and information gain 0.549; class-change rule keeps 2.5, 4.5, 6 only.

STEP 1 (A)

Midpoints: $\frac{0.5+1}{2} = 0.75$; 1.5; 2.5; 3.5; 4.5; $\frac{5+7}{2} = 6$; $\frac{7+10}{2} = 8.5$.

STEP 2 (B)

c	Left (Yes, No)	Gini left	Right (Yes, No)	Gini right	Weighted Gini
0.75	(0, 1)	0	(4, 3)	$1 - \frac{16}{49} - \frac{9}{49} = 0.490$	$\frac{1}{8}(0) + \frac{7}{8}(0.490) = 0.429$
1.5	(0, 2)	0	(4, 2)	0.444	$\frac{6}{8}(0.444) = 0.333$
2.5	(0, 3)	0	(4, 1)	0.320	$\frac{5}{8}(0.320) = 0.200$
3.5	(1, 3)	0.375	(3, 1)	0.375	0.375
4.5	(2, 3)	0.480	(2, 1)	0.444	$\frac{5}{8}(0.48) + \frac{3}{8}(0.444) = 0.467$
6	(2, 4)	0.444	(2, 0)	0	$\frac{6}{8}(0.444) = 0.333$
8.5	(3, 4)	0.490	(1, 0)	0	$\frac{7}{8}(0.490) = 0.429$

STEP 3 (C)

Best threshold $c = 2.5$: "Years < 2.5 " (all three refused) vs "Years ≥ 2.5 " (4 Yes, 1 No), weighted Gini 0.200.

Entropy of the same split: parent $H = 1$; left (0, 3): 0; right (4, 1): $H = -0.8 \log_2 0.8 - 0.2 \log_2 0.2 = 0.258 + 0.464 = 0.722$; weighted $\frac{5}{8}(0.722) = 0.451$; gain $= 1 - 0.451 = 0.549$.

STEP 4 (D)

Moving a threshold across a boundary between two records of the *same* class only moves a record from one side to the other without changing either side's class mix in the direction of purity — the impurity can only get worse or stay level, never better than at the nearest class change. So the optimum always lies at a class change. Here the class changes between 2 (No) and 3 (Yes), between 4 (Yes) and 5 (No), and between 5 (No) and 7 (Yes): candidates 2.5, 4.5, 6. The other four (0.75, 1.5, 3.5, 8.5) can be skipped — and indeed none of them beats 2.5.

Best split at 2.5 (Gini 0.200, entropy gain 0.549); class-change rule reduces the scan to three candidates.

WHERE MARKS ARE LOST Using the data values themselves as thresholds; forgetting to weight by child size; picking the *highest* Gini.

Problem 5.6 · The ID column and gain ratio

NUMERICAL · 7 MARKS

The loan data also has an **Application ID** column (1 to 8, all different). Compare three candidate root attributes — Income, Employed and Application ID — by information gain and by gain ratio. (From Problem 5.2: Gain(Income) = 0.656, Gain(Employed) = 0.189.)

- Compute the information gain of Application ID.
- Compute the split information of each of the three attributes.
- Compute the three gain ratios and choose the root.
- Explain why gain ratio does not simply "penalise many values" but corrects a specific bias of information gain. Does gain ratio always beat the ID trap?

WHAT IT TESTS Problem Type 6 in full: gain, split information, ratio, and understanding what the ratio fixes.

HOW TO ATTEMPT

1. ID: 8 pure singleton children $\Rightarrow$ gain = $H(S) = 1$.
2. SplitInfo = entropy of the branch sizes.
3. Divide.

ANSWERS AT A GLANCE Gain(ID) = 1; SplitInfo: Income 1.561, Employed 1, ID 3; ratios: Income 0.420, Employed 0.189, ID 0.333 $\Rightarrow$ Income.

STEP 1 (A)

Application ID creates 8 branches with one record each; every child is pure ($H = 0$). Gain = $1 - 0 = 1.0$ — the maximum possible, higher than Income's 0.656.

STEP 2 (B)

Income: branch sizes 2, 3, 3 of 8: SplitInfo = $-\frac{2}{8} \log_2 \frac{2}{8} - 2 \times \frac{3}{8} \log_2 \frac{3}{8} = 0.500 + 2(0.531) = 1.561$.

Employed: sizes 4, 4: SplitInfo = $-2 \times \frac{1}{2} \log_2 \frac{1}{2} = 1$.

Application ID: 8 sizes of 1: SplitInfo = $-8 \times \frac{1}{8} \log_2 \frac{1}{8} = \log_2 8 = 3$.

STEP 3 (C)

GainRatio(Income) = $0.656/1.561 = 0.420$; GainRatio(Employed) = $0.189/1 = 0.189$; GainRatio(ID) = $1/3 = 0.333$.

Root = Income by gain ratio, although ID had the highest gain.

STEP 4 (D)

Information gain is biased toward attributes with many outcomes because fragmenting the data into many small subsets makes purity cheap — in the limit, one record per branch is pure for free, yet it has learnt nothing that applies to a new record. SplitInfo measures exactly that fragmentation (the entropy of the branch sizes), so dividing by it charges the attribute for the "information" it spent on splitting rather than on predicting. It is a correction, not a blanket penalty: an attribute with many values that *also* predicts well still wins.

It does not always defeat the trap: here ID's ratio (0.333) still beats Employed's (0.189) — with more classes or fewer useful attributes an ID can still win on ratio. The safe practice is to **remove identifiers before training** (as the slide's practice-exercise answer says: "Object ID cannot be used as a splitting feature").

Gain(ID) = 1 but ratio 0.333; Income wins with ratio 0.420; gain ratio corrects the fragmentation bias but identifiers should simply be dropped.

WHERE MARKS ARE LOST SplitInfo computed from class counts instead of branch sizes; $\log_2 8 = 3$; concluding that gain ratio "solves" the problem.

MODULE 5 · PART C

Keeping the Tree Honest

Problems 5.7–5.10: pre-pruning, post-pruning, MDL, and reading an overfitting table.

Problem 5.7 · Stop early: pre-pruning

SCENARIO · 6 MARKS

Rebuild the loan tree of Problem 5.2 under each of these pre-pruning rules, separately:

- Rule A: do not split a node whose best information gain is ≤ 0.7 .
- Rule B: do not split a node holding fewer than 4 records.
- Rule C: maximum depth 1 (only the root may be a decision node).

For each rule give the resulting tree (with leaf labels by majority vote), its training accuracy, and its accuracy on the test set of Problem 5.3(c). Comment on the trade-off.

WHAT IT TESTS Problem Type 8 — applying stopping conditions during construction and seeing their effect on training vs test accuracy.

HOW TO ATTEMPT

- Root gain 0.656; Medium-node gain 0.918. Apply each rule where it bites.
- Majority vote for any stopped node.
- Score on the 8 training rows and the 4 test rows.

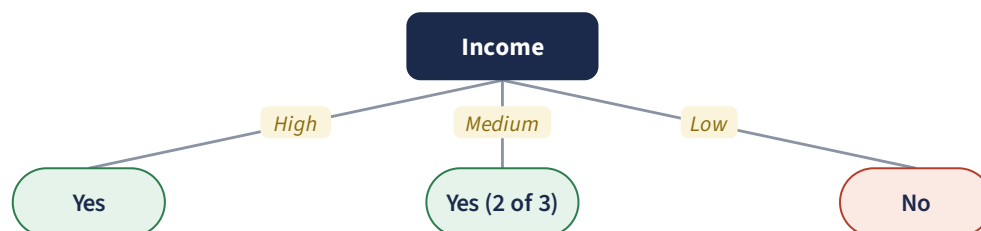
ANSWERS AT A GLANCE Rule A: root not split $\Rightarrow$ single leaf (tie 4:4), training 50%. Rules B and C: same tree — Income root, Medium $\rightarrow$ Yes (2 of 3); training $7/8 = 87.5\%$, test $3/4 = 75\%$ — equal to the full tree's test accuracy with one node fewer.

STEP 1 RULE A (GAIN ≤ 0.7)

Root: best gain is Income's $0.656 \leq 0.7 \Rightarrow$ **do not split**. The whole tree is one leaf; the training set is 4 Yes / 4 No — a tie, broken arbitrarily (say Yes). Training accuracy $4/8 = 50\%$; test accuracy with "Yes": $1/4 = 25\%$. The threshold is far too strict: severe **underfitting**.

STEP 2 RULE B (MIN 4 RECORDS)

Root has 8 records $\Rightarrow$ split on Income. Children: High (2 records), Medium (3), Low (3) — all below 4, so none may be split. High $\rightarrow$ Yes (pure), Low $\rightarrow$ No (pure), Medium $\rightarrow$ majority of (2 Yes, 1 No) = **Yes**.



The pre-pruned tree under Rules B and C.

Training: row 4 (Medium, Bad $\rightarrow$ No) is now predicted Yes — one error — accuracy $7/8 = 87.5\%$. Test: (High, Bad, No $\rightarrow$ Yes) Yes ✓; (Low, Good, Yes $\rightarrow$ No) No ✓; (Medium, Good, No $\rightarrow$ No) Yes ✗; (Low, Bad, Yes $\rightarrow$ No) No ✓ — $3/4 = 75\%$.

STEP 3 RULE C (DEPTH 1)

Only the root may test an attribute: Income, with the three children as leaves — exactly the Rule B tree. Same accuracies.

STEP 4 TRADE-OFF

The full tree scored 100% on training and 75% on test. Rules B/C gave up one training row and scored the *same* 75% on test with a simpler tree — the extra Credit split fitted a single training record and did not generalise. Rule A pruned everything and underfits badly. Choosing the pre-pruning threshold is tricky (slide): too strict $\Rightarrow$ shallow, underfit; too loose $\Rightarrow$ still overfits. The sensible choice is guided by validation accuracy.

A: one leaf, 50% train. B, C: Income-only tree, 87.5% train, 75% test — as good on test as the full tree.

WHERE MARKS ARE LOST Forgetting the majority label of a stopped node; applying the record-count rule to the root; not reporting both accuracies.

Problem 5.8 · Cut back: post-pruning with a prune set

NUMERICAL · 7 MARKS

Take the fully grown loan tree of Problem 5.2 and this prune (validation) set:

Income	Credit	Employed	Approve?
High	Bad	No	Yes
Medium	Bad	Yes	Yes
Medium	Good	No	Yes
Low	Good	Yes	No

- Compute the prune-set error of the full tree.
- Apply reduced-error pruning bottom-up: for the Credit node, compare the error with the node retained versus replaced by a majority leaf, and decide.
- Could the root itself be pruned? Test it.
- State the final tree and explain in one sentence why post-pruning is usually preferred to pre-pruning.

WHAT IT TESTS Problem Type 9 — the reduced-error pruning procedure with a decision at each node.

HOW TO ATTEMPT

- Predict each prune row with the full tree; count errors.
- Replace Credit by "Yes" (majority of its 2 Yes / 1 No training rows); re-predict; compare.
- Replace the root by the overall majority (4:4 tie); re-predict.

ANSWERS AT A GLANCE Full tree: 1 error. Pruning Credit $\rightarrow$ Yes: 0 errors $\Rightarrow$ prune. Pruning the root: 2 or 3 errors $\Rightarrow$ retain. Final tree: Income root with leaves Yes / Yes / No.

STEP 1 (A)

Prune row	True	Full tree	Credit pruned (Medium $\rightarrow$ Yes)	Root pruned (all $\rightarrow$ Yes)
High, Bad, No	Yes	Yes ✓	Yes ✓	Yes ✓
Medium, Bad, Yes	Yes	No ✗	Yes ✓	Yes ✓
Medium, Good, No	Yes	Yes ✓	Yes ✓	Yes ✓
Low, Good, Yes	No	No ✓	No ✓	Yes ✗
Errors		1 / 4	0 / 4	1 / 4 (with "Yes"; 3/4 with "No")

Full-tree error rate = $1/4 = 25\%$ (the Medium/Bad applicant is refused by the Credit rule but was approved).

STEP 2 (B)

Bottom-up, the only internal node below the root is Credit. Retained: 1 error. Replaced by a leaf labelled with the majority of its training records (2 Yes, 1 No $\rightarrow$ Yes): the Medium/Bad row is now approved — 0 errors. Error fell $\Rightarrow$ **prune the Credit node**.

STEP 3 (C)

Replace the root by a single leaf. Its training records are 4 Yes / 4 No — a tie; with label Yes the prune-set error is $1/4$ (the Low applicant), with label No it is $3/4$. Either way the error is *higher* than the pruned tree's $0/4 \Rightarrow$ **retain the root**.

STEP 4 (D)

Final tree: Income $\rightarrow$ High: Yes; Medium: Yes; Low: No. Post-pruning is preferred because it lets the tree grow first and then removes only the branches that a validation set proves useless, whereas pre-pruning must guess a threshold in advance and can stop before a useful combination of splits appears.

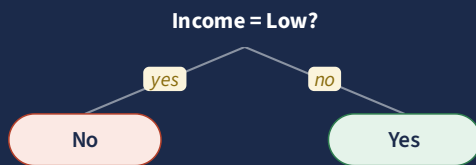
Prune Credit (error $1/4 \rightarrow 0/4$); keep the root; final tree Income $\rightarrow$ Yes / Yes / No.

WHERE MARKS ARE LOST Labelling the pruned node by the prune-set majority instead of the training majority; pruning top-down; not testing the root.

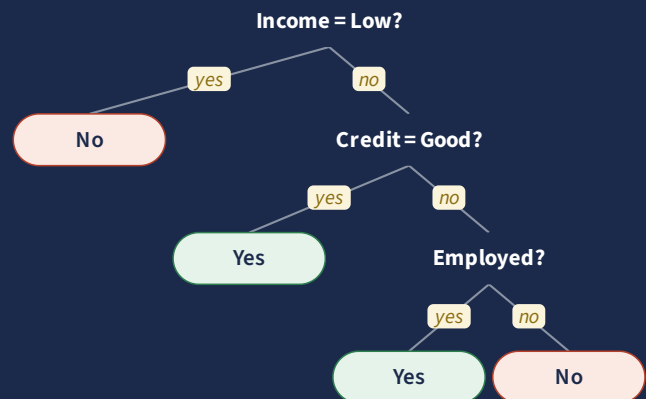
Problem 5.9 · Count the bits: MDL

NUMERICAL · 7 MARKS

Two trees are proposed for a loan dataset with n training records, **8 binary attributes** and **2 classes**. Tree (a): a single binary test (Income = Low?) with two leaves; it makes 3 training errors. Tree (b): three binary tests and four leaves; it makes no training errors.



(a): 1 internal node, 2 leaves, 3 errors.



(b): 3 internal nodes (binary tests), 4 leaves, 0 errors.

- State the cost, in bits, of an internal node, of a leaf, and of one misclassified record.
- Write the total description length of each tree as a function of n .
- For $n = 8$, which tree does MDL prefer? For which values of n does the answer flip?
- Explain in one sentence what the MDL principle trades off, and how it relates to Occam's razor.

WHAT IT TESTS Problem Type 10 with the standard cost conventions, including the "which n " reasoning.

HOW TO ATTEMPT

- $\log_2 8 = 3$ per node; $\log_2 2 = 1$ per leaf; $\log_2 n$ per error.
- Nodes $\times 3$ + leaves $\times 1$ + errors $\times \log_2 n$.
- Compare; solve the inequality.

ANSWERS AT A GLANCE Node 3 bits, leaf 1 bit, error $\log_2 n$; $L(a) = 5 + 3 \log_2 n$, $L(b) = 13$; at $n = 8$: $14 > 13 \Rightarrow (b)$; (a) wins only for $n \leq 6$.

STEP 1 (A)

Internal node: must name one of 8 attributes $\Rightarrow \log_2 8 = 3$ bits. Leaf: one of 2 classes $\Rightarrow \log_2 2 = 1$ bit. Error: identify which of the n records is an exception $\Rightarrow \log_2 n$ bits each.

STEP 2 (B)

$$L(a) = 1 \times 3 + 2 \times 1 + 3 \log_2 n = 5 + 3 \log_2 n.$$

$$L(b) = 3 \times 3 + 4 \times 1 + 0 \times \log_2 n = 13.$$

STEP 3 (C)

$n = 8$: $\log_2 8 = 3$, so $L(a) = 5 + 9 = 14$ bits vs $L(b) = 13$ bits $\Rightarrow$ **tree (b)** is preferred — listing three exceptions costs more than the extra structure.

(a) is preferred when $5 + 3 \log_2 n < 13 \iff \log_2 n < \frac{8}{3} \iff n < 2^{8/3} \approx 6.35$, i.e. for $n \leq 6$. With more data each exception is more expensive to name, so the bigger, error-free tree becomes worth its bits.

STEP 4 (D)

MDL trades off hypothesis complexity (bits to describe the tree) against the number of training errors (bits to describe the exceptions), choosing the tree with the shortest total — a quantitative form of Occam's razor: prefer the shortest hypothesis, but allow a longer one when it saves more bits in errors than it costs in structure.

$L(a) = 5 + 3 \log_2 n$, $L(b) = 13$; at $n = 8$ choose (b); (a) only for $n \leq 6$.

WHERE MARKS ARE LOST $\log_2 8 = 4$; charging errors for tree (b) (it has none); forgetting the leaves' bits.

Problem 5.10 · Reading an overfitting table

SCENARIO · 6 MARKS

A decision tree was grown on 1,000 credit applications and evaluated on 9,000 held-out ones, stopping at various sizes:

Nodes	1	3	7	15	40	120	300
Training error	0.45	0.28	0.17	0.12	0.06	0.02	0.00
Test error	0.46	0.30	0.19	0.15	0.17	0.23	0.31

- Which tree size should be chosen? Identify the underfitting and overfitting regions.
- Give three causes of overfitting in decision trees from the slides and, for each, one remedy.
- The 300-node tree has zero training error. Explain, using the PlayTennis noisy-example argument, why that is a warning rather than an achievement.

WHAT IT TESTS Problem Type 7 — model comparison by test error, and the causes/remedies vocabulary.

HOW TO ATTEMPT

- Minimum test error.
- Noise/outliers, lack of representative data, irrelevant attributes chosen by chance $\rightarrow$ pruning, more data, min-samples/depth.
- A noisy example forces an extra split that fits noise.

ANSWERS AT A GLANCE 15 nodes (test 0.15); underfitting below, overfitting above; causes noise, too little data at deep nodes, spurious attributes; zero training error means the noise has been memorised.

STEP 1 (A)

Test error falls from 0.46 to a minimum of 0.15 at **15 nodes**, then rises to 0.31. Choose 15 nodes. Sizes 1–7: both errors high and close — **underfitting**. Sizes 40–300: training error keeps falling toward zero while test error climbs — **overfitting**; the gap widens from 0.03 to 0.31.

STEP 2 (B)

Noise and outliers — branches grow to fit anomalous records → remedy: post-pruning with a validation set, or a minimum information-gain threshold.

Too little data in deep nodes — a leaf built on two records is a guess → remedy: minimum records per leaf / maximum depth, or collect more data.

Irrelevant attributes chosen by chance — with many attributes some will correlate with the class accidentally in a small subset → remedy: feature selection beforehand, gain-ratio or pruning, and (Module 9) ensembles that average many trees.

STEP 3 (C)

Add one contradictory day to PlayTennis — $\langle \text{Sunny, Normal, Strong} \rangle \rightarrow \text{No}$. The tree previously said Yes for Sunny/Normal; to fit the new example it splits that leaf on Wind. The tree is now "perfect" on training data, but that split encodes one noisy day, and every future Sunny/Normal/Strong day will be misclassified. Zero training error on 1,000 applications with 300 nodes (about 150 leaves — six or seven applicants per leaf) means the tree has memorised individual cases, including the erroneous ones, which is exactly why its test error is the worst of all.

Choose 15 nodes; underfit ≤ 7 , overfit ≥ 40 ; three causes with remedies; zero training error = memorised noise.

WHERE MARKS ARE LOST Choosing 300 nodes; describing overfitting without the widening gap; giving remedies not tied to causes.

MODULE 5 · PART D

Structure and Judgement

Problems 5.11–5.14: rules and algorithms, test conditions, multi-class impurity, and when a tree is the right tool.

Problem 5.11 · Trees as rules, rules as trees

CONCEPTUAL · 6 MARKS

- State Hunt's algorithm and the three standard stopping conditions.
- Convert this tree into rules: root *Outlook*; Sunny $\rightarrow$ *Humidity* (High: No, Normal: Yes); Overcast $\rightarrow$ Yes; Rain $\rightarrow$ *Wind* (Strong: No, Weak: Yes). Express "PlayTennis = Yes" as a disjunction of conjunctions.
- Classify $\langle \text{Rain, Cool, High, Weak} \rangle$ and $\langle \text{Sunny, Hot, Normal, Strong} \rangle$ by the rules. Which attribute is never used?
- ID3 is described as "greedy". What does that mean here, and what is its main consequence?

WHAT IT TESTS The construction algorithm, the rule representation, and the word "greedy".

HOW TO ATTEMPT

- Pure $\rightarrow$ leaf; else best test, children, recurse. Stop: pure / identical attributes / no records.
- One conjunction per root-to-Yes path; OR them.
- Follow the paths.
- Greedy = locally best split, never revisited.

ANSWERS AT A GLANCE (b) $(\text{Sunny} \wedge \text{Normal}) \vee \text{Overcast} \vee (\text{Rain} \wedge \text{Weak})$. (c) Yes; Yes; Temperature unused. (d) Greedy = best immediate gain, no backtracking $\Rightarrow$ may miss the smallest tree.

STEP 1 (A)

Hunt's algorithm: at a node with records D_t — if all records belong to one class, make a leaf with that class; otherwise choose the best attribute test condition, create a child for each outcome, distribute the records, and apply the same procedure to each child.

Stopping conditions: (1) all records at the node have the same class; (2) all records have identical attribute values (nothing left to split on) — label with the majority class; (3) no records reach the node — label with the parent's majority class. Plus optional early-stopping constraints: maximum depth, minimum records per node, minimum gain.

STEP 2 (B)

Paths ending in Yes: Sunny $\wedge$ Humidity = Normal; Overcast; Rain $\wedge$ Wind = Weak.

PlayTennis = Yes $\iff (\text{Outlook} = \text{Sunny} \wedge \text{Humidity} = \text{Normal}) \vee (\text{Outlook} = \text{Overcast}) \vee (\text{Outlook} = \text{Rain} \wedge \text{Wind} = \text{Weak})$.

STEP 3 (C)

$\langle \text{Rain, Cool, High, Weak} \rangle$: Outlook = Rain $\rightarrow$ Wind = Weak $\rightarrow$ **Yes** (third rule).

$\langle \text{Sunny, Hot, Normal, Strong} \rangle$: Sunny $\rightarrow$ Humidity = Normal $\rightarrow$ **Yes** (first rule). **Temperature** is never tested — its information gain was lowest at every node where it was a candidate.

STEP 4 (D)

Greedy: at each node ID3 picks the attribute with the highest gain *right now* and never reconsiders that choice. Consequence: the space of possible trees is exponentially large and the greedy path may miss the globally smallest or most accurate tree (an attribute with low gain at the root might have enabled a much simpler tree below). Greedy search is used because exhaustive search is infeasible; pruning and ensembles compensate.

Hunt's recursion with three stops; three-rule disjunction; both days Yes; Temperature unused; greedy = locally optimal, no backtracking.

WHERE MARKS ARE LOST Writing rules for No as well when only Yes was asked; misreading a path; defining greedy as "fast".

Problem 5.12 · Test conditions for every attribute type

SCENARIO · 5 MARKS

A dataset has: **Owns car** (Yes/No), **City** (Delhi/Mumbai/Pune/Chennai), **Education** (School < Graduate < Postgraduate), **Age** (continuous).

- For each attribute, give the form(s) of test condition a decision tree can use, with one concrete example.
- How many binary groupings does City admit? Which of these groupings of Education is *not* allowed, and why: {School, Postgraduate} vs {Graduate}?
- Give one multi-way and one binary test for Age and explain how the cut points would be chosen.

WHAT IT TESTS The "methods for expressing attribute test conditions" slides.

HOW TO ATTEMPT

- Binary; nominal multi-way or grouped; ordinal grouped in order; continuous threshold or bins.
- $2^k - 1$; ordinal groups must be contiguous.
- Threshold at a class-change midpoint; bins by ranges.

ANSWERS AT A GLANCE Binary / multi-way or grouped / contiguous groups / threshold or bins; City has 7 groupings; {School, Postgraduate} vs {Graduate} breaks the order; Age < 30 or bins {< 25, 25–40, > 40}.

STEP 1 (A)

Owns car (binary): one test with two outcomes — Yes / No.

City (nominal, 4 values): a **multi-way** split with one branch per city; or a **binary** split by grouping, e.g. {Delhi, Mumbai} vs {Pune, Chennai} — CART requires binary.

Education (ordinal): multi-way (one branch per level) or binary groupings that **respect the order**, e.g. {School} vs {Graduate, Postgraduate} or {School, Graduate} vs {Postgraduate}.

Age (continuous): a binary threshold test, e.g. Age < 30 vs ≥ 30 ; or a multi-way split into ranges.

STEP 2 (B)

City: $2^{4-1} - 1 = 7$ binary groupings. Education: {School, Postgraduate} vs {Graduate} puts the two ends together and the middle alone — it violates the ordering (an ordinal split must be a cut at one point of the order), so it is not allowed; only the two contiguous cuts are.

STEP 3 (C)

Binary: Age < 30 . The threshold is chosen by sorting the training ages, taking midpoints between neighbours (at least where the class changes), and keeping the one with the lowest weighted impurity. Multi-way: $\{< 25, 25-40, > 40\}$ — bins chosen by equal width, equal depth, or by the same impurity scan applied to several cut points at once. The tree treats each bin as a nominal value afterwards.

Test forms as listed; 7 groupings for City; non-contiguous ordinal grouping disallowed; Age via threshold scan or binning.

WHERE MARKS ARE LOST Allowing arbitrary groupings for an ordinal attribute; 2^k groupings; picking an Age threshold without saying how.

Problem 5.13 · Three classes in one node

NUMERICAL · 6 MARKS

A node holds loan applications of three classes: Approve = 4, Refer = 2, Refuse = 2.

- Compute the node's entropy and Gini index. What are the maximum possible values for three classes, and is this node at them?
- A candidate split sends the 4 Approve records to one child and the 2 Refer + 2 Refuse records to the other. Compute the information gain and the Gini gain.
- Another split produces children (2, 1, 1) and (2, 1, 1). What is its gain? What does that tell you?

WHAT IT TESTS Impurity with more than two classes — the " $0 \leq \text{entropy} \leq \log_2 k$ " remark — and gain with three classes.

HOW TO ATTEMPT

- $p = (0.5, 0.25, 0.25)$; $-\sum p \log_2 p$; $1 - \sum p^2$; maxima $\log_2 3$ and $2/3$.
- Child impurities, weighted, subtract.
- Identical proportions $\Rightarrow$ zero gain.

ANSWERS AT A GLANCE $H = 1.5$ bits, Gini 0.625; maxima 1.585 and 0.667 — not reached. Split (a): gain 1.0 bits, Gini gain 0.375. Split (b): gain 0.

STEP 1 (A)

$$p = \left(\frac{4}{8}, \frac{2}{8}, \frac{2}{8}\right) = (0.5, 0.25, 0.25).$$

$$H = -0.5 \log_2 0.5 - 0.25 \log_2 0.25 - 0.25 \log_2 0.25 = 0.5 + 0.5 + 0.5 = 1.5 \text{ bits.}$$

$$\text{Gini} = 1 - 0.25 - 0.0625 - 0.0625 = 0.625.$$

Maxima for $C = 3$ (all classes equal, $p = \frac{1}{3}$ each): $H_{\max} = \log_2 3 = 1.585$; $\text{Gini}_{\max} = 1 - 3(\frac{1}{9}) = 0.667$. The node is close to, but below, both maxima — Approve is a mild majority.

STEP 2 (B)

Child 1: (4, 0, 0) — pure, $H = 0$, Gini = 0. Child 2: (0, 2, 2) — $H = -2 \times 0.5 \log_2 0.5 = 1$; Gini = $1 - 0.25 - 0.25 = 0.5$.

Weighted entropy = $\frac{4}{8}(0) + \frac{4}{8}(1) = 0.5$; information gain = $1.5 - 0.5 = \mathbf{1.0}$ bit.

Weighted Gini = $\frac{4}{8}(0) + \frac{4}{8}(0.5) = 0.25$; Gini gain = $0.625 - 0.25 = \mathbf{0.375}$.

STEP 3 (C)

Each child (2, 1, 1) has the same proportions (0.5, 0.25, 0.25) as the parent, so its entropy is 1.5 and the weighted entropy is 1.5: gain = 0 (Gini likewise: $0.625 \rightarrow 0.625$). A split that merely divides the records without changing the class mix carries no information — the attribute is independent of the class on this node and should not be chosen.

$H = 1.5$, Gini 0.625 (maxima 1.585, 0.667); split (b) gains 1.0 bit / 0.375; the proportional split gains nothing.

WHERE MARKS ARE LOST Capping entropy at 1 for three classes; $\log_2 0.25 = -1$; forgetting to weight the children.

Problem 5.14 · When to pick a tree

CONCEPTUAL · 6 MARKS

- List three advantages and three disadvantages of decision trees from the slides.
- A dataset's two classes are separated by the diagonal line $x_1 = x_2$. Sketch (describe) what a decision tree's boundary looks like and why; how would logistic regression fare on the same data?
- A bank must give every rejected applicant a written reason. Which of logistic regression and a decision tree would you deploy, and what would you do about the tree's tendency to overfit?
- Name two application areas where trees are well suited, with the property of the data that makes them so.

WHAT IT TESTS The advantages/disadvantages slide, the axis-parallel limitation, and the interpretability trade-off.

HOW TO ATTEMPT

- Reproduce the slide's lists.
- Trees test one attribute at a time $\Rightarrow$ staircase.
- Tree + pruning (or a constrained depth).
- Attribute–value data with discrete targets and possibly missing values.

ANSWERS AT A GLANCE (b) A staircase of axis-parallel steps needing many splits; logistic regression fits the diagonal with one line. (c) A pruned tree. (d) Medical diagnosis, credit scoring, manufacturing, make-or-buy.

STEP 1 (A)

Advantages: inexpensive to construct; extremely fast at classifying unknown records; easy to interpret for small trees; handle redundant/irrelevant attributes (they are simply not chosen) and missing values; no feature scaling needed.

Disadvantages: the space of possible trees is exponentially large and greedy growth is often unable to find the best tree; attribute interactions are not taken into account; each decision boundary involves a single attribute (axis-parallel); prone to overfitting without pruning; small changes in data can change the tree a lot (high variance).

STEP 2 (B)

Every test is of the form " $x_1 < c$ " or " $x_2 < c$ ", so every boundary segment is horizontal or vertical. To approximate the diagonal $x_1 = x_2$ the tree must build a **staircase** of many small axis-parallel steps — many nodes, each fitted to a few points — and it still cannot be exact. Logistic regression's boundary is $\theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0$, a line of any orientation: it captures $x_1 = x_2$ with three parameters ($\theta = (0, 1, -1)$) and generalises far better here.

STEP 3 (C)

Deploy the **decision tree**: a rejection is explained by the path — "income below 80K and no home ownership" — which a customer and a regulator can read; a logistic score of 0.42 is not a reason. To control overfitting: grow the tree fully and **post-prune** with a validation set (or constrain depth / minimum records per leaf), keeping the tree small enough to remain readable; in Module 9, ensembles improve accuracy but sacrifice the single readable path.

STEP 4 (D)

Trees suit problems whose instances are attribute–value pairs with a discrete target, where disjunctive rules may be needed and data may have missing values: **medical diagnosis** (symptoms → disease, with explainable rules), **credit scoring / banking** (regulated decisions), **manufacturing** (quality control from sensor and process attributes), **make-or-buy decisions**.

Lists as above; staircase vs one diagonal line; a pruned tree for explainability; diagnosis, credit, manufacturing, make-or-buy.

WHERE MARKS ARE LOST Listing only advantages; saying trees "can't handle continuous data"; deploying logistic regression despite the explainability requirement.